

signus

Blind PSK/QAM/FSK/Chirp Demodulator — 필사용 코드북

실행에 필요한 코드 전량 · 22개 파일 · 3,786줄 · 총 77쪽 | 2026-07-26 · c6a9bc6+수정중

한 줄이 100칸을 넘으면 왼쪽 줄번호 자리에 ↵ 표시가 붙고 아랫줄로 이어집니다 — 원래는 한 줄이니 붙여서 입력하세요. 세로 점선은 들여쓰기 4칸 눈금입니다.

0 · 프로젝트 설정

□ pyproject.toml	패키지 정의 · 의존성 · ruff/pytest 설정	38줄	2쪽
□ signus/__init__.py	패키지 진입점	3줄	3쪽

1 · 신호 입출력과 기준표

□ signus/sigio.py	샘플 파일 I/O — 파일명이 fs·iq·real·샘플타임을 실어 나른다	145줄	4쪽
□ signus/constellations.py	성상도 · 그레이 비트 매핑 · FSK 레벨 (공용 기준표)	99줄	7쪽
□ signus/gen.py	합성 신호 생성기 — 테스트 하네스의 정답지	196줄	9쪽

2 · 광대역 탐지와 채널 추출

□ signus/spectrum.py	스펙트럼 · 위터폴 뷰 (표시 전용, 탐지엔 미사용)	36줄	13쪽
□ signus/detect.py	광대역 탐지 — 캡처 안의 모든 방사체를 주파수/시간 상자로	107줄	14쪽
□ signus/channelize.py	채널 추출 — 광대역에서 방사체 하나만 기저대역으로 끌어냄	32줄	16쪽
□ signus/triage.py	채널 트리아지 — 복조 가능한 디지털 신호인지 판별	66줄	17쪽

3 · 수신 DSP 코어

□ signus/dsp.py	수신 DSP 단계 (벡터화) — 이 프로젝트의 심장	316줄	19쪽
□ signus/eq.py	블라인드 심볼간격 FIR 등화기 — CMA 획득 후 판정지향 LMS	44줄	25쪽
□ signus/sync.py	블라인드 반복 프리앰블 동기 (정의 불요)	111줄	26쪽
□ signus/classify.py	결정층 — 변조 분류 · 회전 정렬 · 락 품질 · SNR	123줄	28쪽
□ signus/_accel.py	선택적 numba 가속 (없으면 동일한 순수 numpy로 풀백)	100줄	31쪽

4 · 변조별 수신 경로

□ signus/fsk.py	FSK/CPM 경로 — 정포락선 게이트 + 주파수 판별기 복조	181줄	33쪽
□ signus/chirp.py	선형 처프 / CSS(LoRa) 블라인드 탐지 · 특성화	97줄	37쪽

5 · 종단 파이프라인

□ signus/pipeline.py	블라인드 복조 종단 조립 — 두 계열의 진입점	496줄	39쪽
----------------------	---------------------------	------	-----

6 · 사용자 인터페이스

□ signus/cli.py	CLI — analyze / survey / gen / dataset / sweep / serve	406줄	48쪽
□ signus/server.py	표준 라이브러리만 쓰는 웹 서버 + POST /api/analyze	89줄	55쪽

7 · 웹 프론트엔드

□ signus/web/index.html	UI 구조	174줄	57쪽
□ signus/web/style.css	UI 스타일	243줄	61쪽
□ signus/web/app.js	업로드 · 분석 요청 · 성상도/위터폴 캔버스 렌더링	684줄	66쪽

```
1  [build-system]
2  requires = ["setuptools>=68"]
3  build-backend = "setuptools.build_meta"
4
5  [project]
6  name = "signus"
7  version = "2.0.0"
8  description = "Blind PSK/QAM demodulator: auto parameter detection + constellation UI"
9  requires-python = ">=3.11"
10 dependencies = ["numpy>=1.26", "scipy>=1.11"]
11
12 [project.optional-dependencies]
13 dev = ["pytest>=8.0", "ruff>=0.4"]
14 # Optional JIT for the sequential equalizer/carrier loops (16-34x on those loops).
15 # Falls back to identical pure-numpy when absent. NOTE: numba pins numpy<2.5.
16 fast = ["numba>=0.60"]
17
18 [project.scripts]
19 signus = "signus.cli:main"
20
21 [tool.setuptools.packages.find]
22 include = ["signus*"]
23
24 [tool.setuptools.package-data]
25 signus = ["web/*"]  # the served frontend: without this a pip install 404s the whole UI
26
27 [tool.pytest.ini_options]
28 pythonpath = ["."]
29 testpaths = ["tests"]
30 addopts = "-q"
31 markers = ["stretch: slow/marginal cases, opt-in"]
32
33 [tool.ruff]
34 target-version = "py311"
35 line-length = 100
36
37 [tool.ruff.lint]
38 select = ["E", "F", "I", "UP", "B", "SIM", "NPY"]
```

```
1 """signus – blind signal demodulator with automatic parameter detection."""
2
3 __version__ = "2.0.0"
```

```

1  """Sample file I/O. Filename carries read-necessities ONLY (fs, iq|real, sample
2  type, endian, bitrev); ground truth lives in a `<filename>.json` sidecar the
3  analyzer never reads. SigMF sidecars (`<stem>.sigmf-meta`) are also understood."""
4
5  import json
6  import os
7  import re
8  from dataclasses import dataclass
9
10 import numpy as np
11
12 _FS_TOK = re.compile(r"(?:^|_)fs(?:\d+(?:\.\d+)?(?:e[+-]?\d+)?)(?:_|$)", re.I)
13 _RF_TOK = re.compile(r"(?:^|_)rf(?:\d+(?:\.\d+)?(?:e[+-]?\d+)?)(?:_|$)", re.I)
14
15 # token -> (numpy code, full-scale divisor, offset). u-types are offset binary
16 # (e.g. 80 / RTL-SDR u8: 0..255 with 128 = zero).
17 _DTYPES = {
18     "i8": ("i1", 128.0, 0.0), "u8": ("u1", 128.0, 128.0),
19     "i16": ("i2", 32768.0, 0.0), "u16": ("u2", 32768.0, 32768.0),
20     "f32": ("f4", 1.0, 0.0), "f64": ("f8", 1.0, 0.0),
21 }
22 # baudline-style aliases: <bits>t = two's complement, <bits>o = offset binary
23 _ALIAS = {"8t": "i8", "8o": "u8", "16t": "i16", "16o": "u16", "32f": "f32", "64f": "f64"}
24 _BITREV = np.array([int(f"{i:08b}"[::-1]), 2) for i in range(256)], dtype=np.uint8)
25
26 # SigMF core:datatype -> (dtype token, fmt, endian)
27 _SIGMF = {"cf32": ("f32", "iq"), "ci16": ("i16", "iq"), "ci8": ("i8", "iq"),
28           "cu8": ("u8", "iq"), "rf32": ("f32", "real"), "ri16": ("i16", "real")}
29
30
31 @dataclass
32 class Meta:
33     fs: float | None = None
34     fmt: str | None = None # 'iq' | 'real'
35     dtype: str = "i16" # key of _DTYPES
36     endian: str = "le" # 'le' | 'be' (floats included)
37     bitrev: bool = False # bit order reversed within each byte
38     rf_center: float | None = None # RF centre freq (Hz); None = unknown (report baseband)
39
40     def ok(self) -> bool:
41         return self.fs is not None and self.fmt in ("iq", "real") and self.dtype in _DTYPES
42
43
44 def parse_name(name: str) -> Meta:
45     """Extract fs/fmt/dtype/endian/bitrev tokens from a filename; rest ignored."""
46     stem = os.path.splitext(os.path.basename(name))[0].lower()
47     m = Meta()
48     if mt := _FS_TOK.search(stem):
49         m.fs = float(mt.group(1))
50     if rt := _RF_TOK.search(stem):
51         m.rf_center = float(rt.group(1))
52     toks = stem.split("_")
53     m.fmt = next((t for t in toks if t in ("iq", "real")), None)
54     m.dtype = next((_ALIAS.get(t, t) for t in toks if t in _DTYPES or t in _ALIAS), "i16")
55     m.endian = "be" if "be" in toks else "le"
56     m.bitrev = "bitrev" in toks
57     return m
58
59
60 def parse_sigmf(path: str) -> Meta | None:

```

```

61     """Read `stem>.sigmf-meta` (core:sample_rate + core:datatype) if present."""
62     try:
63         with open(os.path.splitext(path)[0] + ".sigmf-meta") as fh:
64             doc = json.load(fh)
65             g = doc["global"]
66             code = g["core:datatype"].replace("_le", "").replace("_be", "")
67             dtype, fmt = _SIGMF[code]
68             endian = "be" if g["core:datatype"].endswith("_be") else "le"
69             fs = float(g["core:sample_rate"])
70     except (OSError, KeyError, ValueError, TypeError, AttributeError):
71         return None # MANDATORY fields malformed: fall back to filename tokens
72     try:
73         # rf centre is OPTIONAL: a malformed value must cost only this field, never the whole
74         # sidecar -- discarding valid fs/fmt/dtype here silently re-read the file with filename
75         # tokens, which can lie about fs (a quiet garbage decode).
76         caps = doc.get("captures") or []
77         rf = caps[0].get("core:frequency") if caps else None
78         rf = None if rf is None else float(rf)
79     except (KeyError, ValueError, TypeError, AttributeError, IndexError):
80         rf = None
81     return Meta(fs, fmt, dtype, endian, rf_center=rf)
82
83
84 def make_name(label: str, m: Meta, ext: str) -> str:
85     fs_tok = str(int(m.fs)) if m.fs == int(m.fs) else f"{m.fs:f}".rstrip("0").rstrip(".")
86     extra = ("_be" if m.endian == "be" else "") + ("_bitrev" if m.bitrev else "")
87     return f"{label}_fs{fs_tok}_{m.fmt}_{m.dtype}{extra}.{ext}"
88
89
90 def decode(raw: bytes, meta: Meta) -> np.ndarray:
91     """Raw bytes -> samples: complex128 for 'iq', float64 for 'real'.
92     Tolerates truncated files (drops trailing partial samples)."""
93     code, scale, off = _DTYPES[meta.dtype]
94     if meta.bitrev:
95         raw = _BITREV[np.frombuffer(raw, dtype=np.uint8)].tobytes()
96     step = np.dtype(code).itemsize
97     dt = ("<" if meta.endian == "le" else ">") + code
98     x = np.frombuffer(raw[: len(raw) - len(raw) % step], dtype=dt).astype(np.float64)
99     x = (x - off) / scale
100    if meta.fmt == "iq":
101        x = x[: x.size & ~1] # drop dangling half-sample
102        return x[0::2] + 1j * x[1::2]
103    return x
104
105
106 def read(path: str, meta: Meta | None = None) -> tuple[np.ndarray, Meta]:
107     meta = meta or parse_sigmf(path) or parse_name(path)
108     if not meta.ok():
109         raise ValueError(f"need fs + fmt (filename tokens, SigMF, or --fs/--fmt): {path}")
110     with open(path, "rb") as fh:
111         x = decode(fh.read(), meta)
112     if x.size < 256:
113         raise ValueError(f"신호가 너무 짧습니다 ({x.size} samples): {path}")
114     return x, meta
115
116
117 def write(path: str, x: np.ndarray, meta: Meta) -> None:
118     """Quantize (ints: scaled to 99.9th-percentile full scale) and write interleaved."""
119     code, scale, off = _DTYPES[meta.dtype]
120     flat = np.column_stack([x.real, x.imag]).ravel() if np.iscomplexobj(x) else x.real

```

```

121     if scale != 1.0: # integer types
122         peak = np.percentile(np.abs(flat), 99.9) or 1.0
123         flat = np.round(flat * (scale - 1) / peak) + off
124         info = np.iinfo(code)
125         flat = np.clip(flat, info.min, info.max)
126     out = flat.astype("<" if meta.endian == "le" else ">") + code)
127     if meta.bitrev:
128         out = np.frombuffer(_BITREV[np.frombuffer(out.tobytes(), np.uint8)].tobytes(),
129                             dtype=out.dtype)
130     out.tofile(path)
131
132
133 def sidecar_write(path: str, meta: Meta, truth: dict, gen: dict) -> None:
134     doc = {"fs": meta.fs, "fmt": meta.fmt, "dtype": meta.dtype,
135           "endian": meta.endian, "bitrev": meta.bitrev, "truth": truth, "gen": gen}
136     with open(path + ".json", "w") as fh:
137         json.dump(doc, fh, indent=1)
138
139
140 def sidecar_read(path: str) -> dict | None:
141     try:
142         with open(path + ".json") as fh:
143             return json.load(fh)
144     except (OSError, ValueError):
145         return None

```

```

1  """Constellations, Gray labels, differential tables, FSK levels (shared reference)."""
2
3  import numpy as np
4
5  MODS = ("bpsk", "qpsk", "8psk", "16qam", "64qam")          # blind-classify targets
6  FSK_MODS = ("fsk2", "fsk4", "msk")
7  DIFF_MODS = ("dbpsk", "dqpsk", "pi4dqpsk")
8  GEN_MODS = MODS + ("32qam", "oqpsk") + DIFF_MODS + FSK_MODS # generator vocabulary
9
10 # (data-alphabet order, constellation rotational symmetry). pi4dqpsk: 4 transition
11 # values per symbol but an 8-point composite constellation.
12 _INFO = {"bpsk": (2, 2), "qpsk": (4, 4), "8psk": (8, 8), "16qam": (16, 4),
13          "64qam": (64, 4), "32qam": (32, 4), "oqpsk": (4, 4), "pi4dqpsk": (4, 8),
14          "dbpsk": (2, 2), "dqpsk": (4, 4), "fsk2": (2, 0), "fsk4": (4, 0), "msk": (2, 0)}
15
16 # differential phase increment per Gray symbol value
17 DIFF_PHASES = {
18     "dbpsk": np.array([0.0, np.pi]),
19     "dqpsk": np.array([0.0, np.pi / 2, np.pi, -np.pi / 2]),
20     "pi4dqpsk": np.array([np.pi / 4, 3 * np.pi / 4, -3 * np.pi / 4, -np.pi / 4]),
21 }
22
23
24 def family(mod: str) -> str:
25     return "fsk" if mod in FSK_MODS else "linear"
26
27
28 def mod_order(mod: str) -> int:
29     return _INFO[mod][0]
30
31
32 def mod_symmetry(mod: str) -> int:
33     """Lowest power p for which x**p strips the modulation (square QAM: 4)."""
34     return _INFO[mod][1]
35
36
37 def ideal_points(mod: str) -> np.ndarray:
38     if mod in ("bpsk", "dbpsk"):
39         return np.array([1.0 + 0j, -1.0 + 0j])
40     if mod in ("qpsk", "oqpsk", "dqpsk"):
41         return np.exp(1j * (np.pi / 4 + np.arange(4) * np.pi / 2))
42     if mod in ("8psk", "pi4dqpsk"):
43         return np.exp(2j * np.pi * np.arange(8) / 8)
44     if mod == "32qam": # 6x6 cross: grid minus the four corners
45         lv = np.arange(6) * 2.0 - 5
46         i, q = np.meshgrid(lv, lv)
47         pts = (i + 1j * q).ravel()
48         pts = pts[np.abs(pts.real * pts.imag) != 25]
49         return pts / np.sqrt(np.mean(np.abs(pts) ** 2))
50     n = 4 if mod == "16qam" else 8
51     lv = np.arange(n) * 2.0 - (n - 1)
52     i, q = np.meshgrid(lv, lv)
53     pts = (i + 1j * q).ravel()
54     return pts / np.sqrt(np.mean(np.abs(pts) ** 2))
55
56
57 def _gray(k: np.ndarray) -> np.ndarray:
58     return k ^ (k >> 1)
59
60

```

```

61 def bit_labels(mod: str) -> np.ndarray:
62     """Gray label per ideal_points index (PSK: phase Gray; square QAM: per-axis).
63     32qam uses plain index labels (cross grid has no clean per-axis Gray)."""
64     m = mod_order(mod)
65     k = np.arange(m)
66     if mod in ("16qam", "64qam"):
67         n = 4 if mod == "16qam" else 8
68         return _gray(k // n) * n + _gray(k % n)
69     if mod == "32qam":
70         return k
71     return _gray(k)
72
73
74 def fsk_levels(mod: str) -> tuple[np.ndarray, np.ndarray]:
75     """(frequency levels, Gray label per level) - e.g. fsk4: [-3,-1,1,3], [0,1,3,2]."""
76     m = mod_order(mod)
77     return np.arange(m) * 2.0 - (m - 1), _gray(np.arange(m))
78
79
80 def to_bits(labels: np.ndarray, mod: str) -> np.ndarray:
81     """Integer labels -> flat 0/1 array, MSB first, log2(M) bits per label."""
82     k = mod_order(mod).bit_length() - 1
83     return (labels[:, None] >> np.arange(k - 1, -1, -1) & 1).ravel().astype(np.uint8)
84
85
86 def demap_bits(symbols: np.ndarray, mod: str) -> np.ndarray:
87     """Hard-decide to nearest ideal point, return Gray bits."""
88     pts = ideal_points(mod)
89     idx = np.argmin(np.abs(symbols[:, None] - pts[None, :]), axis=1)
90     return to_bits(bit_labels(mod)[idx], mod)
91
92
93 def demap_diff_bits(symbols: np.ndarray, mod: str) -> np.ndarray:
94     """Differential demap: quantize successive phase differences to the mod's
95     transition set. Rotation-ambiguity-free (no absolute phase reference)."""
96     ph = DIFF_PHASES[mod]
97     d = np.angle(symbols[1:] * np.conj(symbols[:-1]))
98     err = np.angle(np.exp(1j * (d[:, None] - ph[None, :]))) # wrapped distance
99     return to_bits(np.argmin(np.abs(err), axis=1), mod)

```

```

1  """Synthetic signal generator – the ground-truth fixture for the test harness.
2  Deliberately shares no DSP code with the receiver (only constellation/level
3  reference data), so a receiver bug cannot be masked by a twin generator bug."""
4
5  import os
6  from dataclasses import asdict, dataclass, field
7  from fractions import Fraction
8
9  import numpy as np
10 from scipy.signal import resample_poly
11
12 from .constellations import (
13     DIFF_MODS,
14     DIFF_PHASES,
15     bit_labels,
16     family,
17     fsk_levels,
18     ideal_points,
19     mod_order,
20     to_bits,
21 )
22 from .sigio import Meta, make_name, sidecar_write, write
23
24 _OS = 8          # shaping oversample (samples/symbol)
25 _SPAN = 10       # TX RRC span (symbols)
26
27
28 @dataclass
29 class GenParams:
30     mod: str = "qpsk"
31     n_symbols: int = 6000
32     fs: float = 1e6
33     baud: float = 1e5
34     rolloff: float = 0.35
35     fc: float = 0.0          # iq: carrier offset; real: passband carrier
36     phase: float = 0.0
37     timing: float = 0.0      # fractional-sample offset [0,1)
38     snr: float = 20.0        # sample-power SNR (dB)
39     fmt: str = "iq"
40     dtype: str = "i16"
41     endian: str = "le"
42     bitrev: bool = False
43     seed: int = 0
44     pad: float = 0.0         # leading/trailing noise-only padding (fraction)
45     drift_ppm: float = 0.0   # TX/RX sample-clock offset
46     dc: complex = 0.0        # DC offset / LO leakage
47     h: float = 0.5           # FSK modulation index (msk forces 0.5)
48     taps: tuple = field(default_factory=tuple) # multipath channel gains (complex)
49     tap_sym: float = 1.0     # echo delay between taps, in SYMBOLS
50     preamble: tuple = (0, 0) # (block_len_syms L, repeats R): an L-symbol block tiled R times
51     sync_word: tuple = ()    # fixed symbol-index marker placed after the preamble
52     payload: object = None   # None=random data; else explicit bits (str/0-1 seq) as the data
53
54
55 def _tx_rrc(a: float, sps: int) -> np.ndarray:
56     """TX pulse shaper – vectorized closed form, independent of the RX filter."""
57     t = np.arange(-_SPAN * sps // 2, _SPAN * sps // 2 + 1) / sps
58     with np.errstate(divide="ignore", invalid="ignore"):
59         h = (np.sin(np.pi * t * (1 - a)) + 4 * a * t * np.cos(np.pi * t * (1 + a))) / (
60             np.pi * t * (1 - (4 * a * t) ** 2))

```

```

61     h[np.abs(t) < 1e-9] = 1 - a + 4 * a / np.pi
62     if a > 0:
63         sing = np.abs(np.abs(t) - 1 / (4 * a)) < 1e-9
64         h[sing] = a / np.sqrt(2) * ((1 + 2 / np.pi) * np.sin(np.pi / (4 * a))
65                                     + (1 - 2 / np.pi) * np.cos(np.pi / (4 * a)))
66     return h / np.sqrt(np.sum(h**2))
67
68
69 def _payload_indices(payload: object, mod: str, alpha: int) -> np.ndarray:
70     """Explicit data bits (str '0110...' or a 0/1 sequence) -> symbol indices for `mod`.
71     Inverse of the transmit labelling, so the decoded data bits round-trip to `payload`."""
72     k = max(1, mod_order(mod).bit_length() - 1)
73     if isinstance(payload, str):
74         b = np.array([int(c) for c in payload if c in "01"], dtype=int)
75     else:
76         b = np.asarray(payload, dtype=int).ravel()
77     b = b[: b.size - (b.size % k)] # drop a ragged final symbol
78     if b.size == 0:
79         raise ValueError(f"payload가 너무 짧습니다: {mod}는 심볼당 {k}비트가 필요합니다")
80     labels = (b.reshape(-1, k) * (1 << np.arange(k - 1, -1, -1))).sum(1) # MSB-first (to_bits)
81     if mod in DIFF_MODS:
82         return labels % alpha # diff: the label IS the transition index
83     lab = np.asarray(bit_labels(mod)) # idx -> label; invert to label -> idx
84     inv = np.zeros(int(lab.max()) + 1, dtype=int)
85     inv[lab] = np.arange(lab.size)
86     return inv[labels % (1 << k)]
87
88
89 def _content(p: GenParams, rng: np.random.Generator, alpha: int) -> tuple[np.ndarray, np.ndarray]:
90     """Build [preamble][sync_word][data] symbol indices for alphabet size `alpha`, returning
91     (all_idx, data_idx) -- ground-truth bits come from the DATA portion only. The preamble is an
92     L-symbol block tiled R times, drawn from a DEDICATED stream so the main `rng` is untouched;
93     with no preamble/sync/payload the single data draw matches the legacy output byte-for-byte."""
94     head = []
95     pre = p.preamble if isinstance(p.preamble, (tuple, list)) and len(p.preamble) == 2 else (0, 0)
96     L, R = int(pre[0]), int(pre[1])
97     if L > 0 and R > 0:
98         block = np.random.default_rng(p.seed ^ 0x9E3779B9).integers(0, alpha, L)
99         head.append(np.tile(block, R))
100     if len(p.sync_word):
101         head.append(np.asarray(p.sync_word, dtype=int) % alpha)
102     data = rng.integers(0, alpha, p.n_symbols) if p.payload is None \
103         else _payload_indices(p.payload, p.mod, alpha)
104     return (np.concatenate([*head, data]) if head else data), data
105
106
107 def _linear(p: GenParams, rng: np.random.Generator) -> tuple[np.ndarray, np.ndarray]:
108     """PSK/QAM/OQPSK/differential: symbols -> RRC shaping at _OS samples/symbol."""
109     m = mod_order(p.mod)
110     idx, data = _content(p, rng, m)
111     if p.mod in DIFF_MODS: # data lives in the phase TRANSITIONS
112         bits = to_bits(data, p.mod)
113         sym = np.exp(1j * np.cumsum(DIFF_PHASES[p.mod][idx]))
114     else:
115         bits = to_bits(bit_labels(p.mod)[data], p.mod)
116         sym = ideal_points(p.mod)[idx]
117     up = np.zeros(idx.size * _OS, dtype=complex)
118     up[::_OS] = sym
119     x = np.convolve(up, _tx_rrc(p.rolloff, _OS), mode="same")
120     if p.mod == "oqpsk": # Q rail delayed half a symbol

```

```

121         x = x.real + 1j * np.concatenate([np.zeros(_OS // 2), x.imag[: -_OS // 2]])
122     return x, bits
123
124
125 def _fsk(p: GenParams, rng: np.random.Generator) -> tuple[np.ndarray, np.ndarray]:
126     """CPFSK/MSK: Gray level sequence -> continuous-phase frequency modulation."""
127     levels, labels = fsk_levels(p.mod)
128     idx, data = _content(p, rng, levels.size)
129     bits = to_bits(labels[data], p.mod)
130     h = 0.5 if p.mod == "msk" else p.h
131     f_cps = np.repeat(levels[idx], _OS) * h / 2 # instantaneous freq, cycles/symbol
132     x = np.exp(2j * np.pi * np.cumsum(f_cps) / _OS)
133     return x, bits
134
135
136 def generate(p: GenParams) -> tuple[np.ndarray, np.ndarray]:
137     """Return (samples, tx_bits). Chain: modulate -> resample -> timing/drift ->
138     carrier/phase -> multipath taps -> (real) -> dc -> AWGN -> padding."""
139     rng = np.random.default_rng(p.seed)
140     x, bits = (_fsk if family(p.mod) == "fsk" else _linear)(p, rng)
141
142     r = Fraction(p.fs / (p.baud * _OS)).limit_denominator(2000)
143     if r != 1:
144         x = resample_poly(x, r.numerator, r.denominator)
145     x /= np.sqrt(np.mean(np.abs(x) ** 2))
146
147     if p.timing or p.drift_ppm: # fractional delay + clock drift via one regrid
148         t = np.arange(x.size) * (1 + p.drift_ppm * 1e-6) + p.timing
149         x = np.interp(t, np.arange(x.size), x.real) + 1j * np.interp(
150             t, np.arange(x.size), x.imag)
151
152     n = np.arange(x.size)
153     x = x * np.exp(1j * (2 * np.pi * p.fc / p.fs * n + p.phase))
154     if p.taps: # multipath: echoes delayed by tap_sym symbols (=> real symbol-rate ISI)
155         step = max(1, int(round(p.tap_sym * p.fs / p.baud)))
156         kern = np.zeros((len(p.taps) - 1) * step + 1, dtype=complex)
157         kern[::step] = p.taps
158         x = np.convolve(x, kern, mode="same")
159         x /= np.sqrt(np.mean(np.abs(x) ** 2))
160     if p.fmt == "real":
161         x = x.real.astype(np.float64) * np.sqrt(2)
162     x = x + (p.dc if np.iscomplexobj(x) else complex(p.dc).real)
163
164     nvar = np.mean(np.abs(x) ** 2) / 10 ** (p.snr / 10)
165     if np.iscomplexobj(x):
166         def noise(k: int) -> np.ndarray:
167             return np.sqrt(nvar / 2) * (rng.standard_normal(k) + 1j * rng.standard_normal(k))
168     else:
169         def noise(k: int) -> np.ndarray:
170             return np.sqrt(nvar) * rng.standard_normal(k)
171     x = x + noise(x.size)
172     if p.pad > 0:
173         k = int(x.size * p.pad)
174         x = np.concatenate([noise(k), x, noise(k)])
175     return x, bits
176
177
178 def save(p: GenParams, outdir: str, label: str = "sig") -> str:
179     """Write samples + '<file>.json' ground-truth sidecar; return the data path."""
180     x, _ = generate(p)

```

```
181 meta = Meta(p.fs, p.fmt, p.dtype, p.endian, p.bitrev)
182 name = make_name(label, meta, "iq" if p.fmt == "iq" else "pcm")
183 os.makedirs(outdir, exist_ok=True)
184 path = os.path.join(outdir, name)
185 write(path, x, meta)
186 d = asdict(p)
187 truth = {k: d[k] for k in ("mod", "fc", "baud", "rolloff", "snr")}
188 if family(p.mod) == "fsk":
189     truth["h"] = 0.5 if p.mod == "msk" else p.h
190     truth.pop("rolloff") # meaningless for CPFSK
191 gen_info = {k: d[k] for k in ("seed", "n_symbols", "timing", "pad", "drift_ppm")}
192 if p.taps:
193     gen_info["taps"] = [[complex(t).real, complex(t).imag] for t in p.taps]
194     gen_info["tap_sym"] = p.tap_sym
195 sidecar_write(path, meta, truth, gen_info)
196 return path
```

```

1  """Spectrum + waterfall views of the burst (display only, never used for detection)."""
2
3  import numpy as np
4  from scipy.signal import welch
5
6
7  def _db(p: np.ndarray) -> np.ndarray:
8      return 10 * np.log10(p + 1e-20)
9
10
11 def spectrum(x: np.ndarray, fs: float, bins: int = 256) -> dict:
12     """Averaged PSD of the burst, decimated to `bins` points. Frequencies in kHz."""
13     nper = int(min(4096, max(64, x.size // 8)))
14     f, p = welch(x, fs=fs, nperseg=nper, return_onesided=False, detrend=False)
15     f, p = np.fft.fftshift(f), _db(np.fft.fftshift(p))
16     if f.size > bins: # max-pool so narrow tones survive decimation
17         k = f.size // bins
18         f, p = f[:k * bins].reshape(bins, k).mean(1), p[:k * bins].reshape(bins, k).max(1)
19     return {"f": np.round(f / 1e3, 3).tolist(), "db": np.round(p, 2).tolist()}
20
21
22 def waterfall(x: np.ndarray, fs: float, rows: int = 72, bins: int = 144) -> dict:
23     """Time-frequency magnitude (dB), row-major, robustly scaled by the caller."""
24     nfft = 1 << int(np.ceil(np.log2(max(bins * 2, 64))))
25     step = max(1, (x.size - nfft) // max(rows - 1, 1))
26     idx = np.arange(rows) * step
27     idx = idx[idx + nfft <= x.size]
28     if idx.size == 0:
29         return {"rows": 0, "bins": bins, "db": [], "dt": 0.0}
30     win = np.hanning(nfft)
31     seg = np.stack([x[i:i + nfft] * win for i in idx])
32     s = np.abs(np.fft.fftshift(np.fft.fft(seg, axis=1), axes=1)) ** 2
33     k = nfft // bins
34     s = s[:, :k * bins].reshape(idx.size, bins, k).mean(2)
35     return {"rows": int(idx.size), "bins": bins, "dt": float(step / fs),
36           "db": np.round(_db(s).ravel(), 2).tolist()}

```

```

1  """Wideband signal detection: find every emitter in a capture as a frequency/time
2  box. Turns signus from a single-signal demodulator into a survey tool. Frequency
3  detection runs on the time-averaged spectrogram (robust to signal width); the time
4  extent per band comes from the spectrogram. numpy + scipy only."""
5
6  from dataclasses import dataclass
7
8  import numpy as np
9  from scipy import ndimage
10 from scipy.signal import get_window, stft
11
12
13 @dataclass
14 class Detection:
15     fc: float          # centre frequency (Hz, baseband offset)
16     bw: float          # 99%-power bandwidth (Hz)
17     t0: int            # burst start / end (sample indices into the capture)
18     t1: int
19     snr_db: float      # peak-to-floor of the band (dB)
20
21     @property
22     def baud_hint(self) -> float:
23         """Rough symbol-rate prior: 99%-power BW is ~0.86 of occupied BW  $R_s(1+a)$ ."""
24         return self.bw / (0.86 * 1.25)
25
26
27 def _floor(s: np.ndarray, frac: float = 0.5, pct: int = 20) -> np.ndarray:
28     """Per-bin noise floor via a wide low-percentile filter. A 20th percentile over
29     a half-band window tracks the front-end tilt yet stays on the noise even when a
30     signal fills up to ~half the window -- so it survives both a narrowband emitter
31     and a single signal occupying 10-50% of the band."""
32     w = max(11, int(frac * s.size) | 1)
33     return ndimage.percentile_filter(s, pct, size=w, mode="nearest")
34
35
36 def detect(x: np.ndarray, fs: float, *, nfft: int = 4096, overlap: float = 0.5,
37           thr_db: float = 6.0, min_bw_bins: int = 3) -> list[Detection]:
38     """Detect emitters as (fc, bw, t0, t1) boxes, time/frequency ordered by fc.
39     Empty capture -> []; a single signal filling the band -> one whole-band box."""
40     if x.size == 0:
41         return []
42     # nfft never exceeds the record (else stft rejects an over-long window)
43     nfft = int(min(nfft, x.size, 1 << max(8, int(np.log2(max(x.size // 16, 256)))))
44     win = get_window("blackmanharris", nfft)
45     f, t, z = stft(x, fs=fs, window=win, nperseg=nfft, noverlap=int(nfft * overlap),
46                   return_onesided=False, boundary=None, padded=False)
47     f = np.fft.fftshift(f)
48     p = np.fft.fftshift(np.abs(z) ** 2, axes=0)          # (freq, time) linear power
49     s = p.mean(axis=1)                                  # time-averaged PSD
50     floor = _floor(s)
51     mask = s > floor * 10 ** (thr_db / 10)
52     close = max(3, int(0.012 * s.size) | 1)             # bridge intra-signal nulls
53     mask = ndimage.binary_opening(mask, structure=np.ones(2))
54     mask = ndimage.binary_closing(mask, structure=np.ones(close))
55
56     lab, n = ndimage.label(mask)
57     regions = [np.where(lab == i)[0] for i in range(1, n + 1)]
58     regions = [r for r in regions if r.size >= min_bw_bins]
59     merged: list[np.ndarray] = []
60     for r in sorted(regions, key=lambda r: r[0]):

```

```

61         # heal an intra-signal split (gap small vs the signal's OWN width) without
62         # swallowing a genuinely separate neighbour -- cap the merge gap at half the
63         # narrower blob, so adjacent channels (e.g. the 25 kHz marine raster) survive
64         if merged and r[0] - merged[-1][-1] <= max(close, min(merged[-1].size, r.size) // 2):
65             merged[-1] = np.arange(merged[-1][0], r[-1] + 1)
66         else:
67             merged.append(r)
68     if len(merged) >= 2: # frequency is circular: heal a signal straddling +-fs/2
69         wrap_gap = (s.size - 1 - merged[-1][-1]) + merged[0][0]
70         if wrap_gap <= max(close, min(merged[0].size, merged[-1].size) // 2):
71             merged[0] = np.concatenate([merged.pop(), merged[0]]) # wraps the edge
72
73     dets = [_measure(r, s, floor, p, f, t, fs, x.size) for r in merged]
74     if not dets and s.max() > 10 ** (thr_db / 10) * np.percentile(s, 20):
75         # floor defeated by a signal filling most of the band: one whole-band box,
76         # so survey() falls back to analysing the entire capture (legacy behaviour)
77         dets = [Detection(0.0, fs * 0.5, 0, x.size, 0.0)]
78     return dets
79
80
81 def _measure(r: np.ndarray, s: np.ndarray, floor: np.ndarray, p: np.ndarray,
82             f: np.ndarray, t: np.ndarray, fs: float, n: int) -> Detection:
83     """Estimate fc (floor-subtracted centroid), 99%-power bw, and time extent.
84     Handles a wrapped index set (a signal straddling +-fs/2, non-contiguous)."""
85     wrapped = r.size > 1 and not np.all(np.diff(r) == 1)
86     if wrapped:
87         idx = r # already spans the edge
88         fr = f[idx].astype(float).copy()
89         fr[idx < s.size // 2] += fs # lift the -fs/2 side onto a line
90     else:
91         idx = np.arange(max(0, r[0] - 3), min(s.size, r[-1] + 4)) # widen for skirts
92         fr = f[idx]
93     wgt = np.maximum(s[idx] - floor[idx], 0)
94     fc = float(np.sum(fr * wgt) / (np.sum(wgt) + 1e-30))
95     o = np.argsort(fr)
96     cs = np.cumsum(wgt[o]) / (np.sum(wgt) + 1e-30)
97     f_lo = fr[o][np.searchsorted(cs, 0.005)]
98     f_hi = fr[o][min(np.searchsorted(cs, 0.995), idx.size - 1)]
99     bw = float(abs(f_hi - f_lo))
100    if fc >= fs / 2: # fold a wrapped centroid back
101        fc -= fs
102    band = p[idx, :].sum(axis=0) # time profile of the band
103    on = np.where(band > max(np.median(band) * 2.0, band.max() * 0.1))[0]
104    t0 = int(t[on[0]] * fs) if on.size else 0
105    t1 = int(t[min(on[-1], t.size - 1)] * fs) if on.size else n
106    snr = float(10 * np.log10(s[idx].max() / (np.median(floor) + 1e-30)))
107    return Detection(fc, bw, t0, t1, snr)

```

```

1  """Channel extraction: pull one detected emitter out of a wideband capture as a
2  baseband stream, sized so the signal occupies ~1/4-1/3 of the reduced rate -- the
3  empirically-validated sweet spot for the downstream blind estimators (fill the band
4  and the matched filter starves; leave it too wide and noise dominates)."""
5
6  import numpy as np
7  from scipy.signal import firwin, resample_poly
8
9  from . import dsp
10 from .detect import Detection
11
12
13 def extract(x: np.ndarray, fs: float, det: Detection, *, target_frac: float = 0.28,
14            pad: float = 1.5, ntaps: int = 129) -> tuple[np.ndarray, float]:
15     """Mix det.fc to DC, low-pass to the channel, decimate; trim to the burst.
16     Returns (baseband IQ, channel sample rate)."""
17     bb = dsp.mix(x, fs, det.fc)
18     want = max(det.bw * pad, fs * 1e-4) # passband to keep
19     d = max(1, int(fs / (want / target_frac))) # want ~= target_frac * fs/d
20     fs_ch = fs / d
21     # Low-pass to the box ALWAYS, even when d==1 (a wide box, bw > ~0.28*fs): otherwise the
22     # "channel" is the whole capture merely mixed, and analyze locks onto a DIFFERENT, stronger
23     # emitter (reported as a confident duplicate). Decimate only when there is room to.
24     cutoff = min(0.9 * fs_ch / 2, want / 2) / (fs / 2)
25     bb = np.convolve(bb, firwin(ntaps, cutoff), "same")
26     if d > 1:
27         bb = resample_poly(bb, 1, d)
28     a, b = int(det.t0 // d), int(det.t1 // d)
29     if b - a >= 256: # trim empty time, keep guard
30         g = (b - a) // 20
31         bb = bb[max(0, a - g):b + g]
32     return bb.astype(np.complex128), fs_ch

```

```

1  """Per-channel triage: decide whether an extracted channel is a demodulable digital
2  signal or something the demodulator must NOT force onto a constellation (analog FM
3  voice, a CW tone/spur). Envelope CV is the SNR-robust separator: RRC-shaped PSK/QAM
4  always sit at CV>=0.25, while FSK/FM/CW are constant-envelope (CV<=0.05)."""
5
6  import numpy as np
7
8  from .chirp import is_chirp, sweeps_band
9  from .fsk import fsk_gate
10
11  _CV_CE = 0.15      # constant-envelope ceiling: below it a non-FSK signal is analog/tone
12  _TONE_PAR = 500.0  # M-th-power peak-to-mean above which a constant-envelope signal is CW
13
14
15  def _carrier_par(x: np.ndarray, powers: tuple[int, ...] = (1, 2, 4, 8)) -> float:
16      """Best peak-to-mean of the M-th-power magnitude spectrum. A CW tone spikes at
17      p=1; analog FM and noise stay flat at every power."""
18      x = x - x.mean()
19      nfft = 1 << int(np.ceil(np.log2(x.size)))
20      win = np.hanning(x.size)
21      best = 0.0
22      for p in powers:
23          spec = np.abs(np.fft.fft(x ** p * win, nfft))
24          best = max(best, float(spec.max() / (spec.mean() + 1e-30)))
25      return best
26
27
28  def _active(x: np.ndarray) -> np.ndarray:
29      """Trim leading/trailing dead air (envelope below half the top-quartile median). A bursty
30      channel's dead air corrupts BOTH triage discriminators -- it inflates the envelope CV past
31      fsk_gate's ceiling AND pollutes sweeps_band's IF histogram -- so every gate below must see
32      the emitter, not the silence around it. A continuous channel is returned unchanged."""
33      env = np.abs(x)
34      top = env[env > np.percentile(env, 75)]
35      if top.size == 0:
36          return x
37      on = np.flatnonzero(env > 0.5 * np.median(top))
38      if on.size < 256:
39          return x
40      return x[on[0]:on[-1] + 1]
41
42
43  def family(x: np.ndarray, fs: float) -> str:
44      """One of 'fsk' | 'chirp' | 'linear' | 'analog' | 'tone'. 'linear'/'fsk' go to the
45      demod; 'chirp'/'analog'/'tone' are reported as-is (never force-fit to a constellation)."""
46      x = _active(x)
47      if fsk_gate(x, fs):
48          # a linear FMCW chirp / CSS (LoRa) trips fsk_gate too -- its swept IF reads bimodal to
49          # the gate -- and would then be force-demodulated into confident garbage FSK symbols.
50          # is_chirp fires on both a chirp and (at some h/ baud) real FSK, so the IF-SWEEP test
51          # breaks the tie: a genuine band-sweep is characterized as chirp, real FSK stays FSK.
52          if is_chirp(x, fs) and sweeps_band(x, fs):
53              return "chirp"
54          return "fsk"
55      if is_chirp(x, fs) and sweeps_band(x, fs):
56          # linear chirp / CSS (LoRa) -- constant-envelope, would otherwise fall through to
57          # 'analog' (no M-power tone). The tie-break is REQUIRED here too: a bursty integer-h
58          # FSK channel (dead air inflates envelope CV past fsk_gate) also trips is_chirp via
59          # its CPFSK beat tones -- without sweeps_band it was labeled 'chirp' and never
60          # demodulated. A non-sweeping channel falls through, and the demod's burst isolation

```

```
61         # recovers the FSK.
62         return "chirp"
63     a = np.abs(x)
64     if a.std() / (a.mean() + 1e-12) < _CV_CE:          # constant envelope, not FSK
65         return "tone" if _carrier_par(x) >= _TONE_PAR else "analog"
66     return "linear"
```

```

1  """Receiver DSP stages for blind PSK/QAM demodulation. Vectorized; per the house
2  anti-shared-bug rule this imports only constellation reference data, never gen.py."""
3
4  from fractions import Fraction
5
6  import numpy as np
7  from scipy.ndimage import uniform_filter1d
8  from scipy.signal import hilbert, resample_poly, welch
9
10 from ._accel import dd_carrier
11 from .constellations import ideal_points
12
13
14 def analytic(x: np.ndarray) -> np.ndarray:
15     """Complex passthrough as complex128; real input -> Hilbert analytic signal. Non-finite
16     samples are zeroed first: a single NaN/Inf otherwise spreads through every FFT (and Inf
17     overflows on the |x|^2 the estimators take)."""
18     x = np.nan_to_num(x, posinf=0.0, neginf=0.0)
19     x = np.where(np.abs(x) > 1e150, 0.0, x) # a finite spike that overflows |x|^2 is corrupt too
20     if np.iscomplexobj(x):
21         return x.astype(np.complex128)
22     return hilbert(np.asarray(x, dtype=np.float64)).astype(np.complex128)
23
24
25 def _parab(ydb: np.ndarray, k: int) -> float:
26     """Sub-bin peak offset from a 3-point parabola fit (inputs already in dB)."""
27     if k <= 0 or k >= ydb.size - 1:
28         return 0.0
29     a, b, c = ydb[k - 1], ydb[k], ydb[k + 1]
30     d = a - 2 * b + c
31     # clamp to +-half a bin: a true peak's sub-bin offset cannot exceed that, but a near-flat
32     # (d~0) or non-max triple can explode the ratio and drive baud negative -> a resample crash.
33     return float(np.clip(0.5 * (a - c) / d, -0.5, 0.5)) if d != 0 else 0.0
34
35
36 def _kmeans1d(a: np.ndarray, k: int, iters: int = 50) -> tuple[np.ndarray, np.ndarray, float]:
37     """1-D k-means on quantile-seeded centers; return (centers, labels, rms spread).
38     Assignment is a running min over the k centers (no (N, k) broadcast temporary);
39     strict '<' keeps the lowest-index center on ties, so labels are identical to a
40     broadcast argmin. Shared by the classify amplitude-ring test and the FSK level demod."""
41     c = np.quantile(a, (np.arange(k) + 0.5) / k)
42     lab = np.zeros(a.size, dtype=int)
43     for _ in range(iters):
44         d = np.abs(a - c[0])
45         lab = np.zeros(a.size, dtype=int)
46         for j in range(1, k):
47             dj = np.abs(a - c[j])
48             closer = dj < d
49             lab[closer] = j
50             d[closer] = dj[closer]
51         nc = np.array([a[lab == j].mean() if np.any(lab == j) else c[j] for j in range(k)])
52         if np.allclose(nc, c):
53             break
54         c = nc
55     return c, lab, float(np.sqrt(np.mean((a - c[lab]) ** 2)))
56
57
58 # --- burst detection -----
59
60 _SPIKE_PAR = 60.0 # raw peak-to-mean power above which a candidate run is an impulse smear, not a

```

```

61 #          burst. Modulated bursts sit low: constant-envelope FSK ~1, RRC-shaped PSK/QAM
62 #          peaks ~6-12, high-order QAM tails to ~20; a single-sample impulse smeared over
63 #          a ~win-wide run scores ~win (hundreds+). 60 clears every real mod with margin.
64
65
66 def find_bursts(x: np.ndarray, fs: float) -> list[tuple[int, int]]:
67     """Dual-threshold energy detector with an Otsu log-power floor; return every
68     merged burst in time order, or [(0, size)] when nothing stands out."""
69     n = x.size
70     win = max(64, n // 1000)
71     pw = np.abs(x) ** 2
72     ps = uniform_filter1d(pw, win)
73     lp = np.log10(ps + 1e-20)
74
75     # Otsu split of the log-power histogram: a floor that survives an 80%-full
76     # record because it is drawn from the (small) below-threshold noise class.
77     hist, edges = np.histogram(lp, bins=128)
78     centers = (edges[:-1] + edges[1:]) / 2
79     w = np.cumsum(hist).astype(float)
80     m = np.cumsum(hist * centers)
81     mb = m / np.where(w > 0, w, 1)
82     mf = (m[-1] - m) / np.where(w[-1] - w > 0, w[-1] - w, 1)
83     btw = w * (w[-1] - w) * (mb - mf) ** 2
84     thr = centers[int(np.argmax(btw))]
85     noise = lp[lp < thr]
86     floor = noise.mean() if noise.size else lp.min()
87
88     above_hi = lp >= floor + 1.0 # 10 dB (power decade) over floor
89     above_lo = lp >= floor + 0.6 # 6 dB
90     dif = np.diff(above_lo.astype(np.int8))
91     starts = list(np.where(dif == 1)[0] + 1)
92     ends = list(np.where(dif == -1)[0] + 1)
93     if above_lo[0]:
94         starts.insert(0, 0)
95     if above_lo[-1]:
96         ends.append(n)
97     runs = [(s, e) for s, e in zip(starts, ends, strict=True) if above_hi[s:e].any()]
98     if not runs:
99         return [(0, n)]
100
101     gap = max(256, n // 200) # merge gap heals intra-signal splits: proportional is fine
102     mlen = max(256, min(n // 200, 1024)) # ...but the MIN burst length must not scale with the
103     # record: a short packetized burst (preamble-sync target, ~1k samples) in a long capture was
104     # dropped by n//200 and the [(0,n)] fallback buried it in noise. 1024 keeps the blip filter
105     # for records up to ~200k while capping it where the packet population starts.
106     merged = [list(runs[0])]
107     for s, e in runs[1:]:
108         if s - merged[-1][1] < gap:
109             merged[-1][1] = e
110         else:
111             merged.append([s, e])
112     # The mlen cap alone lets a short high-energy transient survive in a long record: the smoother
113     # spreads a single impulse into a ~win-wide run that clears 1024, and its energy can outrank
114     # the real signal so analyze() auto-selects the blip. A modulated burst has a near-flat
115     # envelope (raw peak/mean power ~ a few); an impulse smear is one spike over noise (ratio ~win).
116     # Reject runs whose RAW power is spike-dominated -- this separates them by shape, at any length.
117     out = []
118     for s, e in merged:
119         if e - s < mlen:
120             continue

```

```

121         seg = pw[s:e]
122         if float(seg.max() / (seg.mean() + 1e-30)) > _SPIKE_PAR:
123             continue
124         out.append((int(s), int(e)))
125     return out or [(0, n)]
126
127
128 def find_burst(x: np.ndarray, fs: float) -> tuple[int, int]:
129     """The most energetic burst of find_bursts."""
130     return max(find_bursts(x, fs), key=lambda b: float(np.sum(np.abs(x[b[0]:b[1]]) ** 2)))
131
132
133 # --- carrier estimation -----
134
135 def est_carrier(
136     x: np.ndarray, fs: float, powers: tuple[int, ...] = (2, 4, 8),
137     nfft: int = 65536, blocks: int = 4,
138 ) -> tuple[float, int, bool]:
139     """Blind M-th-power carrier estimate; returns (fc, symmetry, ambiguous). No DC
140     nulling: the front-end is DC-blocked, so a peak at DC is a genuine fc=0."""
141     freqs = np.fft.fftshift(np.fft.fftfreq(nfft, 1 / fs))
142     best_par, best_s, sym = -1.0, None, powers[0]
143     for p in powers:
144         xp = x ** p
145         blk = xp.size // blocks
146         segs = [xp[i * blk:(i + 1) * blk] for i in range(blocks)] if blk >= 8 else [xp]
147         acc = np.zeros(nfft)
148         for s in segs:
149             acc += np.abs(np.fft.fftshift(np.fft.fft(s * np.hanning(s.size), nfft)))
150         spec = acc / len(segs)
151         par = spec.max() / spec.mean()
152         if par > best_par:
153             best_par, best_s, sym = par, spec, p
154
155     if best_s is None: # degenerate input (all-zero / no energy): no M-th-power tone exists
156         return 0.0, int(powers[0]), True
157     ydb = 20 * np.log10(best_s + 1e-20)
158     k = int(np.argmax(best_s))
159     f_peak = freqs[k] + _parab(ydb, k) * (freqs[1] - freqs[0])
160     fc = f_peak / sym
161     ambiguous = abs(sym * fc) > 0.4 * fs
162     return float(fc), int(sym), bool(ambiguous)
163
164
165 def resolve_alias(x: np.ndarray, fs: float, fc: float, sym: int) -> float:
166     """The M-th-power tone wraps mod fs, so fc is only known mod fs/sym. Candidates tile the
167     whole band at spacing fs/sym; pick the one whose alias CELL holds the most signal power
168     (wrap-aware). Cell-energy beats a spectral centroid at the band edge, where a signal
169     straddling +-fs/2 corrupts the two-sided centroid (bpsk fc=0.495*fs picked fc=-5000)."""
170     f, pxx = welch(x, fs=fs, nperseg=min(4096, x.size), return_onesided=False)
171     p = np.maximum(pxx - np.median(pxx), 0)
172     # range must reach +-fs/2 for EVERY sym: |k| up to sym//2 (an old fixed range(-2,3) left
173     # 8psk beyond 0.3125*fs with no candidate).
174     cands = [fc + k * fs / sym for k in range(-(sym // 2) - 1, sym // 2 + 2)]
175     if abs(fc + k * fs / sym) < 0.5 * fs]
176     half = fs / (2 * sym) # each candidate owns a fs/sym-wide cell
177     return max(cands, key=lambda c: float(p[np.abs(((f - c + fs / 2) % fs) - fs / 2) < half].sum()))
178
179
180 def mix(x: np.ndarray, fs: float, fc: float) -> np.ndarray:

```

```

181     """Downconvert by fc (complex heterodyne)."""
182     return x * np.exp(-2j * np.pi * fc * np.arange(x.size) / fs)
183
184
185 # --- symbol rate + rolloff -----
186
187 def est_baud(
188     x: np.ndarray, fs: float, lo: float | None = None, hi: float | None = None,
189     blocks: int = 4,
190 ) -> tuple[float, float]:
191     """Cyclostationary line at the symbol rate in  $|x|^2$ ; returns (baud, confidence).
192     Low confidence flags a weak line (near-zero rolloff); caller decides. Fewer blocks =
193     a longer, stronger line (worth trying on a short burst where the default 4 is too weak)."""
194     u = np.abs(x) ** 2
195     u = u - u.mean()
196     blk = u.size // blocks
197     segs = [u[i * blk:(i + 1) * blk] for i in range(blocks)] if blk >= 8 else [u]
198     nfft = 1 << max(16, int(np.ceil(np.log2(max(s.size for s in segs))))))
199     acc = np.zeros(nfft // 2 + 1)
200     for s in segs:
201         acc += np.abs(np.fft.rfft(s * np.hanning(s.size), nfft))
202     spec = acc / len(segs)
203     freqs = np.fft.rfftfreq(nfft, 1 / fs)
204
205     lo = lo if lo is not None else 0.005 * fs
206     hi = hi if hi is not None else 0.45 * fs
207     band = (freqs >= lo) & (freqs <= hi)
208     if not band.any(): # degenerate window (e.g. noise-derived prior): default band
209         band = (freqs >= 0.005 * fs) & (freqs <= 0.45 * fs)
210     idx = np.where(band)[0]
211     k = idx[int(np.argmax(spec[idx]))]
212     ydb = 20 * np.log10(spec + 1e-20)
213     baud = freqs[k] + _parab(ydb, k) * (freqs[1] - freqs[0])
214     conf = spec[k] / (np.median(spec[band]) + 1e-20)
215     return float(baud), float(conf)
216
217
218 def occupied_bw(x: np.ndarray, fs: float) -> float:
219     """Two-sided occupied bandwidth without a baud prior: threshold 10% of the way
220     from the 25th-percentile floor to the 90th-percentile in-band level, walk
221     outward from DC, stop at the first >20-bin below-threshold gap (notch tolerant).
222     Returns 0.0 when no band hugs DC (caller must not trust the prior then)."""
223     f, pxx = welch(x, fs=fs, nperseg=min(8192, x.size), return_onesided=False)
224     floor = np.percentile(pxx, 25)
225     order = np.argsort(np.abs(f))
226     idx = np.flatnonzero(pxx[order] > floor + 0.10 * (np.percentile(pxx, 90) - floor))
227     if idx.size == 0:
228         return 0.0
229     last = idx[-1]
230     return float(2 * np.abs(f[order][last]))
231
232
233 def est_rolloff(x: np.ndarray, fs: float, baud: float) -> float:
234     """Occupied-bandwidth band-edge estimate mapped through the v1 rolloff calibration."""
235     n = x.size
236     nper = min(8192, max(256, n // 8))
237     f, pxx = welch(x, fs=fs, nperseg=nper, return_onesided=False)
238     f, pxx = np.fft.fftshift(f), np.fft.fftshift(pxx)
239     af = np.abs(f)
240     inband = np.median(pxx[af < 0.25 * baud])

```

```

241     noise = np.median(pxx[af > 0.9 * baud])
242     thr = noise + 0.10 * (inband - noise)
243     band = (af > 0.3 * baud) & (af < 1.0 * baud) & (pxx > thr)
244     if not band.any():
245         return 0.35
246     raw = 2 * af[band].max() / baud - 1
247     return float(np.clip(1.5 * raw + 0.026, 0.05, 1.0))
248
249
250 # --- resampling + matched filter -----
251
252 def to_sps(x: np.ndarray, fs: float, baud: float, sps: int = 4) -> np.ndarray:
253     """Rational resample to exactly sps samples/symbol."""
254     r = Fraction(sps * baud / fs).limit_denominator(2000)
255     return resample_poly(x, r.numerator, r.denominator)
256
257
258 def _rx_rrc(alpha: float, span: int, sps: int) -> np.ndarray:
259     """RX matched RRC taps, unit energy, odd length; singularities at t=0 and
260     |t|=1/4a patched. Written independently of gen.py's TX shaper (anti-shared-bug)."""
261     m = (span * sps) // 2
262     t = np.arange(-m, m + 1) / sps
263     a = alpha
264     num = np.sin(np.pi * t * (1 - a)) + 4 * a * t * np.cos(np.pi * t * (1 + a))
265     den = np.pi * t * (1 - (4 * a * t) ** 2)
266     h = np.divide(num, den, out=np.zeros_like(t), where=den != 0)
267     h[t == 0] = 1 - a + 4 * a / np.pi
268     if a > 0:
269         s = np.isclose(np.abs(t), 1 / (4 * a))
270         h[s] = a / np.sqrt(2) * ((1 + 2 / np.pi) * np.sin(np.pi / (4 * a))
271                                + (1 - 2 / np.pi) * np.cos(np.pi / (4 * a)))
272     return h / np.sqrt(np.sum(h ** 2))
273
274
275 def matched(x: np.ndarray, sps: int, alpha: float, span: int = 10) -> np.ndarray:
276     """Apply the RX matched RRC filter."""
277     return np.convolve(x, _rx_rrc(alpha, span, sps), "same")
278
279
280 # --- timing recovery -----
281
282 def timing(x: np.ndarray, sps: int, block: int = 256, out: int = 1) -> np.ndarray:
283     """Block-wise Oerder-Meyr timing recovery; returns `out` samples/symbol
284     (1 = decisions, 2 = clock-tracked T/2 stream for the fractionally-spaced eq).
285     Per block the spectral-line phase gives the offset; unwrapped across blocks
286     to track clock drift, then interpolated per symbol."""
287     n = sps
288     nsym = x.size // n
289     if nsym < 1:
290         return x[:0].astype(np.complex128)
291     xt = x[:nsym * n]
292     p = np.abs(xt) ** 2
293     ph = np.exp(-2j * np.pi * np.arange(xt.size) / n)
294     bs = block * n
295     if xt.size <= bs: # single block: one global fractional offset
296         eps = np.full(nsym, -np.angle(np.sum(p * ph)) / (2 * np.pi))
297     else:
298         nblk = xt.size // bs
299         c = (p[:nblk * bs].reshape(nblk, bs) * ph[:nblk * bs].reshape(nblk, bs)).sum(1)
300         eps_b = np.unwrap(-np.angle(c)) / (2 * np.pi)

```

```

301         eps = np.interp(np.arange(nsym), (np.arange(nblk) + 0.5) * block, eps_b)
302     pos = (np.arange(nsym * out) // out + np.repeat(eps, out)) * n
303     idx = np.arange(xt.size)
304     return np.interp(pos, idx, xt.real) + 1j * np.interp(pos, idx, xt.imag)
305
306
307 # --- decision-directed carrier sync -----
308
309 def ddsync(symbols: np.ndarray, mod: str, alpha: float = 0.05, beta: float = 0.002) -> np.ndarray:
310     """Decision-directed PI carrier loop, general across PSK/QAM (never a PSK-only
311     Costas loop, which jitters QAM at the correct points). Sequential feedback (each
312     phase update depends on the prior decision) -> numba kernel when available."""
313     pts = ideal_points(mod)
314     z = symbols / np.sqrt(np.mean(np.abs(symbols) ** 2)) # AGC to unit power
315     return dd_carrier(np.ascontiguousarray(z, dtype=np.complex128),
316                      np.ascontiguousarray(pts, dtype=np.complex128), alpha, beta)

```

```

1  """Blind symbol-spaced FIR equalizer: CMA acquisition then decision-directed LMS,
2  to undo the multipath ISI (the generator's `taps=` channel) that a phase-only
3  carrier loop cannot remove."""
4
5  import numpy as np
6
7  from ._accel import cma_dd_equalize, cma_dd_equalize_fse
8  from .constellations import ideal_points
9
10
11 def equalize(symbols: np.ndarray, mod: str, n_taps: int = 11, mu_cma: float = 5e-3,
12             mu_dd: float = 2e-3, warmup: int = 1500) -> np.ndarray:
13     """Blind FIR equalizer for symbol-spaced samples; returns the cleaned symbols.
14
15     Phase 1 is constant-modulus (Godard) acquisition, phase 2 refines by decisions,
16     then the converged taps refilter the whole record so early symbols are clean too.
17     """
18     pts = ideal_points(mod)
19     r2 = np.mean(np.abs(pts) ** 4) / np.mean(np.abs(pts) ** 2) # Godard dispersion const
20     x = symbols / np.sqrt(np.mean(np.abs(symbols) ** 2)) # AGC to unit power
21     # The adaptive passes (every tap update depends on the previous output/decision) are
22     # the one part that cannot be vectorized; they run as a numba kernel when available.
23     return cma_dd_equalize(np.ascontiguousarray(x, dtype=np.complex128),
24                           np.ascontiguousarray(pts, dtype=np.complex128),
25                           float(r2), n_taps, mu_cma, mu_dd, warmup)
26
27
28 def equalize_fse(x2: np.ndarray, mod: str, n_taps: int = 75, mu_cma: float = 1e-3,
29                 mu_dd: float = 8e-4, warmup: int = 4000) -> np.ndarray:
30     """T/2 fractionally-spaced CMA->DD equalizer: 2 samples/symbol in, symbols out.
31
32     Sees the unaliased spectrum, so it inverts channels whose symbol-rate folding
33     puts zeros near the unit circle (e.g. a strong 1.5-symbol echo) where the
34     symbol-spaced `equalize` cannot. A strong echo's inverse is LONG (0.8 gain
35     needs ~70 T/2 taps for -20 dB); the clock must already be tracked, so feed it
36     `timing(ym, sps, out=2)` -- fixed taps cannot follow a drifting clock.
37     Calibrated on 2ray(0.8, 1.5 sym) x seeds 0-2: lock 77-80 (61 taps: 57-66)."""
38     pts = ideal_points(mod)
39     r2 = np.mean(np.abs(pts) ** 4) / np.mean(np.abs(pts) ** 2) # Godard dispersion const
40     x = x2 / np.sqrt(np.mean(np.abs(x2) ** 2)) # AGC to unit power
41     # Seven sequential passes (2 CMA + 4 DD + 1 tracked output); numba kernel when available.
42     return cma_dd_equalize_fse(np.ascontiguousarray(x, dtype=np.complex128),
43                               np.ascontiguousarray(pts, dtype=np.complex128),
44                               float(r2), n_taps, mu_cma, mu_dd, warmup)

```

```

1  """Blind repeated-preamble synchronization (definition-free).
2
3  A real packetized emission begins with a preamble: a short block repeated a few times so a
4  receiver can lock. The repetition is detectable and exploitable WITHOUT knowing its content --
5  the delay-and-correlate (Schmidl-Cox) metric  $M(d) = |\text{Sum conj}(x[d+k]) x[d+k+P]|^2 / (\text{Sum } |x|^2)^2$ 
6  rises to  $\sim 1$  over the span where two period- $P$  windows coincide (inside the preamble), and the PHASE
7  of that correlation is a low-variance carrier-frequency-offset (CFO) estimate that works from only a
8  few symbols -- where the blind  $M$ -th-power carrier estimator, needing many symbols to average, fails.
9
10 Used as a gated, keep-best rescue in pipeline.analyze(): a signal with no preamble yields no
11 plateau -> no rescue -> the existing blind chain is untouched (sweep stays byte-identical)."""
12
13 from dataclasses import dataclass
14
15 import numpy as np
16
17 _CONF_MIN = 0.80      # plateau strength floor (median M over the run); calibrated vs no-preamble
18 _RUN_MIN = 4.0        # plateau length >= this * period. RRC pulse memory spans a FIXED  $\sim 4.5$  symbols,
19 #                      so in periods it exceeds this only at a 1-2 symbol lag and dies by  $P \sim 3$  syms;
20 #                      a real preamble's plateau is  $(R-1)$  periods long -> this rejects the short
21 #                      memory (needs  $R \sim 5$  repeats; residual random false-alarms are caught by the
22 #                      pipeline keep-best-lock gate -- a spurious CFO lowers lock and is discarded).
23 _HARM_MIN = 0.50      # the SAME plateau must also correlate at lag  $2P$  (a real preamble,  $\geq 3$  repeats)
24 _VALLEY_MAX = 0.80    # ...AND DROP at lag  $1.5P$  (between harmonics): a real preamble is peaked at
25 #                      multiples of  $P$  (a valley in between), so  $\text{median}(M@1.5P) < 0.85 * \text{conf}$ . RRC pulse
26 #                      memory is a smooth decay ( $\sim$ equal at  $P$  and  $1.5P$ ) -> rejected. Peak/valley +
27 #                      run length is what lets the period be scanned directly, WITHOUT a reliable
28 #                      baud (est_baud is exactly what fails on the short bursts we target).
29 _PMIN = 12            # smallest period to scan (samples); below this is intra-symbol pulse memory
30 _PMAX = 512           # largest period to scan; a preamble block is short ( $L \leq \sim 12$  syms,  $\text{sps} \leq \sim 40$ )
31 _WMAX = 8192          # only the leading window is scanned -- a preamble sits at the burst START
32
33
34 @dataclass
35 class Preamble:
36     start: int         # sample index where the repeated region begins
37     end: int           # sample index where the preamble ends (data starts ~here)
38     period: int        # repetition period in samples (=  $L * \text{sps}$ )
39     cfo_hz: float      # carrier-frequency offset estimated from the preamble phase slope
40     conf: float        # 0..1 plateau confidence (median metric over the detected run)
41
42
43 def _metric(x: np.ndarray, P: int) -> tuple[np.ndarray, np.ndarray]:
44     """Normalized delay-correlation  $M(d)$  in  $[0,1]$  and the raw correlation  $P_{\text{met}}(d)$ , lag  $P$ , window
45      $W=P$ . Normalized by BOTH windows' energy (Cauchy-Schwarz) so  $M \leq 1$  -- a true correlation
46     coefficient,  $\sim 1$  only where the two period- $P$  windows are genuinely identical (a preamble)."""
47     prod = np.conj(x[:-P]) * x[P:]          # conj( $x[n]$ )  $x[n+P]$ 
48     ea = np.abs(x[:-P]) ** 2                # first-window energy
49     eb = np.abs(x[P:]) ** 2                 # second-window energy
50     cp = np.concatenate([[0.0 + 0j], np.cumsum(prod)])
51     ca = np.concatenate([[0.0], np.cumsum(ea)])
52     cb = np.concatenate([[0.0], np.cumsum(eb)])
53     W = P
54     pm = cp[W:] - cp[:-W]                  # Sum prod[d:d+W]
55     da = ca[W:] - ca[:-W]
56     db = cb[W:] - cb[:-W]
57     return np.abs(pm) ** 2 / (da * db + 1e-30), pm
58
59
60 def _seg_median(x: np.ndarray, lag: int, s: int, ln: int) -> float:

```

```

61     """Median of the delay-correlation metric at `lag` over the plateau [s, s+ln)."""
62     m = _metric(x, lag)[0]
63     e = min(s + ln, m.size)
64     return float(np.median(m[s:e])) if e > s else 0.0
65
66
67 def _longest_run(mask: np.ndarray) -> tuple[int, int]:
68     """(start, length) of the longest True run in a boolean array; (0, 0) if none."""
69     if not mask.any():
70         return 0, 0
71     d = np.diff(np.concatenate([[0], mask.view(np.int8), [0]]))
72     starts = np.where(d == 1)[0]
73     ends = np.where(d == -1)[0]
74     i = int(np.argmax(ends - starts))
75     return int(starts[i]), int(ends[i] - starts[i])
76
77
78 def find_preamble(x: np.ndarray, fs: float, baud_hint: float | None = None) -> Preamble | None:
79     """Detect a repeated preamble and estimate its (start, end, period, CF0, confidence).
80     Returns None when no repeated block stands out. The period is scanned DIRECTLY (not derived
81     from baud) so it works even when est_baud fails on the short burst -- exactly the target case.
82     baud_hint, if given, only tightens the scan's upper bound for speed."""
83     x = np.asarray(x, dtype=np.complex128)
84     x = x - x.mean()
85     if x.size < 256 or not np.all(np.isfinite(x)):
86         return None
87     xw = x[:_WMAX] # a preamble sits at the burst start
88     n = xw.size
89     pmax = _PMAX if not (baud_hint and baud_hint > 0) else min(_PMAX, int(12 * fs / baud_hint))
90     pmax = min(pmax, (n - 8) // 3) # need >=3 windows for the 2P harmonic check
91     best = None
92     for P in range(_PMIN, max(_PMIN + 1, pmax + 1)):
93         M, pm = _metric(xw, P)
94         run_start, run_len = _longest_run(M >= _CONF_MIN)
95         if run_len < _RUN_MIN * P:
96             continue
97         conf = float(np.median(M[run_start:run_start + run_len]))
98         # peak/valley gate: a genuinely periodic preamble correlates at 2P (harmonic) but DROPS at
99         # 1.5P (between harmonics); RRC pulse memory (which fakes a plateau at a 1-2 symbol lag) is
100         # a smooth decay -- high at both -> rejected, letting the period be scanned directly.
101         harm = _seg_median(xw, 2 * P, run_start, run_len)
102         valley = _seg_median(xw, int(round(1.5 * P)), run_start, run_len)
103         if harm < _HARM_MIN or valley > _VALLEY_MAX * conf:
104             continue
105         score = run_len * conf # long AND strong wins
106         if best is None or score > best[0]:
107             ang = float(np.angle(pm[run_start:run_start + run_len].sum())) # coherent, wrap-safe
108             cfo = ang * fs / (2 * np.pi * P)
109             end = run_start + run_len + 2 * P # past the last correlated repeat -> data
110             best = (score, Preamble(run_start, min(end, x.size), P, cfo, conf))
111     return best[1] if best else None

```

```

1  """Decision layer: modulation classification, rotation alignment, lock quality, SNR."""
2
3  from dataclasses import dataclass
4
5  import numpy as np
6
7  from .constellations import ideal_points, mod_symmetry
8  from .dsp import _kmeans1d
9
10 # Amplitude-ring separators, calibrated through the real front-end (seeds 0-3, snr 14-25):
11 # 16qam <=0.80 and 64qam <=0.99 vs 32qam >=1.16 on s3*s9/s5**2.
12 _D32 = 1.07
13 _C42_PSK = 0.80 # constant-modulus 4th cumulant: qpsk ~1.0, QAM <0.7
14 _CM_TOL = 0.2 # 1-ring spread guard for constant-modulus signals
15 _R39 = 2.7 # s3/s9: 16qam ~2.1, 64qam >=3.0
16
17
18 def _agc(z: np.ndarray) -> np.ndarray:
19     """Scale to unit average power."""
20     return z / np.sqrt(np.mean(np.abs(z) ** 2))
21
22
23 def _c42(z: np.ndarray) -> float:
24     """Magnitude of the normalized 4th-order cumulant C42 (rotation invariant)."""
25     z = _agc(z)
26     p = np.mean(np.abs(z) ** 2)
27     return float(abs(np.mean(np.abs(z) ** 4) - abs(np.mean(z**2)) ** 2 - 2 * p**2))
28
29
30 def _spread(a: np.ndarray, k: int) -> float:
31     """RMS within-cluster spread from the shared 1-D k-means (sorted-value ring test)."""
32     return _kmeans1d(a, k)[2]
33
34
35 def classify(z: np.ndarray, symmetry: int) -> str:
36     """Symmetry 2->bpsk, 8->8psk; 4-> qpsk/16qam/32qam/64qam via C42 then amplitude rings.
37
38     Not blind classes (documented): oqpsk collides with square-QAM on every ring/cumulant
39     feature; pi4dqpsk's 4th power carries a pi-per-symbol rotation, so the carrier estimate
40     absorbs baud/8 and it arrives here looking exactly like qpsk (its bits still decode
41     correctly via differential demapping).
42     """
43     if symmetry == 2:
44         return "bpsk"
45     if symmetry == 8:
46         return "8psk"
47     z = _agc(z)
48     if _c42(z) >= _C42_PSK:
49         return "qpsk"
50     a = np.sort(np.abs(z))
51     s1 = _spread(a, 1)
52     if s1 <= _CM_TOL: # constant-modulus guard (a PSK cloud that slipped the C42 gate)
53         return "qpsk"
54     s3, s5, s9 = _spread(a, 3), _spread(a, 5), _spread(a, 9)
55     if s3 * s9 / (s5 * s5 + 1e-18) > _D32: # only a 5-ring cross collapses at k=5
56         return "32qam"
57     # amplitude-ring ratio, NEVER best-fit EVM (a denser grid always fits closer)
58     return "16qam" if s3 / (s9 + 1e-12) < _R39 else "64qam"
59
60

```

```

61 _SNR_CEIL = 40.0 # above this the moments cannot resolve the noise term
62
63
64 def snr_m2m4(z: np.ndarray, mod: str) -> float:
65     """Blind symbol-SNR (dB) from the 2nd/4th moments, corrected for the
66     constellation kurtosis. Saturates at _SNR_CEIL (noise below the moments'
67     resolution); NaN when the moments leave the valid region entirely."""
68     ka = np.mean(np.abs(ideal_points(mod)) ** 4) # unit-power points => already /M2**2
69     m2, m4 = np.mean(np.abs(z) ** 2), np.mean(np.abs(z) ** 4)
70     disc = 2 * m2**2 - m4
71     if disc <= 0:
72         return float("nan")
73     s = np.sqrt(disc / (2 - ka))
74     n = m2 - s
75     return min(float(10 * np.log10(s / n)), _SNR_CEIL) if n > 0 else _SNR_CEIL
76
77
78 def _nearest_dist(w: np.ndarray, pts: np.ndarray) -> np.ndarray:
79     """Distance to nearest ideal point. Running min over the M points instead of a
80     (... , M) broadcast temporary: identical values, far less memory, ~1.5-5x faster."""
81     d = np.abs(w - pts[0])
82     for p in pts[1:]:
83         d = np.minimum(d, np.abs(w - p))
84     return d
85
86
87 def align(z: np.ndarray, mod: str) -> np.ndarray:
88     """AGC + resolve the M-fold rotation ambiguity by minimizing nearest-point distance."""
89     z = _agc(z)
90     pts = ideal_points(mod)
91     sub = z[:1000]
92     ang = np.linspace(0, 2 * np.pi / mod_symmetry(mod), 64, endpoint=False)
93     rot = sub[None, :] * np.exp(1j * ang)[: , None]
94     cost = _nearest_dist(rot, pts).mean(axis=1)
95     i = int(np.argmin(cost))
96     c0, c1, c2 = cost[(i - 1) % 64], cost[i], cost[(i + 1) % 64] # parabolic sub-grid refine
97     denom = c0 - 2 * c1 + c2
98     # clip: on a flat cost surface (e.g. pure noise) the fit can explode
99     delta = float(np.clip(0.5 * (c0 - c2) / denom, -1, 1)) if denom else 0.0
100    return z * np.exp(1j * (ang[i] + delta * (ang[1] - ang[0])))
101
102
103 @dataclass
104 class Quality:
105     evm: float
106     mer_db: float
107     lock: float
108     occupied: int
109
110
111 def quality(z: np.ndarray, mod: str) -> Quality:
112     """Alignment + EVM/MER, plus an occupancy-gated lock score in [0, 100]."""
113     z = align(z, mod)
114     pts = ideal_points(mod)
115     m = pts.size
116     idx = np.argmin(np.abs(z[:, None] - pts[None, :]), axis=1)
117     d = pts[idx]
118     evm = float(np.sqrt(np.mean(np.abs(z - d) ** 2) / np.mean(np.abs(d) ** 2)))
119     mer_db = float(-20 * np.log10(evm + 1e-12))
120     thr = max(3, 0.6 * len(z) / m)

```

```
121     occupied = int(np.sum(np.bincount(idx, minlength=m) >= thr))
122     lock = float(np.clip((mer_db - 6) / 19 * 100, 0, 100) * (occupied / m))
123     return Quality(evm, mer_db, lock, occupied)
```

```

1  """Optional numba acceleration for the sequential feedback loops (adaptive equalizer
2  taps, decision-directed carrier) that CANNOT be vectorized -- each update depends on
3  the previous output, so they are the only python-loop hot spots in an otherwise
4  vectorized pipeline. Numba is an OPTIONAL extra ('pip install signus[fast]'): when it
5  is absent 'njit' degrades to a no-op and the identical bodies run as plain numpy, so
6  results are unchanged and the numpy+scipy-only baseline still works. The kernels use
7  the same expressions as the pure-numpy originals, so the numba path stays numerically
8  equivalent (verified: byte-identical 'w @ win', argmin/abs/angle exact)."""
9
10 import os
11
12 import numpy as np
13
14 HAVE_NUMBA = False
15 if not os.environ.get("SIGNUS_NO_NUMBA"): # escape hatch: force the pure-numpy path
16     try:
17         import numba
18         njit = numba.njit(cache=True, fastmath=False) # fastmath off: keep IEEE results
19         HAVE_NUMBA = True
20     except ImportError: # numba absent: same bodies run as plain numpy (slower, identical)
21         pass
22 if not HAVE_NUMBA:
23     def njit(fn):
24         return fn
25
26
27 @njit
28 def cma_dd_equalize(x, pts, r2, n_taps, mu_cma, mu_dd, warmup):
29     """Symbol-spaced CMA acquisition -> decision-directed LMS -> converged refilter."""
30     n = x.size
31     mid = n_taps // 2
32     xp = np.concatenate((np.zeros(mid, dtype=np.complex128), x,
33                          np.zeros(mid, dtype=np.complex128)))
34     w = np.zeros(n_taps, dtype=np.complex128)
35     w[mid] = 1.0
36     out = np.empty(n, dtype=np.complex128)
37     warm = min(warmup, n)
38     for i in range(warm): # Phase 1: CMA (blind, modulus-only)
39         win = xp[i:i + n_taps]
40         y = w @ win
41         w = w - mu_cma * (y * (abs(y) ** 2 - r2)) * np.conj(win)
42     for i in range(warm, n): # Phase 2: decision-directed LMS
43         win = xp[i:i + n_taps]
44         y = w @ win
45         d = pts[np.argmin(np.abs(y - pts))]
46         w = w - mu_dd * (y - d) * np.conj(win)
47     for i in range(n): # refilter whole record with final taps
48         win = xp[i:i + n_taps]
49         y = w @ win
50         d = pts[np.argmin(np.abs(y - pts))]
51         w = w - mu_dd * (y - d) * np.conj(win)
52         out[i] = y
53     return out
54
55
56 @njit
57 def cma_dd_equalize_fse(x, pts, r2, n_taps, mu_cma, mu_dd, warmup):
58     """T/2 fractionally-spaced: 2 CMA passes -> 4 DD passes -> tracked output pass."""
59     nsym = x.size // 2
60     mid = n_taps // 2

```

```

61     xp = np.concatenate((np.zeros(mid, dtype=np.complex128), x,
62                             np.zeros(n_taps, dtype=np.complex128)))
63     w = np.zeros(n_taps, dtype=np.complex128)
64     w[mid] = 1.0
65     out = np.empty(nsym, dtype=np.complex128)
66     warm = min(warmup, nsym)
67     for _ in range(2):                                     # CMA acquisition, 2 data-reuse passes
68         for i in range(warm):
69             win = xp[2 * i:2 * i + n_taps]
70             y = w @ win
71             w = w - mu_cma * (y * (abs(y) ** 2 - r2)) * np.conj(win)
72     for _ in range(4):                                     # decision-directed refinement
73         for i in range(nsym):
74             win = xp[2 * i:2 * i + n_taps]
75             y = w @ win
76             d = pts[np.argmin(np.abs(y - pts))]
77             w = w - mu_dd * (y - d) * np.conj(win)
78     for i in range(nsym):                                   # final tracked output pass
79         win = xp[2 * i:2 * i + n_taps]
80         y = w @ win
81         d = pts[np.argmin(np.abs(y - pts))]
82         w = w - mu_dd * (y - d) * np.conj(win)
83         out[i] = y
84     return out
85
86
87 @njit
88 def dd_carrier(z, pts, alpha, beta):
89     """Per-symbol decision-directed PI carrier loop (general PSK/QAM, not PSK Costas)."""
90     out = np.empty(z.size, dtype=np.complex128)
91     phase = 0.0
92     freq = 0.0
93     for i in range(z.size):
94         y = z[i] * np.exp(-1j * phase)
95         d = pts[np.argmin(np.abs(y - pts))]
96         e = np.angle(y * np.conj(d))
97         out[i] = y
98         phase += freq + alpha * e
99         freq += beta * e
100    return out

```

```

1  """FSK/CPM receive path: constant-envelope gate + frequency-discriminator demod.
2  Runs on the DC-blocked analytic burst, before any carrier work (linear front-end
3  does not apply). Imports only constellation reference data (anti-shared-bug rule)."""
4
5  import numpy as np
6  from scipy.ndimage import uniform_filter1d
7  from scipy.signal import firwin, resample_poly, welch
8
9  from .constellations import fsk_levels, to_bits
10 from .dsp import _kmeans1d, _parab, analytic, mix
11
12 _CV_MAX = 0.24      # envelope coeff-of-variation ceiling. Recalibrated on a broad grid: real
13 # FSK (all h x SNR>=10 x high baud) tops out at cv 0.224, but constant-modulus PSK at very
14 # high baud (fs/baud~3) or near +-fs/2 dips to cv>=0.250 and used to slip the old 0.30 gate
15 # (false FSK). 0.24 sits in the 0.026 gap -> keeps every real FSK, rejects the PSK misfires.
16 _SEP_MIN = 3.1      # instantaneous-freq 2-means separation floor (FSK>=3.39, rest<=2.85)
17 _K4_RATIO = 2.6      # sp(k=2)/sp(k=4) collapse above which 4 levels beat 2
18 _MIN_SYMS = 8        # fewer symbols than this cannot cluster 2/4 levels (empty-quantile crash)
19 _PROM_MIN = 10.0     # a symbol-rate line peak/median below this is unreliable -- BUT only confident-
20 #                    wrong when paired with too few symbols (a long low-index burst has a weak line
21 #                    yet a correct baud). The sweep sits at prom 23-73.
22 _NSYM_MIN = 200      # ...so reject a weak line only below this symbol count. A 100 floor was tried
23 #                    (to recover more short bursts) but an independent grid found weak-prominence
24 #                    half-rate folds in the 100-199 band that still cluster tight enough to lock >=60
25 #                    (msk h=0.5 baud 24-32% off, lock 61-65) -- confident-wrong. 200 keeps them out;
26 #                    precision over recall (the short-burst confident-wrong is the cardinal sin).
27 _DECIM_FRAC = 0.30   # decimate a heavily-oversampled burst so the tones fill ~this of Nyquist
28 _DECIM_MIN = 3        # ...but only when that needs a factor >= this, so the sps~10 demod grid
29 #                    (fsk2/fsk4/msk baud=fs/10) is untouched -> the sweep stays byte-identical
30
31
32 def _bw99(x: np.ndarray, fs: float) -> float:
33     """Two-sided 99%-power occupied bandwidth (Hz), noise-pedestal subtracted."""
34     f, p = welch(x, fs=fs, nperseg=min(4096, x.size), return_onesided=False, detrend=False)
35     f, p = np.fft.fftshift(f), np.fft.fftshift(p)
36     p = np.maximum(p - np.median(p), 0.0)
37     cs = np.cumsum(p) / (p.sum() + 1e-30)
38     lo = f[min(np.searchsorted(cs, 0.005), f.size - 1)] # clamp BOTH edges: a degenerate PSD
39     hi = f[min(np.searchsorted(cs, 0.995), f.size - 1)] # (all-zero after floor subtraction)
40     return float(abs(hi - lo))                          # otherwise indexes one past the end
41
42
43 def _dcblock(x: np.ndarray) -> np.ndarray:
44     """Complex128 analytic burst, DC-blocked (front-end convention)."""
45     x = analytic(x)
46     return x - x.mean()
47
48
49 def _ifreq(x: np.ndarray, fs: float) -> np.ndarray:
50     """Instantaneous frequency (Hz) from sample-to-sample phase differences."""
51     return np.angle(x[1:] * np.conj(x[:-1])) * fs / (2 * np.pi)
52
53
54 def _sep(a: np.ndarray) -> float:
55     """2-means between/within separation ratio of a 1-D sample set."""
56     c, _, sp = _kmeans1d(a, 2)
57     return abs(c[1] - c[0]) / (sp + 1e-9)
58
59
60 def _est_baud(f: np.ndarray, fs: float) -> tuple[float, float]:

```

```

61 """Symbol rate from the spectral line of |diff(smoothed f)| (transition impulses),
62 harmonic-folded to the fundamental (2-level lines alias to 2*baud). Returns
63 (baud, prominence) where prominence = peak/median of the in-band line: a weak line
64 (few symbols, no clean symbol clock) means the baud is UNRELIABLE -> caller rejects."""
65 d = np.abs(np.diff(uniform_filter1d(f, 2)))
66 d = d - d.mean()
67 n = 1 << int(np.ceil(np.log2(d.size)))
68 spec = np.abs(np.fft.rfft(d * np.hanning(d.size), n))
69 freqs = np.fft.rfftfreq(n, 1 / fs)
70 df = freqs[1] - freqs[0]
71 lo, hi = 0.002 * fs, 0.45 * fs
72 idx = np.where((freqs >= lo) & (freqs <= hi))[0]
73 prom = float(spec[idx].max() / (np.median(spec[idx]) + 1e-20)) if idx.size else 0.0
74 k = idx[int(np.argmax(spec[idx]))]
75
76 def _hp(kk: int) -> float:                                     # harmonic-comb energy: the TRUE fundamental
77     """return float(sum(spec[m * kk] for m in range(1, 6) if m * kk < spec.size)) # captures most
78     # in-range harmonics ONLY: clamping out-of-range ones to the last bin multi-counted the
79     # Nyquist bin, inflating _hp at a 2*baud peak enough to veto the legitimate fold -> 2x baud.
80
81 for div in (3, 2):
82     ks = int(round(freqs[k] / div / df))
83     if freqs[ks] >= lo:
84         w = spec[max(0, ks - 2):ks + 3]
85         kk = max(0, ks - 2) + int(np.argmax(w)) if w.size else k
86         # fold to the sub-harmonic ONLY if it is genuinely the fundamental -- its harmonic comb
87         # must beat the current peak's. A real baud/2 fundamental captures MORE harmonic lines
88         # than the 2*baud peak; spurious low-h/low-SNR energy at baud/2 does not (it once
89         # slipped the 0.34 gate and folded baud -> baud/2: a confident WRONG decode). The comb
90         # guard vetoes that. On the sps~10 grid the fold never fired (0.34 unmet) -> sweep same.
91         if w.size and w.max() > 0.34 * spec[k] and _hp(kk) >= _hp(k):
92             k = kk
93     return float(freqs[k] + _parab(20 * np.log10(spec + 1e-20), k) * df), prom
94
95
96 def fsk_gate(x: np.ndarray, fs: float) -> bool:
97     """True when the burst looks like FSK/CPM: near-constant envelope AND a bimodal
98     instantaneous frequency. Calibrated on generate() over all GEN_MODS x seeds 0..3
99     x snr {25,15}: every linear mod fails one test with margin -- bpsk/dbpsk have
100     cv~0.40 (>_CV_MAX), the rest have freq-sep<=2.85 (<_SEP_MIN); fsk2/fsk4/msk keep
101     cv<=0.22 and freq-sep>=3.39 down to snr 10 (margins ~0.08 and ~0.29).
102     Recenter by the mean instantaneous frequency first: at a large carrier offset a
103     linear mod's phase-transition spikes wrap past +/-fs/2 and fake a second mode."""
104     x = _dcblock(x)
105     a = np.abs(x)
106     if a.std() / (a.mean() + 1e-12) >= _CV_MAX:
107         return False
108     f = _ifreq(x, fs)
109     x = x * np.exp(-2j * np.pi * np.mean(f) / fs * np.arange(x.size))
110     return bool(_sep(uniform_filter1d(_ifreq(x, fs), 4)) > _SEP_MIN)
111
112
113 def analyze_fsk(x: np.ndarray, fs: float) -> dict:
114     """Full FSK/MSK demod by frequency discrimination. Returns mod, fc, baud, h,
115     lock (0-100), level_freqs (Hz, sorted, centered), symbols (normalized f per
116     symbol) and Gray-demapped bits."""
117     x = _dcblock(x)
118     f = _ifreq(x, fs)
119     # A carrier OFFSET (tones sit at fc +/- dev, not around DC) corrupts the symbol-rate estimate two
120     # ways -- off-centre it trips _est_baud's harmonic fold, and the tones fall outside the decim

```

```

121 # LPF passband -- either way a confident WRONG baud. Recentre the discriminator on the carrier
122 # (mean instantaneous freq) first; only for a significant offset, so a centred burst
123 # (fc~0, the whole demod/sweep grid) is byte-identical. Levels are already centred downstream.
124 fc0 = float(np.mean(f))
125 if abs(fc0) > 0.005 * fs:
126     x = mix(x, fs, fc0)
127     f = _ifreq(x, fs)
128 else:
129     fc0 = 0.0
130 baud, prom = _est_baud(f, fs)
131 # heavy oversampling (fs/baud >> 10) buries the symbol-rate line under broadband transition
132 # artifacts -> _est_baud locks a spurious peak (baud ~25% off) while the clean tones still
133 # cluster (lock ~94): a confident WRONG decode. Re-estimate baud on a decimated copy (tones ->
134 # ~_DECIM_FRAC of Nyquist, as the survey channelizer does). Gated at _DECIM_MIN so the sps~10
135 # grid is untouched (fsk2/fsk4/msk baud=fs/10 -> d<3 -> no re-estimate -> sweep byte-identical).
136 bw = _bw99(x, fs)
137 d = int(_DECIM_FRAC * fs / bw) if bw > 0 else 1
138 if d >= _DECIM_MIN:
139     xd = np.convolve(x, firwin(129, min(0.9 * (fs / d) / 2, bw) / (fs / 2)), "same")
140     baud, prom = _est_baud(_ifreq(resample_poly(xd, 1, d), fs / d), fs / d)
141 if baud <= 0:
142     raise ValueError("FSK 버스트에서 심볼율을 추정할 수 없습니다 (너무 짧음)")
143 sps = fs / baud
144 f = uniform_filter1d(f, max(1, int(round(sps / 2))))
145 resid = float(f.mean())
146 fc = fc0 + resid # absolute carrier = recentre offset + in-band residual
147 fcen = f - resid
148 nsym = int(fcen.size / sps) - 1
149 # a WEAK symbol-rate line (prom < _PROM_MIN) is unreliable ONLY when symbols are also too few to
150 # average (nsym < _NSYM_MIN): a short burst's spurious peak still clusters tight (few points ->
151 # high lock) = a confident WRONG baud. A long low-index (h~0.35) burst also has a weak line but
152 # MANY symbols -> its baud is correct -> NOT rejected. Sweep: nsym~3000/prom 23-73 -> untouched.
153 if nsym < _MIN_SYMS or (prom < _PROM_MIN and nsym < _NSYM_MIN):
154     raise ValueError(f"FSK 버스트가 너무 짧거나 심볼율이 불확실합니다 ({nsym} 심볼)")
155 ctr = (np.arange(nsym) + 0.5) * sps
156
157 # symbol sampling phase: the offset whose sampled levels separate best
158 v, best = fcen[:nsym], -1.0
159 for o in np.linspace(0, sps, 8, endpoint=False):
160     s = fcen[np.clip((ctr + o).astype(int), 0, fcen.size - 1)]
161     sep = _sep(s)
162     if sep > best:
163         best, v = sep, s
164
165 # level count via spread-collapse (k=2 vs k=4), then cluster
166 k = 4 if _kmeans1d(v, 2)[2] / (_kmeans1d(v, 4)[2] + 1e-9) > _K4_RATIO else 2
167 c, lab, spread = _kmeans1d(v, k)
168 order = np.argsort(c)
169 ranks = np.argsort(order)[lab] # per-symbol level index, 0 = lowest freq
170 csort = c[order]
171 mid = float(csort.mean()) # debias center: symmetric levels sum ~0
172 fc, csort, v = fc + mid, csort - mid, v - mid
173 spacing = float(np.mean(np.diff(csort)))
174 h = spacing / baud
175
176 mod = "msk" if (k == 2 and abs(h - 0.5) < 0.15) else f"fsk{k}"
177 _, glabels = fsk_levels(mod)
178 bits = to_bits(glabels[ranks], mod).astype(np.uint8)
179 lock = float(np.clip((spacing / (spread + 1e-9) - 1.5) / 6.5 * 100, 0, 100))
180 return dict(mod=mod, fc=fc, baud=baud, h=h, lock=lock, level_freqs=csort,

```

181 |

symbols=v / (spacing / 2), bits=bits)

```

1  """Blind linear-chirp / CSS (LoRa) detection + characterization. A constant-envelope
2  signal whose instantaneous frequency ramps linearly de-chirps to a tone: the delay-
3  conjugate  $x[t] \cdot \text{conj}(x[t-\tau])$  is a beat tone at  $\mu \cdot \tau$  (NONZERO), unlike FSK/CW (beat at
4  DC), PSK/QAM (not constant-envelope), or FM-voice/noise (broadband). Characterize-only --
5  reported like analog/tone, NEVER demodulated: blind CSS symbol decode needs preamble
6  CF0/ST0 sync plus LoRa's reverse-engineered Gray/interleave/Hamming/whitening framing.
7  Calibrated on gen chirps vs every other family: chirp beat-PAR $\geq$ 936, non-chirp $\leq$ 264."""
8
9  import numpy as np
10 from scipy.ndimage import uniform_filter1d
11 from scipy.signal import welch
12
13 _PAR_MIN = 400.0    # delay-conjugate beat-tone peak-to-mean floor (chirp $\geq$ 936, non-chirp $\leq$ 264)
14
15
16 def _bw99(x: np.ndarray, fs: float) -> float:
17     """99%-power occupied bandwidth (a chirp fills its band ~flat). Floor-subtracted so the
18     noise pedestal in the channel's LPF passband does not inflate the width."""
19     f, p = welch(x, fs=fs, nperseg=min(4096, x.size), return_onesided=False, detrend=False)
20     f, p = np.fft.fftshift(f), np.fft.fftshift(p)
21     p = np.maximum(p - np.median(p), 0.0)    # remove the noise pedestal
22     cs = np.cumsum(p) / (p.sum() + 1e-30)
23     lo = f[min(np.searchsorted(cs, 0.005), f.size - 1)]    # clamp BOTH edges: a degenerate PSD
24     hi = f[min(np.searchsorted(cs, 0.995), f.size - 1)]    # (all-zero after floor subtraction)
25     return float(abs(hi - lo))    # otherwise indexes one past the end
26
27
28 def _beat(x: np.ndarray, fs: float) -> tuple[float, float]:
29     """(peak-to-mean,  $\mu$ [Hz/s]) of the DC-nulled delay-conjugate spectrum."""
30     x = x - x.mean()
31     tau = max(1, x.size // 200)
32     y = x[tau:] * np.conj(x[:-tau])
33     nf = 1 << int(np.ceil(np.log2(y.size)))
34     spec = np.abs(np.fft.fftshift(np.fft.fft((y - y.mean()) * np.hanning(y.size), nf))) ** 2
35     fr = np.fft.fftshift(np.fft.fftfreq(nf, 1 / fs))
36     spec[np.abs(fr) < 0.002 * fs] = 0.0    # null DC band: FSK/CW/tone beat here, chirps do not
37     k = int(np.argmax(spec))
38     return float(spec[k] / (spec.mean() + 1e-30)), float(fr[k] * fs / tau)
39
40
41 def is_chirp(x: np.ndarray, fs: float) -> bool:
42     """True when a channel is a linear chirp / CSS (LoRa/FMCW). The beat-tone PAR is the
43     discriminator: a huge margin (chirp $\geq$ 936 vs PSK/QAM $\leq$ 94, FSK $\leq$ 91, FM $\leq$ 264, noise $\leq$ 13)
44     makes an envelope-CV pre-gate redundant AND harmful (it rejects noisy-but-clear chirps)."""
45     if x.size < 256:
46         return False
47     return _beat(x, fs)[0] >= _PAR_MIN
48
49
50 _SWEEP_PM_MAX = 1.45    # instantaneous-freq histogram peak-to-mean: a band-sweep stays ~1
51 _SWEEP_OCC_MIN = 0.6    # ...and fills its band; FSK sits on 2-4 discrete tones (peaky, sparse)
52
53
54 def sweeps_band(x: np.ndarray, fs: float, bins: int = 40) -> bool:
55     """True when the instantaneous frequency SWEEPS the channel (linear chirp / CSS) rather
56     than hopping between a few discrete FSK tones. A linear FMCW ramp reads bimodal to
57     fsk_gate AND trips is_chirp's beat test, so triage needs this to tell a genuine band-sweep
58     from FSK: a sweep's IF histogram is flat and fully occupied; FSK's spikes at its tones.
59     Calibrated on extracted channels: 0/222 FSK/MSK (snr 8-40) pass, 214/216 FMCW + 60/60 LoRa
60     pass (misses only the narrowest, gentlest ramps -- which stay 'fsk', never a regression).

```

```

61     BOTH the raw and a lightly-smoothed IF must look like a sweep: at moderate SNR (~14) noise
62     flattens integer-h FSK's raw histogram to sweep-like pm ~1.44 (knife-edge), but smoothing
63     restores its tone spikes (pm 2.5+) while a genuine ramp stays flat (pm ~1.1) -- and at LOW
64     SNR smoothing can flatten FSK instead, which the raw view still catches. The conjunction
65     only ever narrows 'sweep', so every calibrated chirp above keeps its label (margin >=0.3)."""
66     if x.size < 256:
67         return False
68     x = x - x.mean()
69     f0 = np.diff(np.unwrap(np.angle(x))) * fs / (2 * np.pi)
70     for f in (f0, uniform_filter1d(f0, 4)):
71         lo, hi = np.percentile(f, 1), np.percentile(f, 99)
72         if hi - lo < 1:
73             return False
74         fv = f[(f >= lo) & (f <= hi)]
75         h = np.histogram(fv, bins=bins)[0] / fv.size
76         pm = h.max() / (h.mean() + 1e-30)          # discrete tones spike this; a sweep ~1
77         occ = float((h > 0.005).mean())           # a sweep fills the band; tones occupy few bins
78         if not (pm < _SWEEP_PM_MAX and occ >= _SWEEP_OCC_MIN):
79             return False
80     return True
81
82
83 def analyze_chirp(x: np.ndarray, fs: float) -> dict:
84     """Characterize a chirp: {mu[Hz/s], up, bw, par, sf, rs, tsym}. A LoRa hypothesis
85     (sf, symbol rate, symbol time) is filled when bw^2/|mu| snaps to 2^(7..12); otherwise
86     it stays a generic linear chirp / FMCW. bw is measured on the channel, not asserted."""
87     par, mu = _beat(x, fs)
88     bw = _bw99(x, fs)
89     sf = rs = tsym = None
90     if mu != 0 and bw > 0:
91         r = float(np.log2(bw * bw / abs(mu)))
92         if 7 <= round(r) <= 12 and abs(r - round(r)) < 0.3:    # power-of-two chip count -> LoRa
93             sf = int(round(r))
94             rs = abs(mu) / bw
95             tsym = 2 ** sf / bw
96     return {"mu": mu, "up": bool(mu > 0), "bw": float(bw),
97            "par": par, "sf": sf, "rs": rs, "tsym": tsym}

```

```

1  """End-to-end blind demodulation. Two families:
2      linear (PSK/QAM): front-end -> carrier -> baud -> matched filter -> timing ->
3      ..... classify -> fine sync -> [equalizer rescue] -> quality
4      fsk (CPFSK/MSK): frequency discriminator (gated by constant envelope)
5  Classification runs BEFORE fine sync so the decision-directed loop knows the
6  constellation. Differential demapping is an option, not a detection: dqpsk and
7  qpsk share a constellation."""
8
9  import json
10 from dataclasses import dataclass, field
11
12 import numpy as np
13
14 from . import classify as cl
15 from . import dsp, triage
16 from .channelize import extract
17 from .chirp import analyze_chirp, is_chirp, sweeps_band
18 from .constellations import demap_bits, demap_diff_bits, mod_order
19 from .detect import Detection, detect
20 from .eq import equalize, equalize_fse
21 from .fsk import analyze_fsk, fsk_gate
22 from .sigio import Meta, parse_name, read
23 from .spectrum import spectrum, waterfall
24 from .sync import find_preamble
25
26 _BITS_CAP = 65536
27 _EQ_LOCK = 60.0    # below this a linear signal is worth an equalizer attempt
28 _EQ_GAIN = 3.0     # ...keep it only if lock improves by this much
29 _EQ_FLOOR = 50.0   # ...and clears this floor, so a wrong-mod fit is never locked in
30 _EQ_QAM_FLOOR = 60.0 # ...but a DENSE-QAM (32/64) eq fit must clear a higher bar: the DD loop can
31 #                  drag a lower-order cloud onto a 32/64 lattice at a marginal lock (~51), a
32 #                  confident WRONG order. Genuine multipath recoveries clear ~72; the sweep's
33 #                  eq cases are qpsk/16qam (never 32/64 via eq) so this bar is byte-identical.
34 _BAUD_GAIN = 5.0   # an in-band baud hypothesis must beat the global one by this lock
35 _SHORT_LOCK = 60.0 # below this, retry the baud line with fewer blocks (short-burst rescue)
36 _SYNC_LOCK = 60.0  # below this, try a repeated-preamble carrier/timing sync (packetized bursts)
37 _ALIAS_MARGIN = 10.0 # a preamble carrier alias must beat the next by this lock, else ambiguous.
38 #                  This is the LOAD-BEARING rotation guard: even when the coarse anchor itself
39 #                  aliases and validates a rotation at ~80 lock, that rotation wins by a thin
40 #                  margin (~8) -- so a 10-lock margin refuses it. Because the margin catches it,
41 #                  the lock floor can stay modest (78, not 82) -> ~5% more genuine recoveries.
42 _SYNC_ACCEPT = 78.0 # ...AND clear this absolute lock: below it correct/rotated aliases overlap. A
43 #                  65 floor was tried (to recover more moderate bursts) but an independent
44 #                  snr-12 grid found 8psk rotated aliases locking 71-72 in the opened 65-78 band
45 #                  (carrier off by one baud/L step -> confident garbage bits). Even 78 leaks a
46 #                  a few hard-corner rotations (a separate pre-existing gate limit): hold at 78.
47 _SYNC_ADIST = 1.5  # ...AND sit within this many alias steps of the coarse est_carrier fc -- a
48 #                  soft backstop to the margin: it catches the one rotation the margin misses,
49 #                  but tuned LOOSE (1.5, not 0.75) so it does not needlessly reject genuine
50 #                  large-offset recoveries. The three together give 0 confident-wrong across
51 #                  1920 hard-corner bursts at the max recall this gate reaches.
52 _DIFF_OF = {"bpsk": "dbpsk", "qpsk": "dqpsk"} # pi4dqpsk arrives as qpsk -> dqpsk
53
54
55 @dataclass
56 class Result:
57     meta: Meta
58     family: str          # 'linear' | 'fsk'
59     burst: tuple[int, int]
60     fc: float

```

```

61     baud: float
62     mod: str
63     lock: float
64     symbols: np.ndarray          # aligned symbols (linear) / normalized levels (fsk)
65     bits: np.ndarray
66     iq_corr: np.ndarray          # exportable corrected baseband
67     burst_x: np.ndarray = field(repr=False, default=None) # for spectrum views
68     symmetry: int = 0
69     carrier_ambiguous: bool = False
70     baud_conf: float = 0.0
71     rolloff: float | None = None
72     h: float | None = None       # FSK modulation index
73     evm: float = float("nan")
74     mer_db: float = float("nan")
75     occupied: int = 0
76     snr_est: float = float("nan")
77     eq_applied: bool = False
78     eq_mode: str | None = None   # 'sym' | 'fse'
79     alias_resolved: bool = False
80     baud_fallback: bool = False
81     bursts: list = field(default_factory=list)
82     burst_idx: int = 0
83     preamble: object = None     # sync.Preamble when a repeated preamble drove the decode
84
85     def to_json(self, max_points: int = 6000, views: bool = True) -> dict:
86         z = np.nan_to_num(self.symbols) # a dead/DC capture -> NaN symbols -> invalid JSON
87         if z.size > max_points: # keep TIME ORDER: the UI animates this sequence
88             z = z[np.linspace(0, z.size - 1, max_points).astype(int)]
89         det = {"fc": round(self.fc, 3), "baud": round(self.baud, 3), "mod": self.mod,
90              "symmetry": self.symmetry, "carrier_ambiguous": self.carrier_ambiguous,
91              "baud_conf": round(self.baud_conf, 1),
92              "alias_resolved": self.alias_resolved,
93              "baud_fallback": self.baud_fallback}
94         det["rolloff"] = None if self.rolloff is None else round(self.rolloff, 3)
95         rf = self.meta.rf_center # real RF = capture centre + baseband fc
96         det["rf_hz"] = None if rf is None else round(rf + self.fc, 3)
97         if self.h is not None:
98             det["h"] = round(self.h, 3)
99         if self.preamble is not None: # a repeated preamble drove the sync
100             p = self.preamble
101             det["preamble"] = {"period": p.period, "cfo_hz": round(p.cfo_hz, 1),
102                               "conf": round(p.conf, 2), "start": p.start, "end": p.end}
103         doc = {
104             "fs": self.meta.fs, "fmt": self.meta.fmt, "rf_center": self.meta.rf_center,
105             "family": self.family, "n_samples": int(self.iq_corr.size),
106             "burst": {"start": self.burst[0], "end": self.burst[1]},
107             "bursts": [{"start": s, "end": e} for s, e in self.bursts],
108             "burst_idx": self.burst_idx,
109             "detected": det,
110             "quality": {"lock": _r(self.lock, 1) or 0.0, "mer_db": _r(self.mer_db),
111                        "evm": _r(self.evm, 4)},
112             "snr_est_db": _r(self.snr_est),
113             "eq": {"applied": self.eq_applied, "mode": self.eq_mode},
114             "constellation": {"i": np.round(z.real, 4).tolist(),
115                               "q": np.round(z.imag, 4).tolist()},
116             "bits": "".join(map(str, self.bits[:_BITS_CAP])),
117         }
118         if views and self.burst_x is not None:
119             doc["spectrum"] = spectrum(self.burst_x, self.meta.fs)
120             doc["waterfall"] = waterfall(self.burst_x, self.meta.fs)

```

```

121         return doc
122
123     def save_iq(self, path: str) -> None:
124         np.column_stack([self.iq_corr.real, self.iq_corr.imag]).astype("<f4").tofile(path)
125
126     def save_symbols(self, path: str) -> None:
127         np.save(path, self.symbols.astype(np.complex64))
128
129     def save_bits(self, path: str, packed: bool = False) -> None:
130         if packed:
131             np.packbits(self.bits).tofile(path)
132         else:
133             with open(path, "w") as fh:
134                 fh.write("".join(map(str, self.bits)))
135
136     def save_report(self, path: str) -> None:
137         with open(path, "w") as fh:
138             json.dump(self.to_json(), fh, indent=1)
139
140
141 def _r(v: float, nd: int = 2) -> float | None:
142     return None if v is None or not np.isfinite(v) else round(float(v), nd)
143
144
145 def _fine(z: np.ndarray, mod: str) -> np.ndarray:
146     """Bulk-phase removal, decision-directed carrier loop, final rotation lock."""
147     return cl.align(dsp.ddsync(cl.align(z, mod), mod), mod)
148
149
150 def _blocks(n: int) -> int:
151     """M-th-power spectra are averaged INCOHERENTLY, so a weak tone (high symmetry,
152     low SNR) wants fewer, longer segments; only long records need more for
153     stationarity. Worth ~2 dB at the 8PSK edge over a fixed blocks=4."""
154     return int(np.clip(n // 30000, 1, 8))
155
156
157 def _demod(xd: np.ndarray, fs: float, baud: float, symmetry: int) -> tuple:
158     """Matched filter + timing + classify + fine sync at one baud hypothesis."""
159     alpha = dsp.est_rolloff(xd, fs, baud)
160     ym = dsp.matched(dsp.to_sps(xd, fs, baud), 4, alpha)
161     raw = dsp.timing(ym, 4)
162     mod = cl.classify(raw, symmetry)
163     # align BEFORE ddsync: the coarse carrier leaves only micro-Hz residual, so removing
164     # the bulk phase first kills the loop transient that would corrupt the first symbols
165     syms = _fine(raw, mod)
166     return alpha, ym, raw, mod, syms, cl.quality(syms, mod)
167
168
169 def _rescue(eq_fn, z: np.ndarray, seed_mod: str, symmetry: int) -> tuple:
170     """Equalize with a seed constellation, then let the OPENED eye pick the mod:
171     heavy ISI corrupts the ring test and even the 2/8 symmetry vote, so re-classify
172     with the estimated symmetry AND the neutral order-4 gate, keep the best lock."""
173     z1 = eq_fn(z, seed_mod)
174     cand_s = []
175     for m in {cl.classify(z1, symmetry), cl.classify(z1, 4)}: # symmetry too, per the docstring
176         s = _fine(z1 if m == seed_mod else eq_fn(z, m), m)
177         cand_s.append((cl.quality(s, m), s, m))
178     best = max(cand_s, key=lambda c: c[0].lock)
179     # Occam de-fold for the PSK point-doubling: CMA is modulus-only, so a bpsk signal under a
180     # multipath echo ALSO fits a qpsk ring at a marginally higher lock (phantom 2->4 points).

```

```

181     # A bpsk candidate that clears the accept floor cannot be an over-fit of qpsk, so prefer it.
182     # Only fires when symmetry==2 put bpsk in the set; a genuine qpsk reads symmetry 4 -> the
183     # candidate set is the qpsk singleton -> this branch is unreachable and it stays untouched.
184     lo = [c for c in cands if c[2] == "bpsk" and c[0].lock >= _EQ_FLOOR]
185     if lo and best[2] == "qpsk":
186         return lo[0]
187     return best
188
189
190 def analyze(x: np.ndarray, meta: Meta, diff: bool = False,
191            burst: int | None = None) -> Result:
192     """Run the blind chain. `diff` demaps phase transitions (D-BPSK/D-QPSK);
193     `burst` selects one detected burst (default: the most energetic)."""
194     if x.size:
195         # pathological UNIFORM scale must be fixed BEFORE analytic(): its magnitude sanitizer
196         # zeroes samples above 1e150 sample-WISE, so a whole capture near/over that ceiling was
197         # wiped wholesale (garbage mods, lock=nan at 1e154) before the rms-normalize below could
198         # ever help. The 99th percentile is robust to spike outliers (a single corrupt 1e300
199         # sample leaves it at signal scale -> untouched here, still zeroed sample-wise later);
200         # normal captures sit far inside (1e-100, 1e100) -> untouched -> sweep byte-identical.
201         scale = float(np.percentile(np.abs(x[:16]), 99)) # strided: scale detection needs the
202         # BULK magnitude, not every sample -- 16x cheaper on a large direct capture
203         if np.isfinite(scale) and scale > 0 and not (1e-100 < scale < 1e100):
204             x = x / scale
205     x = dsp.analytic(x)
206     x = x - x.mean() # DC block: LO leakage otherwise corrupts every estimator
207     if x.size < 64: # empty / trivially short: nothing to estimate, and it crashes downstream
208         raise ValueError(f"신호가 너무 짧습니다 ({x.size} 샘플)")
209     rms = float(np.sqrt(np.mean(np.abs(x) ** 2)))
210     if rms and (rms < 1e-6 or rms > 1e6): # only PATHOLOGICAL scale: normalize so the scale-variant
211         x = x / rms # spots (fsk_gate's CV epsilon, est_carrier's x**p over/
212         # underflow) stay invariant. Normal captures rms~0(1) ->
213         # untouched -> the acceptance sweep is byte-identical.
214     bursts = dsp.find_bursts(x, meta.fs)
215     if burst is None or not 0 <= burst < len(bursts):
216         burst = int(np.argmax([np.sum(np.abs(x[s:e]) ** 2) for s, e in bursts]))
217     s, e = bursts[burst]
218     xb = x[s:e] if e - s >= 64 else x
219
220     # chirp gate -- same conjunctive tie-break as triage.family, so direct analyze / survey_web
221     # single mode agree with survey's label: an FMCW/CSS sweep trips fsk_gate (bimodal IF) and
222     # would force-demodulate into confident garbage FSK. sweeps_band keeps real FSK (0/222 pass)
223     # out of this branch, so only a genuine band-sweep is refused -- never demodulated.
224     if is_chirp(xb, meta.fs) and sweeps_band(xb, meta.fs):
225         info = analyze_chirp(xb, meta.fs)
226         kind = f"LoRa 추정 SF{info['sf']}" if info["sf"] else f"μ={info['mu']:.3g} Hz/s"
227         raise ValueError(f"처프(FMCW/CSS) 신호입니다 ({kind}, bw≈{info['bw']:.3g} Hz) - "
228             f"성상도 복조 대상이 아닙니다 (서베이 모드에서 특성 보고)")
229
230     if fsk_gate(xb, meta.fs):
231         r = analyze_fsk(xb, meta.fs)
232         sym = np.asarray(r["symbols"], dtype=np.complex128)
233         return Result(meta, "fsk", (s, e), r["fc"], r["baud"], r["mod"], r["lock"],
234             sym, r["bits"], xb, burst_x=xb, h=r["h"],
235             bursts=bursts, burst_idx=burst)
236
237     fc0, symmetry, _ = dsp.est_carrier(xb, meta.fs, blocks=_blocks(xb.size))
238     fc = dsp.resolve_alias(xb, meta.fs, fc0, symmetry)
239     amb = abs(symmetry * fc) > 0.4 * meta.fs
240     xd = dsp.mix(xb, meta.fs, fc)

```

```

241
242     baud, conf = dsp.est_baud(xd, meta.fs)
243     fell = False
244     alpha, ym, raw, mod, syms, q = _demod(xd, meta.fs, baud, symmetry)
245     bw = dsp.occupied_bw(xd, meta.fs)
246     if bw and not (0.72 * bw <= baud <= 1.05 * bw):
247         # the global |x|^2 peak disagrees with the occupied band: below alpha~0.1
248         # (and under harsh channels) data junk outruns the true line. Demod the
249         # in-band hypotheses too and let lock decide, with a clear margin.
250         for lo in (0.80, 0.67):
251             b2, c2 = dsp.est_baud(xd, meta.fs, lo=lo * bw, hi=1.02 * bw)
252             if abs(b2 - baud) <= 0.01 * b2:
253                 continue
254             t = _demod(xd, meta.fs, b2, symmetry)
255             if t[5].lock > q.lock + _BAUD_GAIN:
256                 alpha, ym, raw, mod, syms, q = t
257                 baud, conf, fell = b2, c2, True
258
259     if q.lock < _SHORT_LOCK:
260         # short / low-SNR burst rescue: the default 4-block |x|^2 line splits the record
261         # too finely and locks a junk peak. Fewer, longer blocks give a stronger line (the
262         # carrier _blocks logic, applied to baud). Run both and keep only a clearly-better
263         # lock -- so a long, already-locked signal (CORE) never enters here and is untouched.
264         for nb in (2, 1):
265             b2, c2 = dsp.est_baud(xd, meta.fs, blocks=nb)
266             if abs(b2 - baud) <= 0.01 * b2:
267                 continue
268             t = _demod(xd, meta.fs, b2, symmetry)
269             if t[5].lock > q.lock + _BAUD_GAIN:
270                 alpha, ym, raw, mod, syms, q = t
271                 baud, conf, fell = b2, c2, True
272
273     eq_applied, eq_mode, pre = False, None, None
274     if q.lock < _EQ_LOCK: # ISI (multipath) is what a phase-only loop cannot fix
275         # CMA is modulus-only, so it opens the eye without knowing the constellation.
276         qe, se, me = _rescue(lambda z, m: equalize(z, m), raw, mod, symmetry)
277         mode = "sym"
278         if qe.lock < max(_EQ_FLOOR, q.lock + _EQ_GAIN):
279             # symbol-spaced FIR could not invert the channel; retry T/2-spaced on
280             # the clock-tracked 2 sps stream (unaliasd spectrum, longer inverse)
281             q2, s2, m2 = _rescue(lambda z, m: equalize_fse(z, m),
282                                dsp.timing(ym, 4, out=2), mod, symmetry)
283             if q2.lock > qe.lock:
284                 qe, se, me, mode = q2, s2, m2, "fse"
285         floor = _EQ_QAM_FLOOR if me in ("32qam", "64qam") else _EQ_FLOOR
286         if qe.lock > q.lock + _EQ_GAIN and qe.lock >= floor:
287             syms, mod, q, eq_applied, eq_mode = se, me, qe, True, mode
288     elif mod in ("16qam", "32qam", "64qam"):
289         # confident square-QAM at high lock: a benign post-echo can fold a LOWER-order signal
290         # onto a QAM lattice with a clean-looking eye, so the low-lock rescue above never fires.
291         # Equalize once and accept ONLY if a strictly lower order emerges -- verified never to
292         # demote a genuine QAM (0/36 across 16/32/64qam x snr x seeds), so real QAM is untouched.
293         qe, se, me = _rescue(lambda z, m: equalize(z, m), raw, mod, symmetry)
294         mode = "sym"
295         if mod_order(me) < mod_order(mod) and qe.lock < _EQ_FLOOR:
296             # a lower order emerged but the symbol-spaced eye stayed shut (echo > ~2 symbols);
297             # retry T/2 fractionally-spaced. Only runs once a de-fold is INDICATED, so a genuine
298             # QAM (no lower order) never pays for the FSE.
299             qe, se, me = _rescue(lambda z, m: equalize_fse(z, m),
300                                dsp.timing(ym, 4, out=2), mod, symmetry)

```

```

301         mode = "fse"
302     if mod_order(me) < mod_order(mod) and qe.lock >= _EQ_FLOOR:
303         syms, mod, q, eq_applied, eq_mode = se, me, qe, True, mode
304
305     if q.lock < _SYNC_LOCK:
306         # packetized-burst rescue -- LAST, after the equalizer: a repeated preamble pins the
307         # carrier/ baud/timing from only a few symbols, where the blind M-th-power estimator (needing
308         # many symbols to average) fails. Runs only if the eq rescue above did NOT lift the lock, so
309         # a multipath signal (which the eq handles, and whose ISI can fake a short period) never
310         # reaches here. Detect the preamble, mix by its CFO, demod the DATA past it, keep-best -- a
311         # random / no-preamble signal yields no plateau or a CFO that does not improve lock and is
312         # dropped (the sweep stays byte-identical). The preamble period P = L*sps supplies the baud
313         # (fs*L/P per block length L), the phase slope gives the carrier (+fs/P resolves the L-fold
314         # alias); every PSK/QAM symmetry is tried -- the right (carrier, baud, sym) locks clean.
315         ps = find_preamble(xb, meta.fs, baud_hint=baud)
316         if ps is not None:
317             # The Schmidl-Cox CFO is ambiguous modulo fs/P = baud/L: an M-PSK carrier off by baud/L
318             # rotates a whole constellation position per symbol, so the eye still looks locked but
319             # the bits shift -> a confident WRONG decode if the wrong alias is trusted. The coarse
320             # M-th-power fc lands within ~1 alias step of the truth even on these short bursts, so
321             # CENTRE the alias scan on it (k0) and sweep +-3 steps -- this reaches the correct alias
322             # for any offset (not just |fc|<baud/2); keep-best over them picks the cleanest eye.
323             step = meta.fs / ps.period
324             k0 = round((fc - ps.cfo_hz) / step)
325             cands = []
326             for b2 in sorted({round(meta.fs * L / ps.period) for L in (3, 4, 6, 8)}):
327                 for k in range(k0 - 3, k0 + 4):
328                     cfo = ps.cfo_hz + k * step
329                     data = dsp.mix(xb, meta.fs, cfo)[ps.end:]
330                     if data.size < 256:
331                         continue
332                     for sm in (2, 4, 8):
333                         t = _demod(data, meta.fs, float(b2), sm)
334                         cands.append((t[5].lock, cfo, float(b2), sm, t))
335             cands.sort(key=lambda c: -c[0])
336             # Adjacent aliases both look like clean M-PSK, so lock alone can pick the WRONG one by a
337             # hair. Require the winner to beat the best DIFFERENT-carrier candidate by a clear gap;
338             # otherwise the alias is genuinely ambiguous -> leave the honest low-lock result rather
339             # than emit a confident wrong decode. (Same-carrier baud/sym variants do not compete.)
340             # PRECISION GATE: the alias ambiguity (carrier off by baud/L) is blind-ambiguous -- a
341             # rotated alias still locks AS HIGH AS the truth (both are clean M-PSK), so lock alone
342             # cannot separate them (a rotated 8psk alias was measured locking 78 with ber 0.38). Two
343             # conjunctive conditions favour precision over recall: (1) a strong absolute lock, and
344             # (2) the winner sits within _SYNC_ADIST alias steps of the coarse est_carrier fc -- a
345             # baud/L rotation is >=1 step off the true carrier, so a far-from-anchor winner is a
346             # rotation. Genuine recoveries have a near-anchor carrier (adist<=0.5, lock 88+); on the
347             # hardest bursts the anchor itself is unreliable -> both conditions fail -> refuse.
348             if cands and cands[0][0] >= _SYNC_ACCEPT and cands[0][0] > q.lock + _EQ_GAIN:
349                 top = cands[0]
350                 other = next((c for c in cands if abs(c[1] - top[1]) > step / 2), None)
351                 anchored = abs(top[1] - fc) <= _SYNC_ADIST * step
352                 if anchored and (other is None or top[0] - other[0] >= _ALIAS_MARGIN):
353                     alpha, ym, raw, mod, syms, q = top[4]
354                     fc, baud, symmetry = top[1], top[2], top[3]
355                     eq_applied, eq_mode, pre = False, None, ps
356                     # diagnostics must describe the ACCEPTED decode, not the abandoned blind
357                     # attempt: the carrier came from the preamble (not resolve_alias -> fc0=fc
358                     # keeps alias_resolved False), the baud from the preamble period (no
359                     # spectral line -> conf 0, no fallback), and ambiguity follows the new fc.
360                     fc0, conf, fell = fc, 0.0, False

```

```

361         amb = abs(symmetry * fc) > 0.4 * meta.fs
362
363     bits = demap_diff_bits(syms, _DIFF_OF[mod]) if diff and mod in _DIFF_OF \
364     else demap_bits(syms, mod)
365     return Result(meta, "linear", (s, e), fc, baud, mod, q.lock, syms, bits, ym,
366     burst_x=xb, symmetry=symmetry, carrier_ambiguous=amb, baud_conf=conf,
367     rolloff=alpha, evm=q.evm, mer_db=q.mer_db, occupied=q.occupied,
368     snr_est=cl.snr_m2m4(syms, mod), eq_applied=eq_applied, eq_mode=eq_mode,
369     alias_resolved=abs(fc - fc0) > 1.0, baud_fallback=fell,
370     bursts=bursts, burst_idx=burst, preamble=pre)
371
372
373 def analyze_file(path: str, fs: float | None = None, fmt: str | None = None,
374 dtype: str | None = None, endian: str | None = None,
375 bitrev: bool | None = None, diff: bool = False,
376 burst: int | None = None, rf: float | None = None) -> Result:
377     """Analyze a file; explicit args override SigMF/filename tokens."""
378     from .sigio import parse_sigmf
379     m = parse_sigmf(path) or parse_name(path)
380     meta = Meta(fs or m.fs, fmt or m.fmt, dtype or m.dtype,
381     endian or m.endian, m.bitrev if bitrev is None else bitrev,
382     rf_center=rf if rf is not None else m.rf_center)
383     x, meta = read(path, meta)
384     return analyze(x, meta, diff, burst)
385
386
387 # --- wideband survey: many emitters in one capture -----
388
389 @dataclass
390 class Emitter:
391     detection: Detection
392     kind: str # linear|fsk|chirp|analog|tone|tooshort|error
393     abs_fc: float # emitter carrier vs capture centre (Hz)
394     result: Result | None = None # demod result for digital kinds
395     info: dict | None = None # characterization for non-digital kinds (e.g. chirp params)
396
397     def to_json(self) -> dict:
398         d = self.detection
399         doc = {"kind": self.kind, "abs_fc": round(self.abs_fc, 3),
400         "det": {"fc": round(d.fc, 3), "bw": round(d.bw, 3),
401         "t0": d.t0, "t1": d.t1, "snr_db": _r(d.snr_db),
402         "baud_hint": round(d.baud_hint, 1)}}
403         if self.info is not None:
404             doc["info"] = self.info
405         if self.result is not None:
406             r = self.result
407             doc.update(mod=r.mod, baud=round(r.baud, 3), lock=round(r.lock, 1),
408             mer_db=_r(r.mer_db), evm=_r(r.evm, 4), family=r.family)
409         return doc
410
411     def to_detail(self, rf_center: float | None = None) -> dict:
412         """Web drill-down payload: the box summary plus, for a digital emitter, the
413         full analyze result so the UI reuses its single-signal detail view verbatim.
414         The channel is demodulated at baseband (no rf_center), and r.fc is only the
415         residual within-channel offset -- so the emitter's absolute RF is the capture
416         centre plus abs_fc (the full offset from capture centre), injected here."""
417         doc = self.to_json()
418         if self.result is not None:
419             doc["result"] = self.result.to_json()
420         if rf_center is not None:

```

```

421         doc["result"]["detected"]["rf_hz"] = round(rf_center + self.abs_fc, 3)
422     return doc
423
424
425 @dataclass
426 class Survey:
427     meta: Meta
428     emitters: list[Emitter] = field(default_factory=list)
429
430     def to_json(self) -> dict:
431         return {"fs": self.meta.fs, "fmt": self.meta.fmt,
432                 "n_emitters": len(self.emitters),
433                 "emitters": [e.to_json() for e in self.emitters]}
434
435
436 def survey(x: np.ndarray, meta: Meta, *, diff: bool = False, **detect_kw) -> Survey:
437     """Detect every emitter in a wideband capture, extract each to its own baseband
438     channel, and demodulate the digital ones with the unchanged single-signal chain.
439     A capture holding one signal that fills the band collapses to a single emitter."""
440     xa = dsp.analytic(x)
441     xa = xa - xa.mean()
442     emitters = []
443     for d in detect(xa, meta.fs, **detect_kw):
444         ch, fs_ch = extract(xa, meta.fs, d)
445         if ch.size < 256:
446             emitters.append(Emitter(d, "tooshort", d.fc))
447             continue
448         kind = triage.family(ch, fs_ch)
449         if kind in ("linear", "fsk"):
450             # triage only gates digital vs not; analyze's own family is authoritative.
451             # Isolate each channel: a degenerate one must not abort the whole survey.
452             try:
453                 r = analyze(ch, Meta(fs_ch, "iq", "f32", "le", False), diff=diff)
454                 emitters.append(Emitter(d, r.family, d.fc + r.fc, r))
455             except Exception:
456                 emitters.append(Emitter(d, "error", d.fc))
457         else:
458             info = analyze_chirp(ch, fs_ch) if kind == "chirp" else None
459             emitters.append(Emitter(d, kind, d.fc, info=info))
460     return Survey(meta, emitters)
461
462
463 def survey_web(x: np.ndarray, meta: Meta, *, diff: bool = False) -> dict:
464     """Web survey payload for /api/survey. When detect sees <=1 emitter the capture
465     is effectively one signal -> 'single' mode returns the UNCHANGED direct analyze()
466     result (correct fc, matches /api/analyze). Otherwise 'survey' mode adds a whole-
467     capture overview waterfall and every emitter's drill-down detail. Additive: does
468     not alter survey()/Survey/Emitter, so CLI + existing tests are unaffected."""
469     xa = dsp.analytic(x)
470     xa = xa - xa.mean()
471     if len(detect(xa, meta.fs)) <= 1:
472         r = analyze(xa, meta, diff=diff)
473         out = {"mode": "single", "result": r.to_json()}
474         if len(r.bursts) > 1: # multi-burst signal -> whole-record map for burst selection
475             out["overview"] = {"n": int(xa.size), "fs": meta.fs,
476                                "waterfall": waterfall(xa, meta.fs)}
477         return out
478     sv = survey(x, meta, diff=diff)
479     return {"mode": "survey", "fs": meta.fs, "fmt": meta.fmt, "rf_center": meta.rf_center,
480            "overview": {"fs": meta.fs, "n": int(xa.size), "spectrum": spectrum(xa, meta.fs),

```

```

481         "waterfall": waterfall(xa, meta.fs)},
482         "emitters": [e.to_detail(meta.rf_center) for e in sv.emitters]}
483
484
485 def survey_file(path: str, fs: float | None = None, fmt: str | None = None,
486                dtype: str | None = None, endian: str | None = None,
487                bitrev: bool | None = None, diff: bool = False,
488                rf: float | None = None) -> Survey:
489     """Survey a capture file; explicit args override SigMF/filename tokens."""
490     from .sigio import parse_sigmf
491     m = parse_sigmf(path) or parse_name(path)
492     meta = Meta(fs or m.fs, fmt or m.fmt, dtype or m.dtype,
493                endian or m.endian, m.bitrev if bitrev is None else bitrev,
494                rf_center=rf if rf is not None else m.rf_center)
495     x, meta = read(path, meta)
496     return survey(x, meta, diff=diff)

```

```

1  """CLI: analyze FILE / gen / sweep (acceptance harness) / serve."""
2
3  import argparse
4  import sys
5
6  import numpy as np
7
8  from .constellations import (
9      GEN_MODS,
10     MODS,
11     bit_labels,
12     demap_bits,
13     demap_diff_bits,
14     family,
15     ideal_points,
16     mod_order,
17     mod_symmetry,
18 )
19 from .gen import GenParams, generate, save
20 from .pipeline import Result, analyze, analyze_file, survey_file
21 from .sigio import Meta, sidecar_read
22
23 _DTYPE_CHOICES = ("i8", "u8", "i16", "u16", "f32", "f64")
24
25 _SNR = {"bpsk": 14, "qpsk": 14, "8psk": 18, "16qam": 22, "64qam": 28}
26 _SNR_LOW = {"bpsk": 6, "qpsk": 8, "8psk": 12, "16qam": 16, "64qam": 22}
27
28
29 # --- BER scoring (blind reception leaves rotation/conjugation/offset ambiguity) ---
30
31 def ber(symbols: np.ndarray, mod: str, tx_bits: np.ndarray) -> float:
32     """Min BER over conjugation, M-fold rotation, and symbol offset (found by
33     correlating against the reconstructed TX symbol stream)."""
34     k = mod_order(mod).bit_length() - 1
35     sym = mod_symmetry(mod)
36     inv = np.empty(mod_order(mod), dtype=int)
37     inv[bit_labels(mod)] = np.arange(mod_order(mod))
38     labels = (tx_bits.reshape(-1, k) << np.arange(k - 1, -1, -1)).sum(1)
39     tx = ideal_points(mod)[inv[labels]]
40
41     best = 1.0
42     for z in (symbols, np.conj(symbols)):
43         # |correlation| is rotation-invariant -> find the symbol offset first
44         offs = range(-32, 33)
45         c = [np.abs(np.vdot(*_overlap(z, tx, o))) for o in offs]
46         o = list(offs)[int(np.argmax(c))]
47         a, b = _overlap(z, tx, o)
48         if a.size < 100:
49             continue
50         tx_b = demap_bits(b, mod)
51         for r in range(sym):
52             rx_b = demap_bits(a * np.exp(-2j * np.pi * r / sym), mod)
53             best = min(best, float(np.mean(rx_b != tx_b)))
54     return best
55
56
57 def _overlap(rx: np.ndarray, tx: np.ndarray, off: int, crop: int = 20) -> tuple:
58     """Aligned overlapping slices of rx[i] vs tx[i+off], edges cropped."""
59     i0 = max(0, -off) + crop
60     i1 = min(rx.size, tx.size - off) - crop

```

```

61     if i1 <= i0:
62         return rx[:0], tx[:0]
63     return rx[i0:i1], tx[i0 + off:i1 + off]
64
65
66 def ber_bits(rx: np.ndarray, tx: np.ndarray, k: int, span: int = 8, crop: int = 20) -> float:
67     """BER of an absolute bit stream over whole-SYMBOL shifts. No rotation search:
68     differential transitions and FSK levels carry no phase ambiguity."""
69     a, b = rx[: rx.size // k * k].reshape(-1, k), tx[: tx.size // k * k].reshape(-1, k)
70     best = 1.0
71     for sh in range(-span, span + 1): # sh = symbol offset of rx within tx
72         i0, i1 = max(0, -sh) + crop, min(len(a), len(b) - sh) - crop
73         if i1 - i0 >= 400:
74             best = min(best, float(np.mean(a[i0:i1] != b[i0 + sh:i1 + sh])))
75     return best
76
77
78 # --- sweep: the acceptance grid -----
79
80 # constellation-identical to their parent (differential is a demap option, not a class)
81 _EXPECT = {"dbpsk": "bpsk", "dqpsk": "qpsk", "pi4dqpsk": "qpsk"}
82 _DIFF = {"dbpsk": "dbpsk", "dqpsk": "dqpsk", "pi4dqpsk": "dqpsk"}
83
84
85 def _score(r: Result, p: GenParams, tx_bits: np.ndarray, core: bool) -> tuple[bool, str]:
86     want = _EXPECT.get(p.mod, p.mod)
87     fsk = family(p.mod) == "fsk"
88     e_fc = abs(r.fc - p.fc)
89     e_bd = abs(r.baud - p.baud) / p.baud
90     checks = [r.mod == want, e_bd < 0.01, r.lock >= (50 if want.endswith("qam") else 60)]
91     # pi4dqpsk's 4th power rotates pi/symbol, so the carrier estimate absorbs baud/8;
92     # the shifted band then also biases the occupied-bandwidth rolloff. Bits stay exact.
93     skew = p.mod == "pi4dqpsk"
94     if not skew:
95         checks.append(e_fc < max(300, 1e-4 * p.fs))
96     if not fsk and not skew and p.rolloff >= 0.15 and not p.taps:
97         checks.append(abs(r.rolloff - p.rolloff) < 0.08)
98     if fsk:
99         checks.append(abs(r.h - (0.5 if p.mod == "msk" else p.h)) < 0.15)
100     b = -1.0
101     if core:
102         k = mod_order(r.mod).bit_length() - 1
103         if fsk:
104             b = ber_bits(r.bits, tx_bits, k)
105         elif p.mod in _EXPECT: # differential: transitions, no rotation search
106             b = ber_bits(demap_diff_bits(r.symbols, _DIFF[p.mod]), tx_bits[k:], k)
107         else:
108             b = ber(r.symbols, want, tx_bits)
109         checks.append(b <= (0.002 if (p.taps or p.mod == "fsk4") else 0.0))
110     extra = f"h{r.h:.2f}" if fsk else f"roll{r.rolloff:.2f}"
111     msg = (f"{r.mod:>6} fc{e_fc:7.1f} baud{e_bd * 100:5.2f}% {extra} lock{r.lock:5.1f}"
112           + (" EQ" if r.eq_applied else "") + (f" ber{b:.4f}" if b >= 0 else ""))
113     return all(checks), msg
114
115
116 def _grid(tier: str, seeds: int) -> list[tuple[str, bool, GenParams]]:
117     """(label, is_core, params) cases. CORE must pass 100%; STRETCH is report-only."""
118     fs, cases = 1e6, []
119     for mod in MODS:
120         for ratio in (10.0, 7.69):

```

```

121         for sd in range(seeds):
122             rng = np.random.default_rng(sd)
123             cases.append((f"{mod}/r{ratio:g}/s{sd}", True, GenParams(
124                 mod=mod, fs=fs, baud=fs / ratio, snr=_SNR[mod], fc=0.008 * fs,
125                 phase=rng.uniform(0, 2 * np.pi), timing=rng.uniform(), seed=sd)))
126         # real passband .pcm: fc > baud*(1+roll)/2 and sym*fc < fs/2 must both hold
127         for mod, baud, fc in (("bpsk", fs / 10, 0.1 * fs), ("qpsk", fs / 10, 0.1 * fs),
128                               ("8psk", fs / 20, 0.05 * fs), ("16qam", fs / 10, 0.1 * fs)):
129             cases.append((f"{mod}/real", True, GenParams(
130                 mod=mod, fs=fs, baud=baud, snr=_SNR[mod], fc=fc, fmt="real", seed=0)))
131         cases += [("qpsk/cfo0", True, GenParams(mod="qpsk", fs=fs, baud=fs / 10, snr=14, seed=1)),
132                  ("qpsk/dc", True, GenParams(mod="qpsk", fs=fs, baud=fs / 10, snr=14,
133                  fc=0.008 * fs, dc=0.3 + 0.3j, seed=2)),
134                  ("qpsk/pad", True, GenParams(mod="qpsk", fs=fs, baud=fs / 10, snr=14,
135                  fc=0.008 * fs, pad=0.5, seed=3)),
136                  ("32qam", True, GenParams(mod="32qam", fs=fs, baud=fs / 10, snr=24,
137                  fc=0.008 * fs, seed=0))]
138         for mod, h in (("fsk2", 0.7), ("fsk4", 0.7), ("msk", 0.5)): # frequency family
139             for sd in range(min(seeds, 2)):
140                 cases.append((f"{mod}/s{sd}", True, GenParams(
141                     mod=mod, fs=fs, baud=fs / 10, snr=18, h=h, seed=sd)))
142         for mod in ("dbpsk", "dqpsk", "pi4dqpsk"): # differential: constellation of the parent
143             cases.append((f"{mod}", True, GenParams(mod=mod, fs=fs, baud=fs / 10, snr=16,
144             fc=0.008 * fs, seed=0)))
145         for mod in ("qpsk", "16qam"): # symbol-spaced multipath -> equalizer rescue
146             cases.append((f"{mod}/multipath", True, GenParams(
147                 mod=mod, fs=fs, baud=fs / 10, snr=18 if mod == "qpsk" else 24, fc=0.008 * fs,
148                 taps=(1.0, 0.45 * np.exp(1j * 0.8), 0.2j), tap_sym=1.0, seed=0)))
149         # v3.1: 1.5-symbol echo (T/2 FSE rescue), low-rolloff baud fallback, deep alias
150         cases += [("qpsk/2ray-fse", True, GenParams(mod="qpsk", fs=fs, baud=fs / 10, snr=18,
151             fc=0.008 * fs, taps=(1.0, 0.8),
152             tap_sym=1.5, seed=0)),
153                  ("qpsk/roll0.05", True, GenParams(mod="qpsk", fs=fs, baud=fs / 10, snr=18,
154                  rolloff=0.05, fc=0.008 * fs, seed=0)),
155                  ("16qam/roll0.08", True, GenParams(mod="16qam", fs=fs, baud=fs / 10, snr=24,
156                  rolloff=0.08, fc=0.008 * fs, seed=0)),
157                  ("qpsk/alias", True, GenParams(mod="qpsk", fs=fs, baud=fs / 10, snr=14,
158                  fc=1.3 * fs / 8, seed=0))]
159         if tier == "full":
160             for mod in MODS:
161                 cases.append((f"{mod}/lowsnr", False, GenParams(
162                     mod=mod, fs=fs, baud=fs / 10, snr=_SNR_LOW[mod], fc=0.008 * fs, seed=0)))
163             for roll in (0.1, 0.2, 0.5):
164                 cases.append((f"qpsk/roll{roll}", False, GenParams(
165                     mod="qpsk", fs=fs, baud=fs / 10, snr=18, rolloff=roll, fc=0.008 * fs, seed=0)))
166             cases += [("qpsk/bigcfo", False, GenParams(mod="qpsk", fs=fs, baud=fs / 10, snr=18,
167             fc=0.03 * fs, seed=0)),
168                      ("qpsk/drift", False, GenParams(mod="qpsk", fs=fs, baud=fs / 10, snr=18,
169                      fc=0.008 * fs, drift_ppm=100, seed=0)),
170                      ("16qam/sps4", False, GenParams(mod="16qam", fs=fs, baud=fs / 4, snr=24,
171                      fc=0.008 * fs, seed=0))]
172         return cases
173
174
175 def _sweep(args: argparse.Namespace) -> int:
176     cases = _grid(args.tier, args.seeds)
177     fails, tally = [], {"CORE": [0, 0], "STRETCH": [0, 0]} # tier -> [pass, total]
178     for label, core, p in cases:
179         x, bits = generate(p)
180         try:

```

```

181         r = analyze(x, Meta(p.fs, p.fmt, p.dtype)) # scoring demaps separately
182         ok, msg = _score(r, p, bits, core)
183     except Exception as exc: # a crash is a failure, not an abort
184         ok, msg = False, f"CRASH {exc}"
185     tier = "CORE" if core else "STRETCH"
186     tally[tier][0] += ok
187     tally[tier][1] += 1
188     print(f"[{'PASS' if ok else 'FAIL'}] {tier:<7} {label:<16} {msg}")
189     if core and not ok:
190         fails.append(label)
191     for tier, (good, n) in tally.items():
192         if n:
193             print(f"{tier}: {good}/{n} pass")
194     if fails:
195         print(f"CORE failures: {' ', '.join(fails)}")
196     return 1
197 return 0
198
199
200 # --- analyze / gen / serve -----
201
202 def _analyze(args: argparse.Namespace) -> int:
203     r = analyze_file(args.file, args.fs, args.fmt, args.dtype,
204                     "be" if args.be else None, args.bitrev or None, args.diff,
205                     None if args.burst is None else args.burst - 1, rf=args.rf)
206     d = r.to_json()["detected"]
207     truth = (sidecar_read(args.file) or {}).get("truth") # display only, never detection
208     if len(r.bursts) > 1:
209         print(f"버스트 {len(r.bursts)}개 감지 - {r.burst_idx + 1}번째 분석 (--burst N 으로 선택)")
210     print(f"모드      {d['mod']} + (f"      (정답 {truth['mod']})" if truth else "")
211     print(f"반송파      {d['fc']:.1f} Hz" + (f"      (정답 {truth['fc']:.1f})" if truth else ""))
212     if d["rf_hz"] is not None:
213         print(f"실제 RF      {d['rf_hz'] / 1e6:.6f} MHz")
214     print(f"심볼레이트 {d['baud']:.1f} Hz" + (f"      (정답 {truth['baud']:.1f})" if truth else ""))
215     fsk = r.family == "fsk"
216     tail = f"변조지수 h {d['h']:.2f}" if fsk else f"롤오프      {d['rolloff']:.2f}"
217     mer = "" if fsk else f"MER {r.mer_db:.1f} dB"
218     print(f"{tail} lock {r.lock:.1f}{mer}")
219     if r.eq_applied:
220         print("등화기 적용: 다중경로 ISI 보정됨"
221             + (" (T/2 분수간격)" if r.eq_mode == "fse" else " (심볼간격)"))
222     if r.alias_resolved:
223         print("반송파 앨리어스 보정: 스펙트럼 중심으로 후보 선택")
224     if r.baud_fallback:
225         print("심볼레이트 폴백: 점유대역폭 사전정보로 재탐색")
226     if r.carrier_ambiguous:
227         print("경고: 반송파 앨리어싱 가능 (|sym*fc| > 0.4*fs)")
228     if args.save_iq:
229         r.save_iq(args.save_iq)
230     if args.save_symbols:
231         r.save_symbols(args.save_symbols)
232     if args.save_bits:
233         r.save_bits(args.save_bits, args.packed)
234     if args.report:
235         r.save_report(args.report)
236     return 0
237
238
239 def _fhz(v: float) -> str:
240     a = abs(v)

```

```

241     if a >= 1e6:
242         return f"{v / 1e6:+.3f} MHz"
243     if a >= 1e3:
244         return f"{v / 1e3:+.2f} kHz"
245     return f"{v:+.0f} Hz"
246
247
248 def _survey(args: argparse.Namespace) -> int:
249     """Wideband survey: detect every emitter, demodulate the digital ones."""
250     s = survey_file(args.file, args.fs, args.fmt, args.dtype,
251                     "be" if args.be else None, args.bitrev or None, args.diff, rf=args.rf)
252     rf0 = s.meta.rf_center
253     note = f" · RF 중심 {rf0 / 1e6:.3f} MHz" if rf0 is not None else ""
254     print(f"{len(s.emitters)}개 신호 감지 (샘플레이트 {s.meta.fs:.0f} Hz{note})")
255     print(f"{'#':>2} {'실제 RF' if rf0 is not None else '중심주파수':>12} {'대역폭':>10}"
256           f" {'분류':>7} {'변조':>7} {'심볼레이트':>11} {'lock':>5}")
257     for i, e in enumerate(s.emitters):
258         r = e.result
259         mod = r.mod if r else "-"
260         baud = f"{r.baud:.0f}" if r else "-"
261         lock = f"{r.lock:.0f}" if r else "-"
262         kind = {"linear": "디지털", "fsk": "FSK", "analog": "아날로그",
263               "tone": "순수톤", "tooshort": "너무짧음", "error": "오류"}.get(e.kind, e.kind)
264         fc = e.abs_fc if rf0 is None else rf0 + e.abs_fc
265         print(f"{'i':>2} {'_fhz(fc):>12} {'_fhz(e.detection.bw):>10} {'kind:>7}"
266               f" {'mod:>7} {'baud:>11} {'lock:>5}")
267     if args.report:
268         import json
269         with open(args.report, "w") as fh:
270             json.dump(s.to_json(), fh, indent=1)
271     return 0
272
273
274 def _gen(args: argparse.Namespace) -> int:
275     if args.fmt == "real" and args.cfo <= args.baud * (1 + args.rolloff) / 2:
276         print("real 포맷은 통과대역 반송파가 필요합니다: --cfo > baud*(1+rolloff)/2")
277         return 1
278     taps = tuple(complex(t) for t in args.taps.split(",")) if args.taps else ()
279     p = GenParams(mod=args.mod, n_symbols=args.n, fs=args.fs, baud=args.baud,
280                  rolloff=args.rolloff, fc=args.cfo, phase=args.phase, timing=args.timing,
281                  snr=args.snr, fmt=args.fmt, dtype=args.dtype,
282                  endian="be" if args.be else "le", bitrev=args.bitrev, seed=args.seed,
283                  pad=args.pad, drift_ppm=args.drift_ppm, dc=complex(args.dc),
284                  h=args.h, taps=taps)
285     print(save(p, args.out, args.label))
286     return 0
287
288
289 def _dataset(args: argparse.Namespace) -> int:
290     """Curated variation matrix: every modulation + sample-type/endian/bitrev
291     variants + impairment cases, all with ground-truth sidecars."""
292     fs, out, made = 1e6, args.out, []
293     for mod in GEN_MODS: # one canonical file per modulation
294         fc = 0.0 if family(mod) == "fsk" else 0.008 * fs
295         h = 0.7 if mod == "fsk2" else 0.5 # fsk2 at h=0.5 IS msk; pick a distinct index
296         made.append(save(GenParams(mod=mod, fs=fs, baud=fs / 10, snr=22, fc=fc, h=h,
297                                   phase=0.6, timing=0.3, seed=1), out, mod))
298     # sample-type variants: label must stay unique (endian/bitrev alone would collide)
299     for i, (dt, be, br) in enumerate(("i8", 0, 0), ("u8", 0, 0), ("u16", 0, 0),
300                                     ("f32", 0, 0), ("f64", 0, 0), ("i16", 1, 0),

```

```

301         ("i16", 0, 1), ("u8", 1, 1))) :
302     made.append(save(GenParams(mod="qpsk", fs=fs, baud=fs / 10, snr=18,
303         fc=0.008 * fs, dtype=dt, endian="be" if be else "le",
304         bitrev=bool(br), seed=2), out, f"fmtvar{i}"))
305     for mod in ("bpsk", "qpsk", "16qam"): # real passband .pcm
306         made.append(save(GenParams(mod=mod, fs=fs, baud=fs / 10, snr=18, fc=0.1 * fs,
307         fmt="real", seed=3), out, f"pcm-{mod}"))
308     impair = [{"dc", {"dc": 0.3 + 0.3j}}, ("pad", {"pad": 0.5}),
309         ("drift", {"drift_ppm": 100}), ("lowsnr", {"snr": 8}),
310         ("taps", {"taps": (1.0, 0.45 * np.exp(1j * 0.8), 0.2j)}))] # 1-symbol echoes
311     for label, kw in impair:
312         made.append(save(GenParams(mod="qpsk", fs=fs, baud=fs / 10, snr=kw.pop("snr", 18),
313         fc=0.008 * fs, seed=4, **kw), out, label))
314     made.append(save(GenParams(mod="fsk4", fs=fs, baud=fs / 10, snr=20, h=1.0, seed=5),
315         out, "fsk4h1"))
316     if len(set(made)) != len(made):
317         print("경고: 파일명 충돌 - 라벨이 겹칩니다")
318     return 1
319     print(f"{len(made)} files -> {out}/ (각 파일에 <이름>.json 정답 사이드카)")
320     return 0
321
322
323 def _read_args(p: argparse.ArgumentParser) -> None:
324     """Read-side sample-format overrides, shared by analyze/survey (sidecar/filename win)."""
325     p.add_argument("--fs", type=float)
326     p.add_argument("--fmt", choices=("iq", "real"))
327     p.add_argument("--dtype", choices=_DTYPE_CHOICES)
328     p.add_argument("--be", action="store_true", help="big-endian 샘플")
329     p.add_argument("--bitrev", action="store_true", help="바이트 내 비트 역순")
330     p.add_argument("--rf", type=float, help="RF 중심주파수 (Hz) - 실제 주파수로 보고")
331
332
333 def main(argv: list[str] | None = None) -> int:
334     ap = argparse.ArgumentParser(prog="signus")
335     sub = ap.add_subparsers(dest="cmd", required=True)
336
337     a = sub.add_parser("analyze", help="블라인드 복조 + 제원 탐지")
338     a.add_argument("file")
339     _read_args(a)
340     a.add_argument("--save-iq")
341     a.add_argument("--save-symbols")
342     a.add_argument("--save-bits")
343     a.add_argument("--packed", action="store_true")
344     a.add_argument("--report")
345     a.add_argument("--diff", action="store_true",
346         help="차동 디맵(D-BPSK/D-QPSK): 회전 모호성 없이 비트 복원")
347     a.add_argument("--burst", type=int, help="분석할 버스트 번호 (1부터; 기본 최강 버스트)")
348
349     sv = sub.add_parser("survey", help="광대역 캡처의 모든 신호 탐지 + 복조")
350     sv.add_argument("file")
351     _read_args(sv)
352     sv.add_argument("--diff", action="store_true", help="차동 디맵")
353     sv.add_argument("--report", help="JSON 리포트 저장 경로")
354
355     g = sub.add_parser("gen", help="합성 신호 + 정답 사이드카 생성")
356     g.add_argument("--mod", default="qpsk", choices=GEN_MODS)
357     g.add_argument("--fs", type=float, default=1e6)
358     g.add_argument("--baud", type=float, default=1e5)
359     g.add_argument("--snr", type=float, default=20)
360     g.add_argument("--rolloff", type=float, default=0.35)

```

```

361     g.add_argument("--cfo", type=float, default=0.0)
362     g.add_argument("--phase", type=float, default=0.0)
363     g.add_argument("--timing", type=float, default=0.0)
364     g.add_argument("--pad", type=float, default=0.0)
365     g.add_argument("--drift-ppm", type=float, default=0.0)
366     g.add_argument("--dc", default="0")
367     g.add_argument("--h", type=float, default=0.5, help="FSK 변조지수")
368     g.add_argument("--taps", default="", help="멀티패스 탭, 예: 1,0,0.35j")
369     g.add_argument("--fmt", default="iq", choices=("iq", "real"))
370     g.add_argument("--dtype", default="i16", choices=_DTYPE_CHOICES)
371     g.add_argument("--be", action="store_true")
372     g.add_argument("--bitrev", action="store_true")
373     g.add_argument("--n", type=int, default=6000)
374     g.add_argument("--seed", type=int, default=0)
375     g.add_argument("--out", default="samples")
376     g.add_argument("--label", default="sig")
377
378     d = sub.add_parser("dataset", help="변조x샘플타입x장래 변형 매트릭스 일괄 생성")
379     d.add_argument("--out", default="samples")
380
381     w = sub.add_parser("sweep", help="전수조사: 그리드 생성->복조->채점")
382     w.add_argument("--tier", default="core", choices=("core", "full"))
383     w.add_argument("--seeds", type=int, default=3)
384
385     s = sub.add_parser("serve", help="웹 UI 서버")
386     s.add_argument("--host", default="127.0.0.1")
387     s.add_argument("--port", type=int, default=8000)
388
389     args = ap.parse_args(argv)
390     if args.cmd == "analyze":
391         ...     return _analyze(args)
392     if args.cmd == "survey":
393         ...     return _survey(args)
394     if args.cmd == "gen":
395         ...     return _gen(args)
396     if args.cmd == "dataset":
397         ...     return _dataset(args)
398     if args.cmd == "sweep":
399         ...     return _sweep(args)
400     from .server import run
401     run(args.host, args.port)
402     return 0
403
404
405 if __name__ == "__main__":
406     sys.exit(main())

```

```

1  """Stdlib-only web server: static frontend + POST /api/analyze (raw body,
2  filename + optional fs/fmt/dtype overrides in the query string – no multipart)."""
3
4  import json
5  from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
6  from pathlib import Path
7  from urllib.parse import parse_qs, urlparse
8
9  from .pipeline import analyze, survey_web
10 from .sigio import Meta, decode, parse_name
11
12 _WEB = Path(__file__).resolve().parent / "web" # inside the package so wheels/sdists ship it
13 _MIME = {"html": "text/html", "css": "text/css", "js": "text/javascript",
14         ".json": "application/json", ".png": "image/png", ".svg": "image/svg+xml"}
15 _MAX_BODY = 256 * 1024 * 1024
16
17
18 class Handler(BaseHTTPRequestHandler):
19     def log_message(self, *_: object) -> None:
20         pass
21
22     def _json(self, code: int, obj: dict) -> None:
23         body = json.dumps(obj).encode()
24         self.send_response(code)
25         self.send_header("Content-Type", "application/json")
26         self.send_header("Content-Length", str(len(body)))
27         self.end_headers()
28         self.wfile.write(body)
29
30     def _meta(self, q: dict) -> Meta:
31         """Build Meta from query params with filename fallback; raise on unknown."""
32         name = q.get("name", [""])[0]
33         m = parse_name(name)
34         meta = Meta(float(q["fs"][0]) if "fs" in q else m.fs,
35                    q.get("fmt", [m.fmt])[0], q.get("dtype", [m.dtype])[0],
36                    q.get("endian", [m.endian])[0],
37                    q.get("bitrev", ["1" if m.bitrev else "0"])[0] == "1",
38                    rf_center=float(q["rf"][0]) if "rf" in q else m.rf_center)
39         if not meta.ok():
40             raise ValueError(f"샘플레이트/포맷을 알 수 없습니다: {name}")
41         return meta
42
43     def do_POST(self) -> None: # noqa: N802 (BaseHTTPRequestHandler API)
44         url = urlparse(self.path)
45         if url.path not in ("/api/analyze", "/api/survey"):
46             self._json(404, {"error": "not found"})
47             return
48         q = parse_qs(url.query)
49         n = int(self.headers.get("Content-Length") or 0)
50         if n <= 0:
51             self._json(411, {"error": "Content-Length 필요"})
52             return
53         if n > _MAX_BODY:
54             self._json(413, {"error": "파일이 너무 큼니다 (최대 256MB)"})
55             return
56         try:
57             meta = self._meta(q)
58             burst = int(q["burst"][0]) if "burst" in q else None
59             x = decode(self.rfile.read(n), meta)
60             if x.size < 256:

```

```

61         raise ValueError("신호가 너무 짧습니다")
62     except ValueError as exc:
63         self._json(400, {"error": str(exc)})
64     return
65     try:
66         out = survey_web(x, meta) if url.path == "/api/survey" \
67             else analyze(x, meta, burst=burst).to_json()
68         self._json(200, out)
69     except Exception as exc: # surface DSP failures to the UI
70         self._json(500, {"error": f"분석 실패: {exc}"})
71
72     def do_GET(self) -> None: # noqa: N802
73         rel = urlparse(self.path).path.lstrip("/") or "index.html"
74         target = (_WEB / rel).resolve()
75         if not (target.is_file() and target.is_relative_to(_WEB.resolve())):
76             self._json(404, {"error": "not found"})
77         return
78         body = target.read_bytes()
79         self.send_response(200)
80         self.send_header("Content-Type", _MIME.get(target.suffix, "application/octet-stream"))
81         self.send_header("Content-Length", str(len(body)))
82         self.end_headers()
83         self.wfile.write(body)
84
85
86     def run(host: str = "127.0.0.1", port: int = 8000) -> None:
87         srv = ThreadingHTTPServer((host, port), Handler)
88         print(f"signus UI -> http://{host}:{srv.server_address[1]}")
89         srv.serve_forever()

```

```

1  <!doctype html>
2  <html lang="ko">
3  <head>
4  <meta charset="utf-8">
5  <meta name="viewport" content="width=device-width, initial-scale=1">
6  <title>Signus - 신호 복조 분석기</title>
7  <link rel="icon" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 1
4    6 16'%3E%3Crect width='16' height='16' rx='5' fill='%233182F6'/%3E%3C/svg%3E">
8  <link rel="stylesheet" href="/style.css">
9  <script defer src="/app.js"></script>
10 </head>
11 <body>
12 <header class="topbar">
13   <div class="brand">
14     <span class="logo" aria-hidden="true"></span>
15     <div>
16       <h1>Signus</h1>
17       <p class="tagline">블라인드 PSK/QAM/FSK 신호 복조 분석기</p>
18     </div>
19   </div>
20 </header>
21
22 <main class="wrap">
23   <section id="dropCard" class="card drop" tabindex="0" role="button"
24     ..... aria-label="신호 파일 선택 또는 드래그">
25     <input id="fileInput" type="file" multiple hidden>
26     <div class="drop-icon" aria-hidden="true"></div>
27     <p class="drop-title">신호 파일을 여기에 놓으세요</p>
28     <p class="drop-sub">여러 개를 놓으면 일괄 분석 · <code>.json</code> 정답 · <code>.sigmf-meta</co
4       de> 지원</p>
29   </section>
30
31   <section id="metaForm" class="card form hidden">
32     <p class="form-title">파일 이름에서 정보를 찾지 못했어요. 직접 입력해주세요.</p>
33     <div class="form-row">
34       <label>샘플레이트 (Hz)</label>
35       <input id="mFs" type="number" min="1" step="any" placeholder="예: 1000000"></label>
36       <label>포맷</label>
37       <select id="mFmt">
38         ..... <option value="">선택</option><option value="iq">IQ</option><option value="real">Real</opt
4         ion>
39       </select></label>
40       <label>데이터 타입</label>
41       <select id="mDtype">
42         ..... <option value="i16">i16 (16t)</option><option value="i8">i8 (8t)</option>
43         ..... <option value="u8">u8 (8o)</option><option value="u16">u16 (16o)</option>
44         ..... <option value="f32">f32</option><option value="f64">f64</option>
45       </select></label>
46     </div>
47     <button id="metaGo" class="btn">분석 시작</button>
48   </section>
49
50   <section id="loading" class="card loading hidden" aria-live="polite">
51     <div class="spinner" aria-hidden="true"></div>
52     <p id="loadMsg">신호를 분석하고 있어요...</p>
53   </section>
54
55   <section id="errorCard" class="card error hidden" role="alert">
56     <div class="err-mark" aria-hidden="true">!</div>
57     <div>

```

```

58     <p class="err-title">분석에 실패했어요</p>
59     <p id="errMsg" class="err-msg"></p>
60 </div>
61 </section>
62
63 <!-- Batch results table -->
64 <section id="batchCard" class="card hidden">
65     <div class="card-head">
66         <h2 class="card-title">일괄 분석 결과</h2>
67         <button id="dlCsv" class="btn ghost small">CSV 내보내기</button>
68     </div>
69     <div class="table-wrap">
70         <table class="batch">
71             <thead><tr><th>파일</th><th>변조</th><th>중심주파수</th><th>심볼레이트</th><th>Lock</th></tr>
72             <tbody id="batchBody"></tbody>
73         </table>
74     </div>
75     <p class="hint">행을 클릭하면 상세 화면으로 이동합니다.</p>
76 </section>
77
78 <!-- Wideband survey: overview waterfall with detected-emitter boxes -->
79 <section id="surveyCard" class="card hidden">
80     <div class="card-head">
81         <h2 class="card-title">광대역 Survey</h2>
82         <span id="survCount" class="pill ok"></span>
83     </div>
84     <div class="wf-wrap">
85         <canvas id="survFall" class="fall"></canvas>
86         <div id="survBoxes" class="boxes"></div>
87     </div>
88     <p class="hint">상자 또는 아래 목록을 클릭하면 그 신호의 상세로 이동합니다. (세로=시간, 가로=주
89     파수)</p>
90     <div id="survInfo" class="survinfo hidden"></div>
91     <div class="table-wrap">
92         <table class="batch">
93             <thead><tr><th>중심주파수</th><th>종류</th><th>변조</th><th>심볼레이트</th><th>Lock</th></tr>
94             <tbody id="survBody"></tbody>
95         </table>
96     </div>
97 </section>
98
99 <section id="results" class="results hidden">
100     <button id="backBatch" class="btn ghost small hidden">← 목록으로</button>
101
102     <div class="card summary">
103         <div class="summary-head">
104             <span id="statusPill" class="pill"></span>
105             <p id="fileName" class="file-name"></p>
106         </div>
107         <div class="summary-body">
108             <div class="gauge">
109                 <svg viewBox="0 0 120 120" class="donut" aria-hidden="true">
110                     <circle class="donut-track" cx="60" cy="60" r="52"></circle>
111                     <circle id="donutArc" class="donut-arc" cx="60" cy="60" r="52"></circle>
112                 </svg>
113                 <div class="gauge-center"><span id="lockNum" class="gauge-num">0</span>
114                 <span class="gauge-cap tip" title="별자리가 얼마나 조밀·정렬됐는지 (0~100). 60 이상이
115                 면 복조 신뢰">LOCK</span></div>

```

```

114     </div>
115     <div class="metrics">
116         <div class="metric"><span class="metric-label tip" title="변조 오차비 - 높을수록 깨끗 (dB)"
117             ">MER</span>
118             <span id="merVal" class="metric-val"></span></div>
119         <div class="metric"><span class="metric-label tip" title="오차 벡터 크기 - 낮을수록 좋음 (
120             %)">EVM</span>
121             <span id="evmVal" class="metric-val"></span></div>
122         <div class="metric"><span class="metric-label tip" title="추정 심볼 신호 대 잡음비 (dB)">S
123             NR</span>
124             <span id="snrVal" class="metric-val"></span></div>
125     </div>
126 </div>
127
128 <div id="burstMap" class="card hidden">
129     <h2 class="card-title">버스트 지도</h2>
130     <div class="wf-wrap">
131         <canvas id="burstFall" class="fall"></canvas>
132         <div id="burstBoxes" class="boxes"></div>
133     </div>
134     <p class="hint">전체 녹음의 시간-주파수. 버스트 구간을 클릭하면 그 구간을 분석합니다.</p>
135 </div>
136
137 <div class="card">
138     <h2 class="card-title">스펙트럼 · 워터폴</h2>
139     <canvas id="specCanvas" class="spec"></canvas>
140     <canvas id="fallCanvas" class="fall"></canvas>
141 </div>
142
143 <div class="card const-card">
144     <div class="card-head">
145         <h2 class="card-title">성상도</h2>
146         <div class="play-row">
147             <button id="playBtn" class="btn ghost small" aria-label="재생/일시정지">⏮ 일시정지</button>
148             <select id="speedSel" class="sel small" aria-label="재생 속도">
149                 <option value="1">1x</option><option value="4" selected>4x</option>
150                 <option value="16">16x</option>
151             </select>
152         </div>
153     </div>
154     <canvas id="constCanvas" class="const"></canvas>
155     <input id="scrub" class="scrub" type="range" min="0" max="1000" value="0"
156         aria-label="재생 위치">
157     <p id="playInfo" class="hint"></p>
158 </div>
159
160 <div id="paramGrid" class="param-grid"></div>
161
162 <div class="card downloads">
163     <h2 class="card-title">다운로드</h2>
164     <div class="dl-row">
165         <button id="copyBits" class="btn ghost">비트 복사</button>
166         <button id="dlBits" class="btn ghost">비트 .txt</button>
167         <button id="dlSyms" class="btn ghost">심볼 .csv</button>
168         <button id="dlReport" class="btn ghost">리포트 .json</button>
169     </div>

```

```
170     </div>
171   </section>
172 </main>
173 </body>
174 </html>
```

```

1  :root {
2    --bg: #f4f6f8; --card: #fff; --ink: #191f28; --sub: #6b7684;
3    --blue: #3182F6; --blue-weak: #e8f1fe; --line: #eef1f4;
4    --green: #12b886; --amber: #f59f00; --red: #fa5252;
5    --field: #fff; --hover: #f7fafd; --border: #dfe4ea; /* theme-sensitive surfaces */
6    --ok-bg: #e6f7f1; --warn-bg: #fff4e0; --bad-bg: #ffecec;
7    --radius: 20px; --shadow: 0 6px 24px rgba(30, 42, 66, .07);
8  }
9  /* dark mode: follows the OS. The signal canvases are already dark, so they blend in. */
10 @media (prefers-color-scheme: dark) {
11   :root {
12     --bg: #0e131b; --card: #161d28; --ink: #e6ebf2; --sub: #94a1b2;
13     --blue-weak: #17273c; --line: #232c39; --field: #1b2330; --hover: #1d2632;
14     --border: #2b3543; --ok-bg: #102c24; --warn-bg: #302512; --bad-bg: #321b1e;
15     --shadow: 0 6px 24px rgba(0, 0, 0, .38);
16   }
17 }
18
19 * { box-sizing: border-box; }
20
21 body {
22   margin: 0; background: var(--bg); color: var(--ink);
23   font-family: -apple-system, 'Apple SD Gothic Neo', 'Segoe UI', 'Malgun Gothic', sans-serif;
24   line-height: 1.5; -webkit-font-smoothing: antialiased;
25 }
26
27 .wrap { max-width: 580px; margin: 0 auto; padding: 8px 20px 64px; }
28
29 .topbar { max-width: 580px; margin: 0 auto; padding: 28px 20px 12px; }
30 .brand { display: flex; align-items: center; gap: 12px; }
31 .logo {
32   width: 40px; height: 40px; border-radius: 12px; flex: none;
33   background: linear-gradient(135deg, var(--blue), #6aa8ff);
34 }
35 h1 { margin: 0; font-size: 22px; font-weight: 800; letter-spacing: -.4px; }
36 .tagline { margin: 2px 0 0; color: var(--sub); font-size: 13px; }
37
38 .card {
39   background: var(--card); border-radius: var(--radius); padding: 20px;
40   box-shadow: var(--shadow); margin-top: 16px; animation: rise .35s ease both;
41 }
42 .card-title { margin: 0 0 12px; font-size: 15px; font-weight: 700; }
43 .hidden { display: none !important; }
44
45 @keyframes rise { from { opacity: 0; transform: translateY(10px); } to { opacity: 1; transform: none
46   ; } }
47
48 /* Dropzone */
49 .drop {
50   text-align: center; cursor: pointer; border: 2px dashed var(--border);
51   transition: border-color .18s, background .18s, transform .18s;
52 }
53 .drop:hover, .drop:focus-visible { border-color: var(--blue); background: var(--hover); outline: none; }
54 .drop.drag { border-color: var(--blue); background: var(--blue-weak); transform: scale(1.01); }
55 .drop-icon {
56   width: 52px; height: 52px; margin: 6px auto 14px; border-radius: 16px;
57   background: var(--blue-weak); position: relative;
58 }
59 .drop-icon::after {

```

```

59   content: ''; position: absolute; inset: 16px 15px; border: 3px solid var(--blue);
60   border-radius: 3px; border-top: none; border-right: none;
61   transform: rotate(45deg) translate(2px, -2px);
62 }
63 .drop-title { margin: 0; font-size: 16px; font-weight: 700; }
64 .drop-sub { margin: 6px 0 0; color: var(--sub); font-size: 13px; }
65 code { background: var(--line); padding: 1px 6px; border-radius: 6px; font-size: 12px; }
66
67 /* Meta form */
68 .form-title { margin: 0 0 12px; font-size: 14px; font-weight: 600; }
69 .form-row { display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 12px; }
70 .form-row label { display: flex; flex-direction: column; gap: 6px; font-size: 12px; color: var(--sub)
  4   ); }
71 input, select {
72   font: inherit; padding: 10px 12px; border: 1px solid var(--border); border-radius: 12px;
73   background: var(--field); color: var(--ink);
74 }
75 input:focus, select:focus { outline: 2px solid var(--blue); border-color: transparent; }
76
77 .btn {
78   margin-top: 16px; width: 100%; padding: 13px; border: none; border-radius: 14px;
79   background: var(--blue); color: #fff; font-size: 15px; font-weight: 700; cursor: pointer;
80   transition: filter .15s, transform .1s;
81 }
82 .btn:hover { filter: brightness(1.05); }
83 .btn:active { transform: scale(.98); }
84 .btn.ghost { background: var(--blue-weak); color: var(--blue); margin-top: 0; }
85
86 /* Loading */
87 .loading { display: flex; align-items: center; gap: 14px; color: var(--sub); }
88 .spinner {
89   width: 26px; height: 26px; border: 3px solid var(--line);
90   border-top-color: var(--blue); border-radius: 50%; animation: spin .8s linear infinite;
91 }
92 @keyframes spin { to { transform: rotate(360deg); } }
93
94 /* Error */
95 .error { display: flex; gap: 14px; align-items: center; border-left: 4px solid var(--red); }
96 .err-mark {
97   width: 34px; height: 34px; flex: none; border-radius: 50%; background: var(--bad-bg);
98   color: var(--red); font-weight: 800; display: grid; place-items: center;
99 }
100 .err-title { margin: 0; font-weight: 700; }
101 .err-msg { margin: 2px 0 0; color: var(--sub); font-size: 14px; }
102
103 /* Summary + gauge */
104 .summary-head { display: flex; align-items: center; gap: 12px; margin-bottom: 16px; }
105 .pill { padding: 6px 14px; border-radius: 999px; font-size: 13px; font-weight: 700; }
106 .pill.ok { background: var(--ok-bg); color: var(--green); }
107 .pill.warn { background: var(--warn-bg); color: var(--amber); }
108 .pill.bad { background: var(--bad-bg); color: var(--red); }
109 .file-name { margin: 0; font-size: 13px; color: var(--sub); overflow: hidden; text-overflow: ellipsi
  4   s; white-space: nowrap; }
110
111 .summary-body { display: flex; align-items: center; gap: 24px; }
112 .gauge { position: relative; width: 120px; height: 120px; flex: none; }
113 .donut { width: 120px; height: 120px; }
114 .donut-track { fill: none; stroke: var(--line); stroke-width: 12; }
115 .donut-arc {
116   fill: none; stroke: var(--blue); stroke-width: 12; stroke-linecap: round;

```

```

117   transform: rotate(-90deg); transform-origin: center;
118   transition: stroke-dashoffset .9s cubic-bezier(.22, .61, .36, 1);
119 }
120 .gauge-center { position: absolute; inset: 0; display: grid; place-items: center; }
121 .gauge-num { font-size: 34px; font-weight: 800; line-height: 1; font-variant-numeric: tabular-nums
122   ; }
123 .gauge-cap { font-size: 11px; color: var(--sub); letter-spacing: 1px; }
124
125 .metrics { display: flex; flex-direction: column; gap: 12px; }
126 .metric { display: flex; flex-direction: column; gap: 2px; }
127 .metric-label { font-size: 12px; color: var(--sub); }
128 .tip { cursor: help; text-decoration: underline dotted; text-underline-offset: 3px;
129   text-decoration-color: color-mix(in srgb, var(--sub) 45%, transparent); }
130 .metric-val { font-size: 20px; font-weight: 700; font-variant-numeric: tabular-nums; }
131
132 /* Card head with actions: spacing lives on the CONTAINER so the title and the
133   action stay vertically centered on the same line */
134 .card-head { display: flex; align-items: center; justify-content: space-between;
135   gap: 12px; margin-bottom: 12px; }
136 .card-head .card-title { margin-bottom: 0; }
137 .btn.small { width: auto; margin-top: 0; padding: 8px 14px; font-size: 13px; border-radius: 10px; }
138 .sel.small {
139   font: inherit; font-size: 13px; font-weight: 600; padding: 7px 10px; border-radius: 10px;
140   border: 1px solid var(--border); background: var(--field); color: var(--ink);
141 }
142 .play-row { display: flex; gap: 8px; align-items: center; }
143 .hint { margin: 8px 0 0; font-size: 12px; color: var(--sub); }
144
145 /* Spectrum + waterfall */
146 .spec { width: 100%; height: 120px; display: block; border-radius: 12px 12px 0 0; background: #0d132
147   0; }
148 .fall { width: 100%; height: 150px; display: block; border-radius: 0 0 12px 12px; background: #0d132
149   0; }
150
151 /* Constellation + playback */
152 .const { width: 100%; aspect-ratio: 1; border-radius: 14px; display: block; background: #0d1320; }
153 .scrub { width: 100%; margin-top: 12px; accent-color: var(--blue); }
154
155 /* Batch table */
156 .table-wrap { overflow-x: auto; }
157 table.batch { width: 100%; border-collapse: collapse; font-size: 13.5px; }
158 table.batch th {
159   text-align: left; font-size: 12px; color: var(--sub); font-weight: 600;
160   padding: 8px 10px; border-bottom: 1px solid var(--line);
161 }
162 table.batch td { padding: 11px 10px; border-bottom: 1px solid var(--line);
163   font-variant-numeric: tabular-nums; }
164 table.batch .pill { padding: 3px 10px; font-size: 12px; }
165 table.batch tbody tr { cursor: pointer; transition: background .12s; }
166 table.batch tbody tr:hover { background: var(--hover); }
167 table.batch .fn { font-weight: 600; max-width: 210px; overflow: hidden;
168   text-overflow: ellipsis; white-space: nowrap; }
169
170 /* Param grid */
171 .param-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin-top: 16px; }
172 .param { padding: 16px; margin: 0; animation: rise .4s ease both; }
173 .param-label { font-size: 12px; color: var(--sub); }
174 .param-val { font-size: 22px; font-weight: 800; margin-top: 4px; letter-spacing: -.3px;
175   font-variant-numeric: tabular-nums; }
176 .param-note { font-size: 11px; color: var(--sub); margin-top: 2px; }

```

```

174 .chips { display: flex; flex-wrap: wrap; gap: 6px; margin-top: 10px; }
175 .chips:empty { display: none; } /* no flags -> no ghost margin */
176 .chip { font-size: 11px; font-weight: 700; padding: 4px 9px; border-radius: 999px; }
177 .chip.hit { background: var(--ok-bg); color: var(--green); }
178 .chip.miss { background: var(--bad-bg); color: var(--red); }
179 .chip.warn { background: var(--warn-bg); color: var(--amber); }
180
181 /* Burst selector (chips that act) */
182 .chip-btn {
183   font: inherit; font-size: 11px; font-weight: 700; padding: 4px 10px;
184   border-radius: 999px; border: 1px solid var(--border); background: var(--field);
185   color: var(--sub); cursor: pointer; transition: color .12s, border-color .12s;
186 }
187 .chip-btn:hover { border-color: var(--blue); color: var(--blue); }
188 .chip-btn.on { background: var(--blue-weak); color: var(--blue); border-color: transparent; }
189
190 /* Downloads */
191 .dl-row { display: flex; gap: 10px; flex-wrap: wrap; }
192 .dl-row .btn { flex: 1; min-width: 120px; }
193
194 /* Survey overview: waterfall + detected-emitter boxes */
195 .wf-wrap { position: relative; }
196 .wf-wrap .fall { height: 230px; border-radius: 12px; }
197 .boxes { position: absolute; inset: 0; pointer-events: none; }
198 .box {
199   position: absolute; box-sizing: border-box; border: 2px solid var(--blue);
200   border-radius: 5px; background: rgba(49, 130, 246, .10); cursor: pointer;
201   pointer-events: auto; min-width: 7px; min-height: 12px;
202   transition: background .12s, box-shadow .12s;
203 }
204 .box:hover, .box:focus-visible {
205   background: rgba(49, 130, 246, .24); box-shadow: 0 0 0 2px rgba(255, 255, 255, .55); outline: none
206   ;
207 }
208 .box-lbl { /* hidden by default (a spectrogram full of labels is unreadable); shown on hover */
209   position: absolute; top: -17px; left: -2px; white-space: nowrap; font-size: 10px;
210   font-weight: 700; color: #fff; background: rgba(13, 19, 32, .85); padding: 1px 6px;
211   border-radius: 6px; pointer-events: none; display: none; z-index: 2;
212 }
213 .box:hover .box-lbl, .box:focus-visible .box-lbl, .box.sel .box-lbl { display: block; }
214 .box.sel { box-shadow: 0 0 0 2px #fff, 0 0 0 4px var(--blue); }
215 .box.ok { border-color: var(--green); background: rgba(18, 184, 134, .13); }
216 .box.ok .box-lbl { background: var(--green); }
217 .box.warn { border-color: var(--amber); background: rgba(245, 159, 0, .13); }
218 .box.warn .box-lbl { background: var(--amber); }
219 .box.bad { border-color: var(--red); background: rgba(250, 82, 82, .13); }
220 .box.bad .box-lbl { background: var(--red); }
221 .box.gray { border-color: #8592a6; background: rgba(133, 146, 166, .15); }
222 .box.gray .box-lbl { background: #8592a6; }
223 .pill.gray { background: var(--line); color: var(--sub); }
224 table.batch .fc { font-weight: 600; font-variant-numeric: tabular-nums; }
225
226 /* Non-digital emitter inline info */
227 .survinfo {
228   background: var(--hover); border-radius: 14px; padding: 14px 16px; margin: 12px 0;
229   border-left: 4px solid #8592a6; animation: rise .3s ease both;
230 }
231 .survinfo h3 { margin: 0 0 10px; font-size: 14px; }
232 .si-grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 10px 16px; }
233 .si-k { font-size: 11px; color: var(--sub); }

```

```
233 .si-v { font-size: 17px; font-weight: 700; font-variant-numeric: tabular-nums; }
234
235 @media (max-width: 420px) {
236   .form-row { grid-template-columns: 1fr; }
237   .param-grid { grid-template-columns: 1fr; }
238   .summary-body { gap: 18px; }
239 }
240
241 @media (prefers-reduced-motion: reduce) {
242   * { animation: none !important; transition: none !important; }
243 }
```

```

1  "use strict";
2  const $ = (id) => document.getElementById(id);
3  const API = "/api/analyze";
4  const SURVEY_API = "/api/survey";
5  const FS_RE = /(?:^|_)fs(\d+(?:\.\d+)?(?:e[+-]?\d+)?(?:_|$)/i;
6  const RF_RE = /(?:^|_)rf(\d+(?:\.\d+)?(?:e[+-]?\d+)?(?:_|$)/i;
7  const REDUCED = matchMedia("(prefers-reduced-motion: reduce)").matches;
8  const state = { file: null, sidecar: null, meta: null, resp: null, batch: [],
9  ...      burst: null, survey: null, drilled: false, overview: null };
10
11  /* --- filename parsing (mirror of sigio.parse_name; read-necessities only) --- */
12  const DYPES = ["i8", "u8", "i16", "u16", "f32", "f64"];
13  // baudline-style aliases: <bits>t = two's complement (signed), <bits>o = offset (unsigned)
14  const DTYPE_ALIAS = { "8t": "i8", "8o": "u8", "16t": "i16", "16o": "u16", "32f": "f32", "64f": "f64",
15  ...      };
16  function parseName(name) {
17  const stem = name.replace(/^.*/, "").replace(/\.|^.*$/, "").toLowerCase();
18  const mt = FS_RE.exec(stem), rt = RF_RE.exec(stem), toks = stem.split("_");
19  const dt = toks.find((t) => DYPES.includes(t) || t in DTYPE_ALIAS);
20  return {
21  fs: mt ? parseFloat(mt[1]) : null,
22  fmt: toks.find((t) => t === "iq" || t === "real") || null,
23  dtype: dt ? (DTYPE_ALIAS[dt] || dt) : "i16",
24  endian: toks.includes("be") ? "be" : "le",
25  bitrev: toks.includes("bitrev"),
26  rf: rt ? parseFloat(rt[1]) : null,
27  };
28  }
29  /* SigMF: core:datatype -> our fmt/dtype/endian */
30  const SIGMF = { cf32: ["iq", "f32"], ci16: ["iq", "i16"], ci8: ["iq", "i8"],
31  ...      cu8: ["iq", "u8"], rf32: ["real", "f32"], ri16: ["real", "i16"] };
32  function metaFromSigmf(doc) {
33  const g = doc && doc.global;
34  if (!g || !g["core:datatype"]) return null;
35  const dt = g["core:datatype"], base = dt.replace(/_[lb]e$/, "");
36  if (!(base in SIGMF)) return null;
37  const [fmt, dtype] = SIGMF[base];
38  const cap = (doc.captures || [])[0] || {};
39  return { fs: Number(g["core:sample_rate"]), fmt, dtype,
40  ...      endian: dt.endsWith("_be") ? "be" : "le", bitrev: false,
41  ...      rf: cap["core:frequency"] !== null ? Number(cap["core:frequency"]) : null };
42  }
43  /* --- number formatting --- */
44  function sig(x) {
45  const a = Math.abs(x);
46  return parseFloat(a >= 100 ? x.toFixed(0) : a >= 10 ? x.toFixed(1) : x.toFixed(2)).toString();
47  }
48  function fmtHz(v) {
49  if (v == null) return "-";
50  const a = Math.abs(v);
51  if (a >= 1e6) return sig(v / 1e6) + " MHz";
52  if (a >= 1e3) return sig(v / 1e3) + " kHz";
53  return sig(v) + " Hz";
54  }
55  function fmtRf(v) { // absolute RF: keep kHz resolution even in the 100s-of-MHz range
56  return (v / 1e6).toFixed(3) + " MHz";
57  }
58
59  /* --- ideal constellation points (mirror of constellations.ideal_points) --- */

```

```

60 function idealPoints(mod) {
61   if (mod === "bpsk") return [{ i: 1, q: 0 }, { i: -1, q: 0 }];
62   if (mod === "qpsk" || mod === "8psk") {
63     const m = mod === "qpsk" ? 4 : 8, off = mod === "qpsk" ? Math.PI / 4 : 0, p = [];
64     for (let k = 0; k < m; k++) p.push({ i: Math.cos(off + 2 * Math.PI * k / m), q: Math.sin(of
65       f + 2 * Math.PI * k / m) });
66     return p;
67   }
68   const n = mod === "16qam" ? 4 : mod === "32qam" ? 6 : 8, lv = [], p = [];
69   for (let k = 0; k < n; k++) lv.push(k * 2 - (n - 1));
70   for (const qi of lv) for (const ii of lv) {
71     if (mod === "32qam" && Math.abs(ii * qi) === 25) continue; // cross: corners removed
72     p.push({ i: ii, q: qi });
73   }
74   let e = 0;
75   for (const pt of p) e += pt.i * pt.i + pt.q * pt.q;
76   const norm = Math.sqrt(e / p.length);
77   return p.map((pt) => ({ i: pt.i / norm, q: pt.q / norm }));
78 }
79 /* --- file intake: one data file (+sidecar) or many (batch) --- */
80 async function acceptFiles(list) {
81   const files = [...list];
82   const metas = files.filter((f) => /\. (json|sigmf-meta)$/i.test(f.name));
83   const data = files.filter((f) => !metas.includes(f));
84   if (!data.length) return showError("데이터 파일을 찾을 수 없어요.");
85   if (data.length > 1) return runBatch(data, metas);
86
87   state.file = data[0];
88   state.resp = null;
89   state.batch = [];
90   state.burst = null;
91   state.survey = null;
92   state.drilled = false;
93   state.overview = null;
94   const sm = metas.find((m) => m.name.toLowerCase().endsWith(".sigmf-meta"));
95   const truth = metas.find((m) => m.name.toLowerCase().endsWith(".json"));
96   state.sidecar = truth ? await readJson(truth) : null; // client-side only, never sent
97   state.meta = (sm && metaFromSigmf(await readJson(sm))) || parseName(state.file.name);
98   if (state.meta && state.meta.fs && state.meta.fmt) runSurvey();
99   else showMetaForm();
100 }
101 const readJson = (f) => f.text().then(JSON.parse).catch(() => null);
102
103 function showMetaForm() {
104   hideAll();
105   state.meta = state.meta || {};
106   if (state.meta.fs) $("mFs").value = state.meta.fs;
107   $("mFmt").value = state.meta.fmt || "";
108   $("mDtype").value = state.meta.dtype || "i16";
109   show($("metaForm"));
110 }
111 $("metaGo").onclick = () => {
112   const fs = parseFloat($("mFs").value), fmt = $("mFmt").value;
113   if (!(fs > 0) || !fmt) return showError("샘플레이트와 포맷을 입력해주세요.");
114   state.meta = { fs, fmt, dtype: $("mDtype").value, endian: "le", bitrev: false };
115   runSurvey();
116 };
117
118 /* --- server call (contract-frozen) --- */

```

```

119 function query(file, m, burst) {
120   const q = new URLSearchParams({ name: file.name });
121   if (m.fs) q.set("fs", m.fs);
122   if (m.fmt) q.set("fmt", m.fmt);
123   if (m.dtype) q.set("dtype", m.dtype);
124   if (m.endian) q.set("endian", m.endian);
125   if (m.bitrev) q.set("bitrev", "1");
126   if (m.rf !== null) q.set("rf", m.rf);
127   if (burst !== null) q.set("burst", burst);
128   return q;
129 }
130 async function postTo(api, file, m, burst) {
131   const res = await fetch(api + "?" + query(file, m, burst), { method: "POST", body: file });
132   const data = await res.json();
133   if (!res.ok || data.error) throw new Error(data.error || `서버 오류 (${res.status})`);
134   return data;
135 }
136 async function runSurvey() { // entry: one capture -> single detail OR survey
137   hideAll();
138   state.drilled = false;
139   $("loadMsg").textContent = "신호를 조사하고 있어요...";
140   show($("loading"));
141   try {
142     const resp = await postTo(SURVEY_API, state.file, state.meta);
143     if (resp.mode === "single") {
144       state.survey = null; state.overview = resp.overview || null; state.resp = resp.result;
145       render(resp.result);
146       if (state.overview) renderBurstMap(state.overview, resp.result);
147     } else renderSurvey(resp);
148   } catch (e) {
149     showError(e.message);
150   }
151 }
152 async function analyze() { // burst re-selection within a single signal
153   hideAll();
154   $("loadMsg").textContent = "신호를 분석하고 있어요...";
155   show($("loading"));
156   try {
157     state.resp = await postTo(API, state.file, state.meta, state.burst);
158     render(state.resp);
159     if (state.overview) renderBurstMap(state.overview, state.resp);
160     $("backBatch").classList.toggle("hidden", !state.batch.length);
161   } catch (e) {
162     showError(e.message);
163   }
164 }
165
166 /* --- batch mode --- */
167 async function runBatch(data, metas) {
168   hideAll();
169   show($("loading"));
170   const truths = new Map(metas.filter((m) => /\.json$/i.test(m.name))
171     .map((m) => [m.name.replace(/\.json$/i, ""), m]));
172   const sigmfs = new Map(metas.filter((m) => /\.sigmf-meta$/i.test(m.name))
173     .map((m) => [m.name.replace(/\.sigmf-meta$/i, ""), m]));
174   const rows = [];
175   for (let i = 0; i < data.length; i++) {
176     const f = data[i];
177     $("loadMsg").textContent = `일괄 분석 중... (${i + 1}/${data.length}) ${f.name}`;
178     // same meta precedence as single-file mode: a .sigmf-meta sidecar (matched by stem)

```

```

179 // beats filename tokens
180 const sm = sigmfs.get(f.name.replace(/\.[^.]*/, ""));
181 const meta = (sm && metaFromSigmf(await readJson(sm))) || parseName(f.name);
182 let resp = null, err = null;
183 try {
184     if (!meta.fs || !meta.fmt) throw new Error("파일명에 fs/포맷 정보가 없어요");
185     resp = await postTo(API, f, meta);
186 } catch (e) { err = e.message; }
187 const tf = truths.get(f.name);
188 // meta is stored per row: a later burst-chip re-analyze posts with THIS file's meta --
189 // it used to read state.meta, which a fresh batch never set (crash) and a previous
190 // single-file run left stale (silent wrong-fs decode)
191 rows.push({ file: f, meta, resp, err, sidecar: tf ? await readJson(tf) : null });
192 }
193 state.batch = rows;
194 hideAll();
195 renderBatch(rows);
196 }
197
198 function renderBatch(rows) {
199     $("batchBody").innerHTML = rows.map((r, i) => {
200         if (r.err) return `<tr data-i="${i}"><td class="fn">${r.file.name}</td>` +
201             `<td colspan="4" style="color:var(--red)">${r.err}</td></tr>`;
202         const d = r.resp.detected, lock = Math.round(r.resp.quality.lock);
203         const cls = lock >= 60 ? "ok" : lock >= 40 ? "warn" : "bad";
204         return `<tr data-i="${i}"><td class="fn">${r.file.name}</td>` +
205             `<td>${d.mod.toUpperCase()}</td><td>${fmtHz(d.fc)}</td><td>${fmtHz(d.baud)}</td>` +
206             `<td><span class="pill ${cls}">${lock}</span></td></tr>`;
207     }).join("");
208     $("batchBody").querySelectorAll("tr").forEach((tr) => {
209         tr.onclick = () => {
210             const r = state.batch[+tr.dataset.i];
211             if (!r.resp) return;
212             state.file = r.file; state.meta = r.meta; state.resp = r.resp; state.sidecar = r.sidecar;
213             state.burst = null; state.drilled = false; state.survey = null; state.overview = null;
214             hideAll();
215             render(r.resp);
216             const b = $("backBatch");
217             b.textContent = "← 목록으로";
218             b.onclick = () => { hideAll(); renderBatch(state.batch); };
219             b.classList.remove("hidden");
220         };
221     });
222     show($("batchCard"));
223 }
224 $("backBatch").onclick = () => { hideAll(); renderBatch(state.batch); };
225 $("dlCsv").onclick = () => {
226     const head = "file,mod,fc_hz,baud_hz,rolloff,lock,mer_db,snr_est_db\n";
227     const body = state.batch.filter((r) => r.resp).map((r) => {
228         const d = r.resp.detected, q = r.resp.quality;
229         return [r.file.name, d.mod, d.fc, d.baud, d.rolloff ?? "", q.lock, q.mer_db,
230             r.resp.snr_est_db ?? ""];
231     }).join("\n");
232     saveBlob("signus_batch.csv", head + body, "text/csv");
233 };
234
235 /* --- wideband survey overview (waterfall + detected-emitter boxes) --- */
236 function renderSurvey(resp) {
237     hideAll();
238     stopPlay();

```

```

239 state.survey = resp;
240 const dig = resp.emitters.filter((e) => e.result).length;
241 $("survCount").textContent = `${resp.emitters.length}개 신호 · 디지털 ${dig}`;
242 $("survInfo").classList.add("hidden");
243 renderSurveyList(resp);
244 show($("#surveyCard")); // show first so the canvas has a real width
245 paintWaterfall($("#survFall"), resp.overview.waterfall, 230);
246 drawBoxes(resp);
247 }
248
249 function boxStyle(e) { // -> [css class, short label]
250   if (e.result) {
251     const lock = Math.round(e.result.quality.lock);
252     return [lock >= 60 ? "ok" : lock >= 40 ? "warn" : "bad",
253           `${e.result.detected.mod.toUpperCase()} · ${lock}`];
254   }
255   if (e.kind === "chirp") {
256     return ["gray", e.info && e.info.sf ? `LoRa SF${e.info.sf}` : "처프"];
257   }
258   return ["gray", { analog: "아날로그", tone: "톤/CW", tooshort: "짧음",
259                   error: "오류" }[e.kind] || e.kind];
260 }
261
262 function drawBoxes(resp) {
263   const host = $("#survBoxes"), fs = resp.overview.fs, n = resp.overview.n || 1;
264   host.innerHTML = "";
265   resp.emitters.forEach((e) => {
266     const d = e.det, [cls, lbl] = boxStyle(e);
267     const wpct = Math.max(0.8, d.bw / fs * 100); // width = bandwidth / fs
268     const left = (d.fc + fs / 2) / fs * 100 - wpct / 2; // x = carrier on -fs/2..fs/2
269     const top = Math.max(0, d.t0 / n * 100); // y = burst start over capture
270     const b = document.createElement("button");
271     b.className = "box " + cls;
272     b.style.left = Math.max(0, Math.min(100 - wpct, left)) + "%";
273     b.style.width = wpct + "%";
274     b.style.top = top + "%";
275     b.style.height = Math.min(100 - top, Math.max(7, (d.t1 - d.t0) / n * 100)) + "%";
276     b.title = lbl;
277     b.innerHTML = `<span class="box-lbl">${lbl}</span>`;
278     b.onclick = () => drillEmitter(e);
279     host.appendChild(b);
280   });
281 }
282
283 function renderSurveyList(resp) {
284   const KIND = { linear: "디지털", fsk: "FSK", chirp: "처프/LoRa", analog: "아날로그",
285                 tone: "톤/CW", tooshort: "짧음", error: "오류" };
286   const rf0 = resp.rf_center; // real RF = capture centre + abs_fc
287   $("survBody").innerHTML = resp.emitters.map((e, i) => {
288     const [cls] = boxStyle(e), r = e.result;
289     const lock = r ? `<span class="pill ${cls}">${Math.round(r.quality.lock)}</span>`
290               : `<span class="pill gray"></span>`;
291     return `<tr data-i="${i}"><td class="fc">${rf0 != null ? fmtRf(rf0 + e.abs_fc) : fmtHz(e.abs_fc)}
292           </td>` +
293           `<td>${KIND[e.kind] || e.kind}</td><td>${r ? r.detected.mod.toUpperCase() : "-"}</td>` +
294           `<td>${r ? fmtHz(r.detected.baud) : "-"}</td><td>${lock}</td></tr>`;
295   }).join("");
296   $("survBody").querySelectorAll("tr").forEach((tr) => {
297     tr.onclick = () => drillEmitter(resp.emitters[+tr.dataset.i]);
298   });

```

```

298 }
299
300 function drillEmitter(e) {
301   if (e.result) { // digital -> reuse the single-signal detail view
302     state.drilled = true;
303     state.overview = null; // a cut channel has no whole-capture burst map
304     state.resp = e.result;
305     render(e.result);
306     const rf0 = state.survey && state.survey.rf_center;
307     $("fileName").textContent =
308       `에미터 @ ${rf0 != null ? fmtRf(rf0 + e.abs_fc) : fmtHz(e.abs_fc)} · ${state.file.name}`;
309     const b = $("backBatch");
310     b.textContent = "← Survey로";
311     b.onclick = backToSurvey;
312     b.classList.remove("hidden");
313   } else { // non-digital -> inline info, stay on the overview
314     showSurveyInfo(e);
315   }
316 }
317 function backToSurvey() { state.drilled = false; renderSurvey(state.survey); }
318
319 function showSurveyInfo(e) {
320   const d = e.det;
321   let title, rows;
322   if (e.kind === "chirp" && e.info) { // linear chirp / CSS (LoRa): characterized only
323     const c = e.info;
324     title = (c.sf ? `LoRa 계열 (SF${c.sf})` : "선형 처프 / CSS") + " - 복조 대상 아님";
325     // rate + direction are robust (coherent from the beat tone); bw/SF only when the
326     // channel is cleanly isolated enough to snap to a LoRa hypothesis.
327     rows = [
328       ["중심주파수", fmtHz(e.abs_fc)],
329       ["처프 방향", c.up ? "▲ 상승" : "▼ 하강"],
330       ["처프율", (c.mu / 1e9).toFixed(3) + " MHz/ms"];
331     if (c.sf) rows.push(
332       ["대역폭", fmtHz(c.bw)], ["심볼레이트", fmtHz(c.rs)],
333       ["심볼시간", (c.tsym * 1e3).toFixed(2) + " ms"]);
334   } else {
335     const KIND = { analog: "아날로그 (FM/음성)", tone: "톤 / CW",
336       tooshort: "너무 짧은 버스트", error: "분석 오류" };
337     title = (KIND[e.kind] || e.kind) + " - 복조 대상 아님";
338     rows = [
339       ["중심주파수", fmtHz(e.abs_fc)], ["대역폭", fmtHz(d.bw)],
340       ["SNR", d.snr_db == null ? "-" : d.snr_db.toFixed(1) + " dB"],
341       ["심볼레이트(추정)", fmtHz(d.baud_hint)];
342   }
343   $("survInfo").innerHTML = `<h3>${title}</h3>` +
344     `<div class="si-grid">` + rows.map(([k, v]) =>
345       `<div><div class="si-k">${k}</div><div class="si-v">${v}</div></div>`).join("") + "</div>";
346   $("survInfo").classList.remove("hidden");
347   $("survInfo").scrollIntoView({ behavior: REDUCED ? "auto" : "smooth", block: "nearest" });
348 }
349
350 /* --- burst map: whole-record waterfall with clickable time-burst boxes (single signal) --- */
351 function renderBurstMap(ov, result) {
352   show($("burstMap"));
353   paintWaterfall($("burstFall"), ov.waterfall, 150);
354   const host = $("burstBoxes"), n = ov.n || 1;
355   host.innerHTML = "";
356   (result.bursts || []).forEach((b, i) => {
357     const top = Math.max(0, b.start / n * 100);
358     const dur = (b.end - b.start) / ov.fs;
359     const lbl = `버스트 ${i + 1} · ${dur >= 1 ? dur.toFixed(2) + " s" : (dur * 1e3).toFixed(0) + " m
360       s"}`;

```

```

357     const box = document.createElement("button"); // full width (one signal), spans t0..t1 in time
358     box.className = "box" + (i === result.burst_idx ? " sel" : "");
359     box.style.left = "0%"; box.style.width = "100%"; box.style.top = top + "%";
360     box.style.height = Math.min(100 - top, Math.max(5, (b.end - b.start) / n * 100)) + "%";
361     box.title = lbl;
362     box.innerHTML = `${lbl}`;
363     box.onclick = () => { state.burst = i; analyze(); }; // re-analyse that burst; map persists
364     host.appendChild(box);
365   });
366 }
367
368 /* --- rendering --- */
369 function statusOf(lock) {
370   if (lock >= 60) return ["복조 성공", "ok"];
371   if (lock >= 40) return ["부분 복조", "warn"];
372   return ["복조 실패", "bad"];
373 }
374 function render(d) {
375   hideAll();
376   show($("#results"));
377   $("#burstMap").classList.add("hidden"); // re-shown by renderBurstMap only in multi-burst single mode
378   const lock = d.quality.lock, [txt, cls] = statusOf(lock);
379   const pill = $("#statusPill");
380   pill.textContent = txt;
381   pill.className = "pill " + cls;
382   $("#fileName").textContent =
383     `${state.file.name} · ${fmtHz(d.fs)} · ${d.fmt === "real" ? "Real PCM" : "IQ"}`;
384   $("#merVal").textContent = d.quality.mer_db == null ? "-" : d.quality.mer_db.toFixed(1) + " dB";
385   $("#evmVal").textContent = d.quality.evm == null ? "-" : (d.quality.evm * 100).toFixed(1) + "%";
386   $("#snrVal").textContent = snrText(d);
387
388   const flags = [];
389   if (d.family === "fsk") flags.push('<span class="chip warn">FSK 계열 · 주파수 판별</span>');
390   if (d.eq && d.eq.applied) flags.push('<span class="chip hit">등화기 적용 · ${
391     d.eq.mode === "fse" ? "T/2 분수간격" : "심볼간격"></span>');
392   if (d.detected.alias_resolved) flags.push('<span class="chip warn">반송파 앨리어스 보정</span>');
393   if (d.detected.baud_fallback) flags.push('<span class="chip warn">심볼레이트 대역폭 폴백</span>');
394   if (d.detected.carrier_ambiguous) flags.push('<span class="chip warn">반송파 모호 · 앨리어싱 가능<
395     /span>');
396   $("#flagRow").innerHTML = flags.join("");
397
398   renderBursts(d);
399   animateGauge(lock, cls);
400   renderParams(d);
401   drawSpectrum(d);
402   drawWaterfall(d);
403   startPlay(d);
404 }
405 function snrText(d) {
406   const s = d.snr_est_db;
407   if (s == null || d.quality.lock < 30) return "-";
408   return (s > 28 ? "≥28" : s.toFixed(1)) + " dB";
409 }
410 function renderBursts(d) {
411   const row = $("#burstRow"), bs = d.bursts || [];
412   // a survey-drilled emitter is already a cut channel: burst re-selection would
413   // re-post the whole capture, so it is disabled while drilled. When the burst MAP
414   // is shown (multi-burst single signal) the map replaces the chips.

```

```

415 if (state.drilled || state.overview || bs.length < 2) { row.innerHTML = ""; return; }
416 const dur = (b) => {
417   const sec = (b.end - b.start) / d.fs;
418   return sec >= 1 ? sec.toFixed(2) + " s" : (sec * 1e3).toFixed(1) + " ms";
419 };
420 row.innerHTML = bs.map((b, i) =>
421   `<button class="chip-btn${i === d.burst_idx ? " on" : ""}" data-i="${i}">` +
422   `버스트 ${i + 1} · ${dur(b)}</button>`).join("");
423 row.querySelectorAll("button").forEach((el) => {
424   el.onclick = () => { state.burst = +el.dataset.i; analyze(); };
425 });
426 }
427
428 function chip(hit, truthTxt) {
429   return `<span class="chip ${hit ? "hit" : "miss"}">정답 ${truthTxt} · ${hit ? "일치" : "불일치"}</span>`;
430 }
431
432 function renderParams(d) {
433   const det = d.detected, t = state.sidecar && state.sidecar.truth;
434   const near = (a, b, tol) => Math.abs(a - b) <= tol;
435   const rows = [
436     ["중심주파수", det.rf_hz != null ? fmtRf(det.rf_hz) : fmtHz(det.fc),
437       t && t.fc != null && chip(near(det.fc, t.fc, Math.max(300, 0.02 * t.baud)), fmtHz(t.fc)),
438       det.rf_hz != null ? `기저대역 ${fmtHz(det.fc)}` : ""],
439     ["심볼레이트", fmtHz(det.baud), t && t.baud != null && chip(near(det.baud, t.baud, 0.03 * t.baud),
440       fmtHz(t.baud)),
441       det.baud_conf ? `스펙트럼 라인 강도 ×${Math.round(det.baud_conf)}` : ""],
442     ["변조방식", det.mod.toUpperCase(), t && t.mod != null && chip(det.mod === t.mod, t.mod.toUpperCase(),
443       t.mod.toUpperCase()),
444       det.symmetry ? `대칭 ${det.symmetry}` : "주파수 레벨"],
445   ];
446   if (d.family === "fsk") {
447     rows.push(["변조지수 h", det.h.toFixed(2),
448       t && t.h != null && chip(near(det.h, t.h, 0.15), t.h.toFixed(2)), ""]);
449   } else {
450     rows.push(["롤오프", det.rolloff.toFixed(2),
451       t && t.rolloff != null && chip(near(det.rolloff, t.rolloff, 0.11), t.rolloff.toFixed(2)), ""]);
452   }
453   $("paramGrid").innerHTML = rows.map(([label, val, hit, note]) => `
454     <div class="card param">
455       <div class="param-label">${label}</div>
456       <div class="param-val">${val}</div>
457       ${note ? `<div class="param-note">${note}</div>` : ""}
458       ${hit ? `<div class="chips">${hit}</div>` : ""}
459     </div>`).join("");
460 }
461
462 function animateGauge(lock, cls) {
463   const arc = $("donutArc"), num = $("lockNum"), C = 2 * Math.PI * 52;
464   arc.style.stroke = { ok: "#12b886", warn: "#f59f00", bad: "#fa5252" }[cls];
465   arc.style.strokeDasharray = C;
466   const target = C * (1 - Math.max(0, Math.min(100, lock)) / 100);
467   if (REDUCED) { arc.style.strokeDashoffset = target; num.textContent = Math.round(lock); return; }
468   arc.style.strokeDashoffset = C;
469   requestAnimationFrame(() => { arc.style.strokeDashoffset = target; });
470   const t0 = performance.now();
471   const step = (t) => {
472     const k = Math.min(1, (t - t0) / 900);
473     num.textContent = Math.round(lock * (1 - Math.pow(1 - k, 3)));
474   };
475 }

```

```

471     if (k < 1) requestAnimationFrame(step);
472   };
473   requestAnimationFrame(step);
474 }
475
476 /* --- canvas helpers --- */
477 function fit(cv, hCss) {
478   const dpr = window.devicePixelRatio || 1, w = cv.clientWidth || 320;
479   const h = hCss || w;
480   cv.width = w * dpr; cv.height = h * dpr;
481   const g = cv.getContext("2d");
482   g.setTransform(dpr, 0, 0, dpr, 0, 0);
483   return [g, w, h];
484 }
485 function pct(a, p) {
486   const s = [...a].sort((x, y) => x - y);
487   return s[Math.min(s.length - 1, Math.max(0, Math.round(p * (s.length - 1))))];
488 }
489
490 /* --- spectrum --- */
491 function drawSpectrum(d) {
492   const [g, w, h] = fit($("#specCanvas"), 120);
493   g.fillStyle = "#0d1320"; g.fillRect(0, 0, w, h);
494   if (!d.spectrum) return;
495   const f = d.spectrum.f, db = d.spectrum.db;
496   const lo = pct(db, 0.02), hi = pct(db, 1) + 2;
497   const fmin = f[0], fmax = f[f.length - 1];
498   const X = (v) => (v - fmin) / (fmax - fmin) * w;
499   const Y = (v) => h - (Math.max(lo, Math.min(hi, v)) - lo) / (hi - lo) * (h - 8) - 4;
500   const det = d.detected, half = det.baud / 2e3;
501   g.strokeStyle = "rgba(120,180,255,.35)"; g.setLineDash([4, 4]);
502   for (const gf of [det.fc / 1e3 - half, det.fc / 1e3 + half]) {
503     g.beginPath(); g.moveTo(X(gf), 0); g.lineTo(X(gf), h); g.stroke();
504   }
505   g.setLineDash([]);
506   g.strokeStyle = "rgba(255,255,255,.45)";
507   g.beginPath(); g.moveTo(X(det.fc / 1e3), 0); g.lineTo(X(det.fc / 1e3), h); g.stroke();
508   g.strokeStyle = "#5aa9ff"; g.lineWidth = 1.4; g.beginPath();
509   f.forEach((v, k) => (k ? g.lineTo(X(v), Y(db[k])) : g.moveTo(X(v), Y(db[k]))));
510   g.stroke();
511   g.fillStyle = "rgba(255,255,255,.5)"; g.font = "10px sans-serif";
512   g.fillText(sig(fmin) + " kHz", 4, h - 4);
513   const tmax = sig(fmax) + " kHz";
514   g.fillText(tmax, w - g.measureText(tmax).width - 4, h - 4);
515 }
516
517 /* --- waterfall --- */
518 function drawWaterfall(d) { paintWaterfall($("#fallCanvas"), d.waterfall, 150); }
519 function paintWaterfall(canvas, wf, hCss) {
520   const [g, w, h] = fit(canvas, hCss);
521   g.fillStyle = "#0d1320"; g.fillRect(0, 0, w, h);
522   if (!wf || !wf.rows) return;
523   const lo = pct(wf.db, 0.05), hi = pct(wf.db, 0.95);
524   const img = g.createImageData(wf.bins, wf.rows);
525   for (let k = 0; k < wf.db.length; k++) {
526     const t = Math.max(0, Math.min(1, (wf.db[k] - lo) / (hi - lo + 1e-9)));
527     const o = k * 4; // dark -> blue -> white ramp
528     img.data[o] = t < 0.5 ? 13 + t * 120 : 73 + (t - 0.5) * 364;
529     img.data[o + 1] = t < 0.5 ? 19 + t * 260 : 149 + (t - 0.5) * 212;
530     img.data[o + 2] = t < 0.5 ? 32 + t * 446 : 255;

```

```

531     img.data[o + 3] = 255;
532   }
533   createImageBitmap(img).then((bmp) => { g.imageSmoothingEnabled = true; g.drawImage(bmp, 0, 0, w, h
534     ); });
535 }
536
537 /* --- constellation playback (the headline) --- */
538 const play = { raf: 0, i: 0, on: false, data: null };
539 let CG = null; // cached constellation geometry
540
541 function stopPlay() { cancelAnimationFrame(play.raf); play.raf = 0; play.on = false; }
542 function setPlayBtn(on) { $("playBtn").textContent = on ? "⏏ 일시정지" : "▶ 재생"; }
543
544 function startPlay(d) {
545   stopPlay();
546   play.data = d;
547   play.i = 0;
548   const n = d.constellation.i.length;
549   $("scrub").max = String(Math.max(1, n));
550   $("playInfo").textContent = `${n.toLocaleString()} 심볼 · 시간 순서로 재생 (잔광 효과)`;
551   drawConstBase(d);
552   if (REDUCED) { // static full view, no motion
553     drawPoints(d, 0, n);
554     $("scrub").value = String(n);
555     setPlayBtn(false);
556     return;
557   }
558   play.on = true;
559   setPlayBtn(true);
560   play.raf = requestAnimationFrame(loop);
561 }
562
563 $("playBtn").onclick = () => {
564   if (!play.data) return;
565   if (play.on) { stopPlay(); setPlayBtn(false); }
566   else { play.on = true; setPlayBtn(true); play.raf = requestAnimationFrame(loop); }
567 };
568
569 $("scrub").oninput = () => {
570   if (!play.data) return;
571   stopPlay(); setPlayBtn(false);
572   play.i = +$("scrub").value;
573   drawConstBase(play.data);
574   drawPoints(play.data, 0, play.i); // redraw history up to the scrub point
575 };
576
577 function loop() {
578   const d = play.data, n = d.constellation.i.length;
579   const step = Math.max(1, Math.round(n / 600) * (+$("speedSel").value));
580   const to = Math.min(n, play.i + step);
581   fade(); // phosphor persistence
582   drawPoints(d, play.i, to);
583   play.i = to >= n ? 0 : to;
584   if (play.i === 0) drawConstBase(d); // loop: clear the trail
585   $("scrub").value = String(play.i);
586   if (play.on) play.raf = requestAnimationFrame(loop);
587 }
588
589 function drawConstBase(d) {
590   const [g, w] = fit($("constCanvas"));
591   const fsk = d.family === "fsk";
592   const I = d.constellation.i, Q = d.constellation.q;

```

```

590 const ideal = fsk ? [] : idealPoints(d.detected.mod);
591 let lim = 0;
592 for (let k = 0; k < I.length; k++) lim = Math.max(lim, Math.abs(I[k]), Math.abs(Q[k]));
593 for (const p of ideal) lim = Math.max(lim, Math.abs(p.i), Math.abs(p.q));
594 lim = (lim || 1) * 1.12;
595 const R = w / 2 * 0.9;
596 CG = { g, w, lim, R, map: (re, im) => [w / 2 + re / lim * R, w / 2 - im / lim * R] };
597
598 g.fillStyle = "#0d1320"; g.fillRect(0, 0, w, w);
599 g.strokeStyle = "rgba(255,255,255,.06)"; g.lineWidth = 1;
600 for (let f = -1; f <= 1; f += 0.5) {
601     const [gx] = CG.map(f, 0), [, gy] = CG.map(0, f);
602     g.beginPath(); g.moveTo(gx, 0); g.lineTo(gx, w); g.stroke();
603     g.beginPath(); g.moveTo(0, gy); g.lineTo(w, gy); g.stroke();
604 }
605 if (fsk) { // frequency levels as dashed bands
606     g.strokeStyle = "rgba(255,255,255,.5)"; g.setLineDash([6, 5]);
607     for (const lv of [...new Set(I.map((v) => Math.round(v)))].filter((v) => Math.abs(v) <= 8)) {
608         const [, yy] = CG.map(0, lv);
609         g.beginPath(); g.moveTo(0, yy); g.lineTo(w, yy); g.stroke();
610     }
611     g.setLineDash([]);
612 } else {
613     g.strokeStyle = "rgba(255,255,255,.75)"; g.lineWidth = 1.4;
614     for (const p of ideal) {
615         const [x, y] = CG.map(p.i, p.q);
616         g.beginPath(); g.arc(x, y, 6, 0, 7); g.stroke();
617     }
618 }
619 }
620 function fade() {
621     if (!CG) return;
622     CG.g.fillStyle = "rgba(13,19,32,.14)"; // translucent wash = phosphor decay
623     CG.g.fillRect(0, 0, CG.w, CG.w);
624 }
625 function drawPoints(d, from, to) {
626     if (!CG || to <= from) return;
627     const g = CG.g, I = d.constellation.i, Q = d.constellation.q;
628     const fsk = d.family === "fsk";
629     const a = Math.max(0.08, Math.min(0.55, 150 / (to - from)));
630     g.globalCompositeOperation = "lighter";
631     g.fillStyle = `rgba(90,170,255,${a})`;
632     for (let k = from; k < to; k++) {
633         // FSK has no I/Q plane: spread levels vertically, jitter horizontally for density
634         const [x, y] = fsk ? CG.map((k % 89) / 89 - 0.5, I[k]) : CG.map(I[k], Q[k]);
635         g.beginPath(); g.arc(x, y, 1.8, 0, 7); g.fill();
636     }
637     g.globalCompositeOperation = "source-over";
638 }
639
640 /* --- downloads --- */
641 function saveBlob(name, text, type) {
642     const url = URL.createObjectURL(new Blob([text], { type }));
643     const a = document.createElement("a");
644     a.href = url; a.download = name; a.click();
645     URL.revokeObjectURL(url);
646 }
647 const stem = () => state.file.name.replace(/\.[^.]*/, "");
648 $("copyBits").onclick = async (e) => { // decoded bitstream -> clipboard, with brief feedback
649     const btn = e.currentTarget, was = btn.textContent;

```

```

650   try { await navigator.clipboard.writeText(state.resp.bits); btn.textContent = "복사됨 ✓"; }
651   catch { btn.textContent = "복사 실패"; }
652   setTimeout(() => { btn.textContent = was; }, 1400);
653 };
654 $("#dlBits").onclick = () => saveBlob(stem() + ".bits.txt", state.resp.bits, "text/plain");
655 $("#dlSyms").onclick = () => {
656   const c = state.resp.constellation;
657   saveBlob(stem() + ".symbols.csv", "i,q\n" + c.i.map((v, k) => `${v},${c.q[k]}`).join("\n"), "text/
658   csv");
659 };
660 $("#dlReport").onclick = () =>
661   saveBlob(stem() + ".report.json", JSON.stringify(state.resp, null, 2), "application/json");
662
663 /* --- view helpers --- */
664 function show(el) { el.classList.remove("hidden"); }
665 function hideAll() {
666   stopPlay();
667   ["metaForm", "loading", "errorCard", "results", "batchCard", "surveyCard"]
668   .forEach((id) => $(id).classList.add("hidden"));
669   $("#backBatch").classList.add("hidden");
670 }
671 function showError(msg) { hideAll(); $("#errMsg").textContent = msg; show($("#errorCard")); }
672
673 /* --- dropzone wiring --- */
674 const drop = $("#dropCard"), input = $("#fileInput");
675 // reset value first: re-picking the SAME file fires no change event otherwise
676 const pick = () => { input.value = ""; input.click(); };
677 drop.onclick = pick;
678 drop.onkeydown = (e) => { if (e.key === "Enter" || e.key === " ") { e.preventDefault(); pick(); } };
679 input.onchange = () => { if (input.files.length) acceptFiles(input.files); };
680 drop.addEventListener("dragover", (e) => { e.preventDefault(); drop.classList.add("drag"); });
681 drop.addEventListener("dragleave", () => drop.classList.remove("drag"));
682 drop.addEventListener("drop", (e) => {
683   e.preventDefault(); drop.classList.remove("drag");
684   if (e.dataTransfer.files.length) acceptFiles(e.dataTransfer.files);
685 });

```